Luanti-Modding-Buch

Luanti-Modding-Buch

Eigene Mods und Spiele für Luanti entwickeln

rubenwardy

Lektorat: Shara

Luanti-Modding-Buch
Originaltitel: *Luanti Modding Book*

Text: rubenwardy, `https://rubenwardy.com`

Quelle: `https://gitlab.com/rubenwardy/minetest_modding_book`
Stand der Quelle: Commit b3904f8 (23. 10. 2026)

Dieses Werk ist lizenziert unter der Creative-Commons-Lizenz „Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International“ (CC BY-SA 4.0):
`https://creativecommons.org/licenses/by-sa/4.0/deed.de`

Die deutsche Übersetzung wurde für diese Ausgabe aus dem englischen Original erstellt. Die Bezeichnung „Minetest“ wurde durchgehend auf „Luanti“, der API-Namensraum `minetest` auf `core` aktualisiert.

Gesetzt aus der IBM Plex mit LaTeX und KOMA-Script.

Inhaltsverzeichnis

Einleitung **xiii**
Feedback und Beiträge xiii

I Grundlagen 1

1 Erste Schritte **3**
1.1 Was sind Spiele und Mods? 3
1.2 Wo werden die Mods gespeichert? 3
1.3 Deinen ersten Mod erstellen 4
1.3.1 Mod-Verzeichnis 4
1.3.2 mod.conf 5
1.3.3 init.lua 5
1.3.4 Zusammenfassung 5
1.4 Abhängigkeiten 6
1.5 Mod-Pakete 6

2 Lua-Skripting **7**
2.1 Programmierung 7
2.1.1 Programmieren in Lua 7
2.2 Code-Editoren 7
2.3 Lokaler und globaler Gültigkeitsbereich 8
2.3.1 Möglichst lokale Variablen verwenden 9
2.4 Andere Lua-Skripte einbinden 11

II Items 13

3 Nodes, Items und Craften **15**
3.1 Was sind Nodes und Items? 15
3.2 Items registrieren 15
3.2.1 Itemnamen 16
3.2.2 Item-Aliase 16
3.2.3 Texturen 17
3.3 Einen einfachen Node registrieren 17
3.4 Crafting 18
3.4.1 Shaped 18

3.4.2 Shapeless . . . 19
3.4.3 Cooking und Fuel . . . 20
3.5 Gruppen . . . 21
3.6 Werkzeuge, Fähigkeiten und Abbautypen . . . 21

4 Node- und Item-Callbacks 25
4.1 Item-Callbacks . . . 25
4.1.1 on_use . . . 25
4.1.2 on_place und on_secondary_use . . . 26
4.1.3 on_drop . . . 26
4.1.4 after_use . . . 26
4.2 item_place oder place_item? . . . 27
4.3 Node-Callbacks . . . 27
4.3.1 Rechtsklick und Platzieren . . . 28
4.3.2 Schlagen und Abbauen . . . 29
4.3.3 ... und mehr! . . . 29

5 Texturen erstellen 31
5.1 Techniken . . . 31
5.1.1 Mit dem Stift arbeiten . . . 31
5.1.2 Kacheln . . . 31
5.1.3 Transparenz . . . 31
5.1.4 Farbpaletten . . . 32
5.2 Editoren . . . 32
5.2.1 MS Paint . . . 32
5.2.2 Aseprite / LibreSprite . . . 32
5.2.3 GIMP . . . 32

6 Zeichnungstypen von Nodes 33
6.1 Würfelförmige Nodes: normal und allfaces . . . 33
6.2 Glasartige Nodes . . . 34
6.2.1 glasslike_framed . . . 35
6.3 Luftartige Nodes . . . 36
6.4 Beleuchtung und Ausbreitung des Sonnenlichts . . . 36
6.5 Flüssige Nodes . . . 37
6.6 Nodeboxen . . . 39
6.6.1 Nodeboxen für Wandmontage . . . 40
6.7 Mesh-Nodes . . . 41
6.8 Signlike-Nodes . . . 42
6.9 Plantlike-Nodes . . . 43
6.10 Firelike-Nodes . . . 43
6.11 Weitere Zeichnungstypen . . . 44

7 ItemStacks und Inventare **45**
7.1 Was sind ItemStacks und Inventare? . . . 45
7.2 ItemStacks . . . 45
7.3 Inventarorte . . . 46
7.3.1 Node-Inventare . . . 46
7.3.2 Spielerinventare . . . 47
7.3.3 Freistehende Inventare . . . 47
7.4 Listen . . . 49
7.4.1 Größe und Breite . . . 49
7.4.2 Inhalt prüfen . . . 50
7.5 Inventare und ItemStacks verändern . . . 50
7.5.1 Zu einer Liste hinzufügen . . . 50
7.5.2 Items herausnehmen . . . 50
7.5.3 Stacks verändern . . . 51
7.6 Abnutzung . . . 52
7.7 Lua-Tabellen . . . 52

III Karte **55**

8 Grundlegende Kartenoperationen **57**
8.1 Aufbau der Karte . . . 57
8.2 Lesen . . . 57
8.2.1 Nodes lesen . . . 57
8.2.2 Nodes finden . . . 58
8.3 Schreiben . . . 60
8.3.1 Nodes schreiben . . . 60
8.3.2 Nodes entfernen . . . 60
8.4 Mapblöcke laden . . . 61
8.5 Mapblöcke löschen . . . 62

9 Node-Timer und ABMs **63**
9.1 Node-Timer . . . 63
9.2 Active Block Modifiers . . . 64
9.3 Jetzt bist du dran . . . 65

10 Speicherung und Metadaten **67**
10.1 Metadaten . . . 67
10.1.1 Was sind Metadaten? . . . 67
10.1.2 Ein Metadaten-Objekt holen . . . 67
10.1.3 Lesen und Schreiben . . . 68
10.1.4 Besondere Schlüssel . . . 68
10.1.5 Tabellen speichern . . . 69
10.1.6 Private Metadaten . . . 69

10.1.7 Lua-Tabellen 69
10.2 Mod-Speicher 69
10.3 Datenbanken 70
10.4 Was soll ich verwenden? 71
10.5 Jetzt bist du dran 72

11 Objekte, Spieler und Entities **73**
11.1 Was sind Objekte, Spieler und Entities? 73
11.2 Position und Geschwindigkeit 73
11.3 Objekteigenschaften 74
11.4 Entities 74
11.5 Lebenspunkte und Schaden 77
11.5.1 Lebenspunkte (HP) 77
11.5.2 Schläge, Schadensgruppen und Rüstungsgruppen . 78
11.5.3 Beispiel für eine Schadensberechnung 78
11.6 Anhängen 80
11.7 Jetzt bist du dran 80

IV Spieler **81**

12 Privilegien **83**
12.1 Wann sind Privilegien sinnvoll? 83
12.2 Privilegien deklarieren 83
12.3 Privilegien prüfen 84
12.4 Privilegien lesen und setzen 85
12.5 Privilegien zu basic_privs hinzufügen 85

13 Chat und Befehle **87**
13.1 Nachrichten senden 87
13.1.1 An alle Spieler 87
13.1.2 An bestimmte Spieler 87
13.2 Chatbefehle 88
13.2.1 Mehrere Argumente 89
13.3 Nachrichten abfangen 90

14 Spielerphysik **93**
14.1 Ein einfaches Beispiel 93
14.2 Verfügbare Overrides 93
14.2.1 Altes Bewegungsverhalten 94
14.3 Unverträglichkeit zwischen Mods 94
14.4 Jetzt bist du dran 94

15 GUIs (Formspecs) 95
15.1 Echte oder alte Koordinaten . . . 95
15.2 Aufbau einer Formspec . . . 96
15.2.1 Elemente . . . 96
15.2.2 Kopf . . . 96
15.3 Ein Ratespiel . . . 97
15.3.1 Innenabstand und Zwischenraum . . . 99
15.3.2 Eingaben empfangen . . . 99
15.3.3 Kontexte . . . 100
15.4 Wege zur Formspec . . . 102
15.4.1 Formspecs in Node-Metadaten . . . 102
15.4.2 Formspecs im Spielerinventar . . . 103
15.4.3 Jetzt bist du dran . . . 103

16 HUD 105
16.1 Positionierung . . . 105
16.1.1 Position und Versatz . . . 105
16.1.2 Ausrichtung . . . 106
16.1.3 Punkteanzeige . . . 106
16.2 Textelemente . . . 107
16.2.1 Parameter . . . 107
16.2.2 Unser Beispiel . . . 107
16.3 Bildelemente . . . 109
16.3.1 Parameter . . . 109
16.3.2 Skalierung . . . 109
16.4 Ein Element ändern . . . 110
16.5 IDs speichern . . . 111
16.6 Weitere Elemente . . . 112

V Fortgeschrittene Kartenbearbeitung 113

17 Biome und Dekorationen 115
17.1 Was sind Biome? . . . 115
17.2 Platzierung von Biomen . . . 115
17.2.1 Wärme und Feuchtigkeit . . . 115
17.2.2 Grenzen mit Voronoi-Diagrammen veranschaulichen 116
17.2.3 Voronoi-Diagramme mit GeoGebra erstellen . . . 117
17.3 Ein Biom registrieren . . . 118
17.4 Was sind Dekorationen? . . . 118
17.5 Eine einfache Dekoration registrieren . . . 119
17.6 Eine Schematic-Dekoration registrieren . . . 119
17.7 Mapgen-Aliase . . . 120

18 Lua Voxel Manipulators **121**
18.1 Konzept . . . 121
18.2 Daten in den LVM lesen . . . 121
18.3 Nodes lesen . . . 122
18.4 Nodes schreiben . . . 123
18.5 Beispiel . . . 124
18.6 Jetzt bist du dran . . . 125

VI Spiele **127**

19 Spiele erstellen **129**
19.1 Was ist ein Spiel? . . . 129
19.2 Spielverzeichnis . . . 129
19.3 Kompatibilität zwischen Spielen . . . 130
19.3.1 API-Kompatibilität . . . 130
19.3.2 Gruppen und Aliase . . . 131
19.4 Jetzt bist du dran . . . 131

VII Qualität **133**

20 Übersetzungen **135**
20.1 Wie funktioniert die Übersetzung auf dem Client? . . . 135
20.1.1 Markierter Text . . . 135
20.1.2 Übersetzungsdateien . . . 136
20.2 Platzhalter . . . 138
20.3 Gute Praxis und verbreitete Irrtümer . . . 138
20.4 Übersetzung auf dem Server . . . 139
20.5 Fazit . . . 139

21 Häufige Fehler **141**
21.1 Vorsicht beim Speichern von ObjectRefs . . . 141
21.2 Formspec-Eingaben nicht vertrauen . . . 141
21.3 ItemStacks nach dem Ändern zurücksetzen . . . 143

22 Automatische Fehlersuche mit LuaCheck **145**
22.1 LuaCheck installieren . . . 145
22.1.1 Windows . . . 145
22.1.2 Linux . . . 145
22.2 LuaCheck ausführen . . . 146
22.3 LuaCheck einrichten . . . 146
22.3.1 Fehlerbehebung . . . 147
22.4 Im Editor verwenden . . . 147

23 Sicherheit **149**
23.1 Grundprinzipien . . . 149
23.2 Formspecs . . . 149
23.2.1 Eingaben niemals vertrauen . . . 149
23.2.2 Zeitpunkt der Prüfung ist nicht Zeitpunkt der Verwendung . . . 150
23.3 (Unsichere) Umgebungen . . . 150

24 Saubere Architektur **153**
24.1 Kohäsion, Kopplung und Trennung der Belange . . . 153
24.2 Beobachter . . . 154
24.3 Model-View-Controller . . . 155
24.3.1 API-View . . . 158
24.4 Fazit . . . 158

25 Automatische Unit-Tests **159**
25.1 Busted installieren . . . 159
25.2 Dein erster Test . . . 159
25.2.1 init.lua . . . 160
25.2.2 api.lua . . . 160
25.2.3 tests/api_spec.lua . . . 160
25.3 Mocking: Externe Funktionen ersetzen . . . 161
25.4 Fazit . . . 163

26 Einen Mod veröffentlichen **165**
26.1 Eine Lizenz wählen . . . 165
26.1.1 LGPL und CC BY-SA . . . 165
26.1.2 CC0 . . . 166
26.1.3 MIT . . . 166
26.2 Verpacken . . . 166
26.2.1 README.txt . . . 166
26.2.2 mod.conf / game.conf . . . 167
26.2.3 screenshot.png . . . 167
26.3 Hochladen . . . 167
26.3.1 Versionsverwaltung . . . 168
26.4 Auf ContentDB veröffentlichen . . . 168
26.5 Forenthema . . . 168

27 Weiterlesen **171**
27.1 Modding für Luanti . . . 171
27.2 Programmieren in Lua . . . 171
27.3 3D-Modellierung . . . 171

Stichwortverzeichnis **173**

Einleitung

Luanti verwendet Lua-Skripte, um Modding zu ermöglichen. Dieses Buch soll dir, beginnend mit den Grundlagen, beibringen, wie du deine eigenen Mods erstellst. Jedes Kapitel konzentriert sich auf einen bestimmten Teil der API und bringt dich schnell dahin, eigene Mods zu schreiben.

Das Buch ist auch online lesbar[1] und als HTML herunterladbar[2].

Feedback und Beiträge

Hast du einen Fehler bemerkt oder möchtest du Feedback geben? Dann teile es bitte mit:

- Erstelle ein Issue auf GitLab[3].
- Schreib im Forenthema[4] oder im deutschen Forenthema[5].
- Nimm Kontakt mit dem Autor auf[6].
- Lust mitzumachen? Lies die README[7].

[1] https://rubenwardy.com/minetest_modding_book/de/
[2] https://gitlab.com/rubenwardy/minetest_modding_book/-/releases
[3] https://gitlab.com/rubenwardy/minetest_modding_book/-/issues
[4] https://forum.luanti.org/viewtopic.php?f=14&t=10729
[5] https://forum.luanti.org/viewtopic.php?t=26723
[6] https://rubenwardy.com/contact/
[7] https://gitlab.com/rubenwardy/minetest_modding_book/-/blob/master/README.md

Teil I

Grundlagen

1 Erste Schritte

Den grundlegenden Aufbau eines Mod-Verzeichnisses zu verstehen, ist beim Erstellen von Mods unerlässlich. In diesem Kapitel erfährst du, wie Modding in Luanti funktioniert, und erstellst deinen ersten Mod.

1.1 Was sind Spiele und Mods?

Die Stärke von Luanti liegt darin, dass sich Spiele einfach entwickeln lassen, ohne eigene Voxel-Grafik, Voxel-Algorithmen oder ausgefeilten Netzwerkcode schreiben zu müssen.

In Luanti ist ein Spiel eine Sammlung von Modulen, die zusammenarbeiten, um den Inhalt und das Verhalten des Spiels bereitzustellen. Ein Modul, allgemein als Mod bezeichnet, ist eine Sammlung von Skripten und Ressourcen. Es ist möglich, ein Spiel mit nur einem Mod zu erstellen, aber das wird selten gemacht, weil sich dann Teile des Spiels nicht mehr so einfach unabhängig voneinander anpassen oder ersetzen lassen.

Ebenso ist es möglich, Mods außerhalb eines Spiels zu verbreiten. Dann sind sie ebenfalls *Mods*, aber im traditionelleren Sinn: *Modifikationen*. Diese Mods verändern oder erweitern die Eigenschaften eines Spiels.

Sowohl die Mods, die in einem Spiel enthalten sind, als auch Mods von Dritten nutzen dieselbe API (Programmierschnittstelle).

Dieses Buch behandelt die wichtigsten Teile der Luanti-API und richtet sich gleichermaßen an Spieleentwickler und Modder.

1.2 Wo werden die Mods gespeichert?

Jeder Mod hat sein eigenes Verzeichnis, in dem sich seine Lua-Quelltexte, Texturen, Modelle und Tondateien befinden. Luanti sucht an verschiedenen Orten nach Mods. Diese Orte werden allgemein *Mod-Ladepfade* genannt.

Für eine bestimmte Welt werden drei Mod-Speicherorte geprüft, und zwar in dieser Reihenfolge:

1. Spiel-Mods. Das sind die Mods des Spiels, in dem die Welt läuft.
   Z. B.: `minetest/games/minetest_game/mods/` oder
   `/usr/share/minetest/games/minetest/`
2. Globale Mods. Der Ort, an dem Mods fast immer installiert werden. Im Zweifelsfall legst du Mods hier ab.
   Z. B.: `minetest/mods/`
3. Welt-Mods. Der Ort für Mods, die zu einer bestimmten Welt gehören.
   Z. B.: `minetest/worlds/world/worldmods/`

`minetest` ist das Benutzerdatenverzeichnis. Seinen Ort findest du, indem du Luanti öffnest und in der Registerkarte „Über“ auf „Benutzerdatenverzeichnis öffnen“ klickst. Wurde Luanti über Flathub installiert, passiert bei diesem Klick unter Umständen nichts. Das Verzeichnis liegt dann normalerweise unter `~/.var/app/org.luanti.luanti/.minetest/`.

Beim Laden prüft Luanti die oben genannten Orte der Reihe nach. Stößt es auf einen Mod mit demselben Namen wie ein zuvor gefundener, wird der spätere anstelle des früheren geladen. Du kannst also Spiel-Mods überschreiben, indem du einen gleichnamigen Mod in das globale Mod-Verzeichnis legst.

1.3 Deinen ersten Mod erstellen

1.3.1 Mod-Verzeichnis

Geh in das globale Mod-Verzeichnis (Über > Benutzerdatenverzeichnis öffnen > mods) und lege einen neuen Ordner namens `mymod` an. `mymod` ist der Name des Mods.

Jeder Mod sollte einen eindeutigen *Mod-Namen* haben, eine technische Kennung (ID), über die auf den Mod verwiesen wird. Mod-Namen dürfen Buchstaben, Ziffern und Unterstriche enthalten. Ein guter Name beschreibt, was der Mod tut. Das Verzeichnis mit den Bestandteilen eines Mods muss genauso heißen wie der Mod. Ob ein Mod-Name noch frei ist, findest du mit einer Suche auf `https://content.luanti.org` heraus.

```
mymod
├── textures
│   └── mymod_node.png
├── init.lua
└── mod.conf
```

Ein Mod benötigt nur eine `init.lua`. Eine `mod.conf` wird jedoch empfohlen, und je nach Funktionsumfang können weitere Bestandteile nötig sein.

1.3.2 mod.conf

Lege eine Datei `mod.conf` mit folgendem Inhalt an:

```
name = mymod
description = Fügt foo, bar und bo hinzu.
depends = default
```

Diese Datei enthält die Metadaten des Mods, darunter seinen Namen, seine Beschreibung und weitere Angaben.

1.3.3 init.lua

Lege eine Datei `init.lua` mit folgendem Inhalt an:

```
print("Diese Datei wird beim Laden ausgeführt!")

core.register_node("mymod:node", {
    description = "Das ist ein Node",
    tiles = { "mymod_node.png" },
    groups = { cracky = 1 },
})

core.register_craft({
    type = "shapeless",
    output = "mymod:node 3",
    recipe = { "default:dirt", "default:stone" },
})
```

Die `init.lua` ist der Einstiegspunkt eines Mods und wird ausgeführt, wenn der Mod geladen wird.

1.3.4 Zusammenfassung

Dieser Mod heißt „mymod“. Er besteht aus zwei Textdateien: `init.lua` und `mod.conf`. Das Skript gibt eine Nachricht aus und registriert dann einen Node und ein Handwerksrezept – beides wird später erklärt. Es gibt eine einzige Abhängigkeit, den default-Mod[1], der normalerweise in Minetest Game enthalten ist. Außerdem liegt in `textures/` eine Textur für den Node.

[1] `https://content.luanti.org/metapackages/default/`

1.4 Abhängigkeiten

Eine Abhängigkeit besteht, wenn ein Mod einen anderen Mod benötigt, der vor ihm geladen werden muss. Ein Mod kann darauf angewiesen sein, dass Code, Gegenstände oder andere Ressourcen eines anderen Mods verfügbar sind.

Es gibt zwei Arten von Abhängigkeiten: feste und optionale. Bei beiden wird der andere Mod zuerst geladen. Ist der Mod, von dem abgehangen wird, nicht verfügbar, führt eine feste Abhängigkeit dazu, dass der eigene Mod nicht geladen wird, während eine optionale Abhängigkeit höchstens dazu führt, dass weniger Funktionen zur Verfügung stehen.

Eine optionale Abhängigkeit ist nützlich, wenn du einen anderen Mod optional unterstützen möchtest; so lassen sich zusätzliche Inhalte aktivieren, wenn der Benutzer beide Mods gleichzeitig verwendet.

Abhängigkeiten werden in der `mod.conf` als kommagetrennte Liste angegeben:

```
depends = modeins, modzwei
optional_depends = moddrei
```

1.5 Mod-Pakete

Mods lassen sich zu Mod-Paketen (Modpacks) zusammenfassen, sodass mehrere Mods gemeinsam verpackt und verschoben werden können. Das ist nützlich, wenn du einem Spieler mehrere Mods bereitstellen möchtest, ohne dass er jeden einzeln herunterladen muss.

```
modpack1
├── modpack.conf (erforderlich, kennzeichnet Mod-Paket)
├── mod1
│   └── ... Mod-Dateien
└── mymod (optional)
    └── ... Mod-Dateien
```

Beachte, dass ein Mod-Paket kein *Spiel* ist. Spiele haben eine eigene Verzeichnisstruktur, die im Kapitel „Spiele“ erklärt wird.

2 Lua-Skripting

In diesem Kapitel lernst du die Skripterstellung in Lua kennen, die Werkzeuge, die dir dabei helfen, und einige Techniken, die du nützlich finden wirst.

2.1 Programmierung

Beim Programmieren geht es darum, ein Problem – etwa das Sortieren einer Liste – in Schritte zu zerlegen, die ein Computer verstehen kann. Den logischen Prozess des Programmierens zu vermitteln, würde den Rahmen dieses Buches sprengen; die folgenden Angebote sind dafür aber sehr hilfreich:

- Codecademy[1] (englisch) ist eine der besten Quellen, um Programmieren zu lernen. Sie bietet interaktive Lerneinheiten.
- Scratch[2] eignet sich gut, um bei den absoluten Grundlagen anzufangen und die Problemlösungstechniken zu erlernen, die man zum Programmieren braucht. Besonders gut geeignet für Kinder und Jugendliche.
- The Morpheus[3] ist ein guter YouTube-Kanal, um programmieren zu lernen.

2.1.1 Programmieren in Lua

Auch Lua selbst zu lehren, würde über dieses Buch hinausgehen. Das Buch Programming in Lua (PiL)[4] (englisch) ist jedoch eine ausgezeichnete Einführung in die Programmierung mit Lua.

2.2 Code-Editoren

Zum Schreiben von Lua-Skripten genügt ein Code-Editor mit Syntaxhervorhebung. Dabei werden Wörter und Zeichen je nach Bedeutung unterschiedlich eingefärbt. So fallen Fehler und Ungereimtheiten leicht auf.

[1] `http://www.codecademy.com/`
[2] `https://scratch.mit.edu`
[3] `https://www.youtube.com/@TheMorpheusTutorials`
[4] `https://www.lua.org/pil/contents.html`

Zum Beispiel:

```
function ctf.post(team, msg)
    if not ctf.team(team) then
        return false
    end
    if not ctf.team(team).log then
        ctf.team(team).log = {}
    end

    table.insert(ctf.team(team).log, 1, msg)
    ctf.save()

    return true
end
```

In diesem Beispiel sind die Schlüsselwörter hervorgehoben, darunter `if`, `then`, `end` und `return`. Funktionen, die Lua standardmäßig mitbringt, etwa `table.insert`, werden ebenfalls hervorgehoben.

Für den Einstieg empfehlen sich diese Editoren:

- VSCode[5] bzw. die quelloffene Variante VSCodium[6]: verbreitet und für alle Betriebssysteme erhältlich. Installiere dazu die Erweiterung Lua[7] (Lua Language Server). Sie vervollständigt Code automatisch, zeigt Fehler schon beim Tippen an und springt auf Wunsch zur Definition einer Funktion. Ergänzend gibt es Erweiterungen speziell für Luanti[8].
- Notepad++[9] (englisch): schlicht, nur für Windows.

Es gibt auch andere geeignete Editoren.

2.3 Lokaler und globaler Gültigkeitsbereich

Ob eine Variable lokal oder global ist, bestimmt, von wo aus sie gelesen und beschrieben werden kann. Auf globale Variablen kann von überall in der Skriptdatei und auch von jedem anderen Mod aus zugegriffen werden:

[5] https://code.visualstudio.com/
[6] https://vscodium.com/
[7] https://marketplace.visualstudio.com/items?itemName=sumneko.lua
[8] https://marketplace.visualstudio.com/items?itemName=GreenXenith.minetest-tools
[9] http://notepad-plus-plus.org/

```
function one()
    foo = "bar"
end

function two()
    print(dump(foo)) -- Ausgabe: "bar"
end

one()
two()
```

Auf eine lokale Variable kann dagegen nur dort zugegriffen werden, wo sie definiert ist. In Lua sind Variablen standardmäßig global; du musst daher ausdrücklich das Schlüsselwort `local` verwenden:

```
-- Innerhalb dieser Skriptdatei zugänglich
local one = 1

function myfunc()
    -- Innerhalb dieser Funktion zugänglich
    local two = one + one

    if two == one then
        -- Innerhalb dieser if-Anweisung zugänglich
        local three = one + two
    end
end
```

2.3.1 Möglichst lokale Variablen verwenden

Wann immer möglich, sollten lokale Variablen verwendet werden. Ein Mod sollte höchstens eine globale Variable anlegen, und zwar mit demselben Namen wie der Mod. Weitere globale Variablen sind unsauberer Programmierstil, und Luanti warnt davor:

```
Assignment to undeclared global 'foo' inside function
at init.lua:2
```

Sinngemäß: Zuweisung an die nicht deklarierte globale Variable `foo` innerhalb einer Funktion in `init.lua`, Zeile 2.

Um das zu beheben, verwende `local`:

```
function one()
    local foo = "bar"
end

function two()
    print(dump(foo)) -- Ausgabe: nil
end

one()
two()
```

Denk daran, dass `nil` **nicht initialisiert** bedeutet: Der Variablen wurde noch kein Wert zugewiesen, sie existiert nicht, oder ihr Wert wurde entfernt (also auf `nil` gesetzt).

Funktionen sind Variablen eines besonderen Typs, sollten aber ebenfalls lokal sein, weil andere Mods Funktionen mit demselben Namen haben könnten.

```
local function foo(bar)
    return bar * 2
end
```

Damit andere Mods deine Funktionen aufrufen können, lege eine Tabelle mit demselben Namen wie der Mod an und füge deine Funktionen dort ein. Diese Tabelle wird oft als API-Tabelle oder Namensraum (*namespace*) bezeichnet.

```
mymod = {}

function mymod.foo(bar)
    return "foo" .. bar
end

-- In einem anderen Mod oder Skript:
mymod.foo("foobar")
```

Die Schreibweise `function mymod.foo()` ist gleichbedeutend mit `mymod.foo = function()`, nur etwas lesbarer.

2.4 Andere Lua-Skripte einbinden

Andere Lua-Skripte bindet man in einen Mod am besten mit *dofile* ein.

```
dofile(core.get_modpath("modname") .. "/script.lua")
```

Ein Skript kann einen Wert zurückgeben. Damit lassen sich private lokale Variablen gezielt weitergeben:

```
-- script.lua
local module = {}
module.message = "Hallo Welt!"
return module
```

```
-- init.lua
local ret =
    dofile(core.get_modpath("modname") .. "/script.lua")
print(ret.message) -- Hallo Welt!
```

Wie man den Code eines Mods am besten aufteilt, wird in einem späteren Kapitel (Kapitel 24) erklärt.

Teil II

Items

3 Nodes, Items und Craften

Neue Nodes und Craftitems zu registrieren und Craftingrezepte anzulegen, gehört zu den Grundlagen vieler Mods.

3.1 Was sind Nodes und Items?

Nodes, Craftitems und Werkzeuge sind alles Items. Ein Item ist etwas, das sich in einem Inventar befinden kann – selbst wenn das im normalen Spielverlauf nicht möglich ist.

Ein Node ist ein Item, das platziert oder in der Welt gefunden werden kann. Jede Position in der Welt ist mit genau einem Node belegt – scheinbar leere Positionen sind normalerweise Luft-Nodes.

Ein Craftitem kann nicht platziert werden. Es kommt nur in Inventaren oder als fallengelassenes Item in der Welt vor.

Ein Werkzeug ist wie ein Craftitem, kann sich aber abnutzen. Während du das Werkzeug benutzt, sinkt der Abnutzungsbalken, bis das Werkzeug zerbricht. Werkzeuge lassen sich außerdem nie stapeln. Vermutlich werden Craftitems und Werkzeuge künftig zu einer einzigen Item-Art verschmelzen, da die Unterscheidung zwischen ihnen recht künstlich ist.

3.2 Items registrieren

Eine Item-Definition besteht aus einem *Itemnamen* und einer *Definitionstabelle*. Die Definitionstabelle enthält Attribute, die das Verhalten des Items bestimmen.

```
core.register_craftitem("modname:itemname", {
    description = "Mein besonderes Item",
    inventory_image = "modname_itemname.png",
})
```

3.2.1 Itemnamen

Jedes Item hat einen Itemnamen, über den es angesprochen wird. Er hat folgendes Format:

```
modname:itemname
```

`modname` ist der Name des Mods, in dem das Item registriert wird, `itemname` der Name des Items selbst. Der Itemname sollte beschreiben, was das Item ist, und darf nicht bereits registriert sein.

Sowohl `modname` als auch `itemname` sollten nur Kleinbuchstaben, Ziffern und Unterstriche enthalten.

3.2.2 Item-Aliase

Items können auch *Aliase* haben, die auf ihren Namen verweisen. Ein *Alias* ist ein Pseudo-Itemname: Die Engine behandelt jedes Vorkommen des Alias so, als stünde dort der eigentliche Itemname. Dafür gibt es zwei häufige Anwendungsfälle:

- Entfernte Items umbenennen. Wird ein Item ohne Korrekturcode aus einem Mod entfernt, können unbekannte Nodes in der Welt und in Inventaren zurückbleiben.
- Abkürzungen anlegen. `/giveme dirt` ist einfacher als `/giveme default:dirt`.

Einen Alias zu registrieren, ist einfach. Die Reihenfolge der Argumente merkst du dir am besten als `von → nach`, wobei *von* der Alias und *nach* das Ziel ist.

```
core.register_alias("dirt", "default:dirt")
```

Mods müssen Aliase selbst auflösen, bevor sie direkt mit Itemnamen arbeiten, denn die Engine tut das nicht. Das geht aber ganz einfach:

```
itemname = core.registered_aliases[itemname] or itemname
```

3.2.3 Texturen

Texturen gehören in den Ordner `textures/` und tragen Namen im Format `modname_itemname.png`. JPEG-Texturen werden unterstützt, können aber keine Transparenz und sehen bei niedrigen Auflösungen meist schlecht aus. Meist ist PNG die bessere Wahl.

Texturen in Luanti sind in der Regel 16 × 16 Pixel groß. Sie können jede Auflösung haben, empfohlen sind aber Zweierpotenzen, etwa 16, 32, 64 oder 128. Andere Auflösungen werden auf älteren Geräten, besonders auf Smartphones, möglicherweise nicht richtig unterstützt und verschlechtern dann die Leistung.

3.3 Einen einfachen Node registrieren

Nodes werden ähnlich wie Items registriert, nur mit einer anderen Funktion:

```
core.register_node("mymod:diamond", {
    description = "Alien-Diamant",
    tiles = { "mymod_diamond.png" },
    is_ground_content = true,
    groups = { cracky = 3, stone = 1 },
})
```

Node-Definitionen können jede Eigenschaft einer Item-Definition enthalten und haben zusätzlich eigene, node-spezifische Eigenschaften.

Die Eigenschaft `tiles` ist eine Tabelle mit den Texturnamen, die der Node verwendet. Gibt es nur eine Textur, wird sie auf allen Seiten verwendet. Für eine eigene Textur je Seite gibst du die Namen von sechs Texturen in dieser Reihenfolge an:

```
oben (+Y), unten (-Y), rechts (+X), links (-X),
hinten (+Z), vorne (-Z)
```

Denk daran, dass +Y in Luanti nach oben zeigt, wie in den meisten 3D-Computerspielen üblich.

```
core.register_node("mymod:diamond", {
    description = "Alien-Diamant",
    tiles = {
        "mymod_diamond_up.png", -- y+
        "mymod_diamond_down.png", -- y-
        "mymod_diamond_right.png", -- x+
        "mymod_diamond_left.png", -- x-
        "mymod_diamond_back.png", -- z+
        "mymod_diamond_front.png", -- z-
    },
    is_ground_content = true,
    groups = { cracky = 3 },
    drop = "mymod:diamond_fragments",
    -- ^ statt des Diamanten Fragmente fallen lassen
})
```

Das Attribut `is_ground_content` erlaubt es, Höhlen durch diesen Node zu graben. Das ist wichtig für jeden Node, der bei der Kartengenerierung unter der Erde platziert werden kann. Höhlen werden aus der Welt herausgeschnitten, nachdem alle anderen Nodes eines Gebiets generiert wurden.

3.4 Crafting

Es gibt mehrere Arten von Craftingrezepten; welche gemeint ist, gibt die Eigenschaft `type` an.

- `shaped` – Die Zutaten müssen an der richtigen Stelle liegen.
- `shapeless` – Es ist egal, wo die Zutaten liegen; nur die Menge muss stimmen.
- `cooking` – Rezepte für den Ofen.
- `fuel` – Legt fest, welche Items in Öfen verbrannt werden können.
- `tool_repair` – Legt fest, welche Items sich als Werkzeug reparieren lassen.

Craftingrezepte sind keine Items und haben daher keinen Itemnamen, der sie eindeutig kennzeichnet.

3.4.1 Shaped

Bei Shaped-Rezepten müssen die Zutaten in der richtigen Form, also im richtigen Muster, angeordnet sein. Im folgenden Beispiel müssen die Fragmente wie

ein Stuhl angeordnet sein, damit das Rezept funktioniert. Die lokale Variable `frag` hält das Rezept übersichtlich.

```
local frag = "mymod:diamond_fragments"

core.register_craft({
    type = "shaped",
    output = "mymod:diamond_chair 99",
    recipe = {
        { frag, "",   "" },
        { frag, frag, "" },
        { frag, frag, "" },
    },
})
```

Beachte die leere Spalte auf der rechten Seite. Sie bedeutet, dass rechts von der Form eine leere Spalte vorhanden sein *muss*, sonst funktioniert das Rezept nicht. Soll diese leere Spalte nicht nötig sein, lässt du die leeren Strings einfach weg:

```
core.register_craft({
    output = "mymod:diamond_chair 99",
    recipe = {
        { frag, ""   },
        { frag, frag },
        { frag, frag },
    },
})
```

Das Feld `type` wird für Shaped-Rezepte eigentlich nicht gebraucht, da `shaped` der Standardtyp ist.

3.4.2 Shapeless

Shapeless-Rezepte verwendest du, wenn es egal ist, wo die Zutaten liegen – Hauptsache, sie sind da.

```
core.register_craft({
    type = "shapeless",
    output = "mymod:diamond 3",
    recipe = {
        "mymod:diamond_fragments",
        "mymod:diamond_fragments",
        "mymod:diamond_fragments",
    },
})
```

3.4.3 Cooking und Fuel

Rezepte vom Typ `cooking` werden nicht im Craftinggitter hergestellt, sondern in Öfen oder anderen Kochgeräten gekocht, die es in Mods geben kann.

```
core.register_craft({
    type = "cooking",
    output = "mymod:diamond_fragments",
    recipe = "default:coalblock",
    cooktime = 10,
})
```

Der einzige echte Unterschied im Code ist, dass `recipe` nur ein einzelnes Item ist statt einer Tabelle (in geschweiften Klammern). Optional gibt es den Parameter `cooktime`, der festlegt, wie lange das Item zum Kochen braucht. Ohne Angabe sind es 3 Sekunden.

Das Rezept oben funktioniert, wenn der Kohleblock im Eingabefeld liegt und darunter irgendein Brennstoff. Nach 10 Sekunden entstehen Diamantfragmente!

Der Typ `fuel` ergänzt den Typ `cooking`: Er legt fest, was in Öfen und anderen Kochgeräten aus Mods verbrannt werden kann.

```
core.register_craft({
    type = "fuel",
    recipe = "mymod:diamond",
    burntime = 300,
})
```

Fuel-Rezepte haben kein Ergebnis wie andere Rezepte, dafür aber eine Brenndauer in Sekunden, die angibt, wie lange sie als Brennstoff reichen. Der Diamant brennt also 300 Sekunden lang!

3.5 Gruppen

Items können Mitglied in vielen Gruppen sein, und Gruppen können viele Mitglieder haben. Gruppen werden mit der Eigenschaft `groups` in der Definitionstabelle festgelegt und haben jeweils einen zugehörigen Wert.

```
groups = { cracky = 3, wood = 1 }
```

Gruppen haben mehrere Zwecke. Erstens beschreiben sie Eigenschaften wie Abbautyp und Brennbarkeit. Zweitens kann in einem Craftingrezept statt eines Itemnamens eine Gruppe stehen; dann passt jedes Item dieser Gruppe.

```
core.register_craft({
    type = "shapeless",
    output = "mymod:diamond_thing 3",
    recipe = { "group:wood", "mymod:diamond" },
})
```

3.6 Werkzeuge, Fähigkeiten und Abbautypen

Abbautypen sind Gruppen, die festlegen, wie widerstandsfähig ein Node beim Abbauen mit verschiedenen Werkzeugen ist. Ein höherer Wert in einer Abbautyp-Gruppe bedeutet, dass sich der Node leichter und schneller abbauen lässt. Mehrere Abbautypen lassen sich kombinieren, damit verschiedene Werkzeugarten effizient eingesetzt werden können. Ein Node ohne Abbautyp kann mit keinem Werkzeug abgebaut werden.

Gruppe	Bestes Werkzeug	Beschreibung
`crumbly`	Schaufel	Erde, Sand
`cracky`	Spitzhacke	Zähes (aber sprödes) Material wie Stein

Gruppe	Bestes Werkzeug	Beschreibung
`snappy`	*beliebig*	Lässt sich mit feinen Werkzeugen schneiden, z. B. Blätter, kleine Pflanzen, Draht, Blech
`choppy`	Axt	Lässt sich mit scharfer Kraft schneiden, z. B. Bäume, Holzbretter
`fleshy`	Schwert	Lebewesen wie Tiere und der Spieler; kann Bluteffekte beim Treffen nahelegen
`explody`	?	Besonders anfällig für Explosionen
`oddly_breakable_by_hand`	*beliebig*	Fackeln und Ähnliches – sehr schnell abzubauen

Jedes Werkzeug hat eine Werkzeugfähigkeit (*tool capability*). Sie umfasst eine Liste der unterstützten Abbautypen und für jeden Typ zugehörige Eigenschaften wie Abbauzeiten und Abnutzung. Werkzeuge können außerdem je Typ eine maximale Härte haben; so lässt sich verhindern, dass schwächere Werkzeuge härtere Nodes abbauen. Sehr häufig enthalten Werkzeuge alle Abbautypen in ihren Fähigkeiten, wobei die weniger geeigneten sehr ineffiziente Werte haben. Hat das Item, das ein Spieler gerade in der Hand hält, keine eigene Werkzeugfähigkeit, wird stattdessen die Fähigkeit der Hand verwendet.

```
core.register_tool("mymod:tool", {
    description = "Mein Werkzeug",
    inventory_image = "mymod_tool.png",
    tool_capabilities = {
        full_punch_interval = 1.5,
        max_drop_level = 1,
        groupcaps = {
            crumbly = {
                maxlevel = 2,
                uses = 20,
                times = { [1] = 1.60, [2] = 1.20, [3] = 0.80 },
            },
        },
        damage_groups = { fleshy = 2 },
    },
```

```
})
```

`groupcaps` ist die Liste der Abbautypen, mit denen das Werkzeug Nodes abbauen kann. Schadensgruppen (*damage groups*) steuern, wie Werkzeuge Objekten Schaden zufügen; das wird später im Kapitel „Objekte, Spieler und Entities“ behandelt.

4 Node- und Item-Callbacks

Luanti setzt beim Modding stark auf Callbacks. Ein Callback ist eine Funktion, die du einer API übergibst und die aufgerufen wird, wenn ein Ereignis eintritt. Du kannst zum Beispiel in einer Node-Definition eine Funktion `on_punch` angeben, die aufgerufen wird, wenn ein Spieler den Node schlägt. Es gibt auch globale Callbacks wie `core.register_on_punchnode`, die Ereignisse für alle Nodes empfangen.

4.1 Item-Callbacks

Hat ein Spieler einen Node, ein Craftitem oder ein Werkzeug im Inventar, kann er damit bestimmte Ereignisse auslösen:

Callback	Standardbelegung	Standardwert
`on_use`	Linksklick	`nil`
`on_place`	Rechtsklick auf einen Node	`core.item_place`
`on_secondary_use`	Rechtsklick ins Leere	`core.item_secondary_use` (tut nichts)
`on_drop`	Q	`core.item_drop`
`after_use`	Abbauen eines Nodes	`nil`

4.1.1 on_use

Hat ein Item einen `on_use`-Callback, kann es nicht mehr zum Abbauen von Nodes verwendet werden. Ein typischer Einsatz von `on_use` sind Lebensmittel:

```
core.register_craftitem("mymod:mudpie", {
    description = "Alien-Matschkuchen",
    inventory_image = "myfood_mudpie.png",
    on_use = core.item_eat(20),
})
```

Die Zahl, die du `core.item_eat` übergibst, ist die Anzahl der Lebenspunkte, die der Verzehr heilt. Jedes Herz des Spielers ist zwei Lebenspunkte wert. Ein Spieler hat normalerweise bis zu 10 Herzen, also 20 Lebenspunkte.

`core.item_eat()` ist eine Funktion, die eine Funktion zurückgibt; diese wird als `on_use`-Callback eingetragen. Der Code oben ist also gleichbedeutend mit:

```
core.register_craftitem("mymod:mudpie", {
    description = "Alien-Matschkuchen",
    inventory_image = "myfood_mudpie.png",
    on_use = function(...)
        return core.do_item_eat(20, nil, ...)
    end,
})
```

Wenn du verstehst, dass `item_eat` einfach eine Funktion zurückgibt, kannst du sie abwandeln und komplexeres Verhalten einbauen, etwa einen eigenen Sound abspielen.

4.1.2 on_place und on_secondary_use

`on_place` wird aufgerufen, wenn der Spieler auf einen Node zeigt, `on_secondary_use`, wenn er das nicht tut.

Beide Callbacks werden für alle Item-Arten aufgerufen. `on_place` ist standardmäßig `core.item_place`; diese Funktion ruft entweder den `on_rightclick`-Callback des anvisierten Nodes auf oder platziert das gehaltene Item, falls es ein Node ist.

4.1.3 on_drop

`on_drop` wird aufgerufen, wenn der Spieler ein Item fallen lassen will, etwa mit der Taste Q oder indem er es aus dem Inventar herauszieht. Standardmäßig ist das `core.item_drop`, die das Fallenlassen übernimmt.

4.1.4 after_use

`after_use` wird beim Abbauen eines Nodes aufgerufen und erlaubt dir, selbst festzulegen, wie sich ein Werkzeug abnutzt. Fehlt `after_use`, entspricht das diesem Code:

```
after_use = function(itemstack, user, node, digparams)
    itemstack:add_wear(digparams.wear)
    return itemstack
end
```

4.2 item_place oder place_item?

Die Luanti-API bringt viele fertige Callback-Implementierungen mit, die du verwenden kannst. Sie sind nach dem Schema *Typ zuerst* benannt, zum Beispiel `core.item_place` und `core.node_dig`. Manche werden direkt eingesetzt, andere sind Funktionen, die den Callback erst zurückgeben:

```
core.register_item("mymod:example", {
    on_place = core.item_place,
    on_use = core.item_eat(10),
})
```

Außerdem enthält die API Funktionen, die etwas *tun*. Sie heißen verwirrend ähnlich wie die Callback-Implementierungen, beginnen aber mit dem Verb. Beispiele sind `core.place_node` und `core.dig_node`: Mit ihnen kannst du Nodes abbauen und platzieren, mit ähnlicher Wirkung, als täte es ein Spieler.

4.3 Node-Callbacks

Solange ein Node in einem Inventar liegt, verwendet er wie oben beschrieben Item-Callbacks. Ist er in der Welt platziert, verwendet er Node-Callbacks. Davon gibt es sehr viele, zu viele für dieses Buch; einige davon kommen aber in späteren Kapiteln vor.

Mehrere dieser Callbacks betreffen Node-Operationen wie das Platzieren und Entfernen in der Welt. Wichtig: Aus Leistungsgründen werden sie bei Massenänderungen – also wenn sehr viele Nodes auf einmal gesetzt werden – nicht aufgerufen. Du kannst dich also nicht darauf verlassen, dass sie immer ausgelöst werden.

4.3.1 Rechtsklick und Platzieren

Klickt der Spieler mit einem Item rechts, während er auf einen Node zeigt, wird der `on_place`-Callback des Items aufgerufen, standardmäßig `core.item_place`. Hat der anvisierte Node einen `on_rightclick`-Callback und hält der Spieler die Schleichtaste (Shift) *nicht* gedrückt, wird `on_rightclick` aufgerufen. Andernfalls platziert `core.item_place` den Node.

Beim Platzieren eines Nodes werden sowohl `on_construct` als auch `after_place_node` aufgerufen. `on_construct` wird bei jedem Setzen eines Nodes außerhalb von Massenänderungen ausgelöst und erhält nur die Position des Nodes. `after_place_node` wird nur beim Platzieren aufgerufen und bekommt daher mehr Informationen, etwa den Platzierenden und den Itemstack.

Beachte, dass nicht nur Spieler Nodes platzieren können; auch Mobs und Mods tun das häufig. Deshalb kann `placer` ein Spieler, eine Entity oder `nil` sein.

```
core.register_node("mymod:mynode", {
    on_rightclick = function(
        pos,
        node,
        clicker,
        itemstack,
        pointed_thing
    )
        if clicker:is_player() then
            core.chat_send_player(
                clicker:get_player_name(),
                "Hallo Welt!"
            )
        end
    end,
    on_construct = function(pos)
        local meta = core.get_meta(pos)
        meta:set_string("infotext", "Mein Node!")
    end,
    after_place_node = function(
        pos,
        placer,
        itemstack,
        pointed_thing
    )
        -- placer immer prüfen
        if placer and placer:is_player() then
```

```
            local meta = core.get_meta(pos)
            meta:set_string(
                "owner",
                placer:get_player_name()
            )
        end
    end,
})
```

4.3.2 Schlagen und Abbauen

Schlagen heißt, dass der Spieler kurz links klickt. Hat das gehaltene Item einen `on_use`-Callback, wird dieser aufgerufen, andernfalls der `on_punch`-Callback des anvisierten Nodes.

Versucht der Spieler, einen Node abzubauen, wird dessen `on_dig`-Callback aufgerufen. Standardmäßig ist das `core.node_dig`: Die Funktion prüft den Gebietsschutz, nutzt das Werkzeug ab, entfernt den Node und führt den `after_dig_node`-Callback aus.

```
core.register_node("mymod:mynode", {
    on_punch = function(pos, node, puncher, pointed_thing)
        if puncher:is_player() then
            core.chat_send_player(
                puncher:get_player_name(),
                "Au!"
            )
        end
    end,
})
```

4.3.3 ... und mehr!

Eine Liste aller Node-Callbacks und weitere Details zu den oben genannten findest du in der Lua-API-Referenz von Luanti.

5 Texturen erstellen

Texturen erstellen und optimieren zu können, ist bei der Entwicklung für Luanti sehr nützlich. Für Pixel-Art-Texturen gibt es viele hilfreiche Techniken, und wer sie versteht, verbessert die Qualität seiner Texturen erheblich.

Ausführliche Anleitungen für gute Pixel-Art würden den Rahmen dieses Buches sprengen; hier geht es nur um die wichtigsten Grundtechniken. Es gibt viele gute Online-Tutorials[1] (englisch), die Pixel-Art viel ausführlicher behandeln.

5.1 Techniken

5.1.1 Mit dem Stift arbeiten

Das Stiftwerkzeug gibt es in den meisten Editoren. Auf die kleinste Größe eingestellt, bearbeitest du damit ein Pixel nach dem anderen, ohne andere Teile des Bildes zu verändern. So entstehen klare, scharfe Texturen ohne ungewollte Unschärfe. Außerdem hast du damit viel Präzision und Kontrolle.

5.1.2 Kacheln

Texturen für Nodes sollten in der Regel kachelbar sein. Das heißt: Setzt du mehrere Nodes mit derselben Textur nebeneinander, passen die Kanten nahtlos aneinander.

Passen die Kanten nicht zusammen, sieht das Ergebnis deutlich weniger ansprechend aus.

5.1.3 Transparenz

Transparenz ist wichtig für die Texturen fast aller Craftitems und mancher Nodes, etwa Glas. Nicht jeder Editor unterstützt Transparenz; achte also darauf, einen Editor zu wählen, der zu deinen Texturen passt.

[1] `http://www.photonstorm.com/art/tutorials-art/16x16-pixel-art-tutorial`

5.1.4 Farbpaletten

Eine einheitliche Farbpalette ist ein einfacher Weg, deine Grafiken deutlich besser aussehen zu lassen. Verwende am besten eine Palette mit wenigen Farben, höchstens etwa 32. Fertige Paletten findest du auf lospec.com[2].

5.2 Editoren

5.2.1 MS Paint

MS Paint ist ein einfacher Editor, der für grundlegendes Texturdesign ausreichen kann; Transparenz unterstützt er allerdings nicht. Für die Seiten von Nodes spielt das meist keine Rolle, brauchst du aber Transparenz, solltest du einen anderen Editor wählen.

5.2.2 Aseprite / LibreSprite

Aseprite[3] ist ein proprietärer Pixel-Art-Editor. Er bringt von Haus aus viele nützliche Funktionen mit, etwa Farbpaletten und Animationswerkzeuge.

LibreSprite[4] ist ein quelloffener Fork von Aseprite aus der Zeit, bevor Aseprite proprietär wurde.

5.2.3 GIMP

GIMP ist in der Luanti-Community weit verbreitet. Die Einarbeitung dauert recht lange, weil viele Funktionen nicht sofort ersichtlich sind.

Verwende in GIMP das Stiftwerkzeug mit dem Pinsel „Pixel“ und der Größe 1. Außerdem empfiehlt es sich, beim Radierer die Option „Harte Kanten“ zu aktivieren.

[2] https://lospec.com/palette-list
[3] https://www.aseprite.org/
[4] https://libresprite.github.io/

6 Zeichnungstypen von Nodes

Die Art, wie ein Node gezeichnet wird, heißt *Zeichnungstyp* (*drawtype*). Es gibt viele davon. Das Verhalten eines Zeichnungstyps steuerst du über Eigenschaften in der Definition des Node-Typs. Diese Eigenschaften gelten für alle Instanzen dieses Nodes; manche lassen sich aber über `param2` auch je Node steuern.

Im vorigen Kapitel wurden Nodes und Items eingeführt, eine vollständige Definition eines Nodes fehlte aber noch. Die Luanti-Welt ist ein 3D-Gitter aus Positionen. Jede Position heißt Node und besteht aus dem Node-Typ (Name) und zwei Parametern (`param1` und `param2`). Der Name der Funktion `core.register_node` ist etwas irreführend: Sie registriert nicht wirklich einen Node, sondern einen neuen *Typ* von Node.

Die Node-Parameter steuern, wie ein einzelner Node dargestellt wird. `param1` speichert die Beleuchtung eines Nodes; die Bedeutung von `param2` hängt von der Eigenschaft `paramtype2` der Node-Typ-Definition ab.

6.1 Würfelförmige Nodes: normal und allfaces



Abbildung 6.1: Zeichnungstyp normal

Der Zeichnungstyp `normal` wird typischerweise für würfelförmige Nodes verwendet. Liegt eine Seite eines normalen Nodes an einer festen Seite an, wird sie nicht gezeichnet. Das bringt einen großen Leistungsgewinn.

Der Zeichnungstyp `allfaces` zeichnet dagegen auch die Innenseite, wenn sie an einem festen Node anliegt. Das eignet sich für Nodes mit teilweise transparenten Seiten, etwa Blätter. Mit `allfaces_optional` können Benutzer

das langsamere Zeichnen abschalten; der Node verhält sich dann wie ein normaler Node.

```
core.register_node("mymod:diamond", {
    description = "Alien-Diamant",
    tiles = { "mymod_diamond.png" },
    groups = { cracky = 3 },
})

core.register_node("default:leaves", {
    description = "Blätter",
    drawtype = "allfaces_optional",
    tiles = { "default_leaves.png" },
})
```

Hinweis: `normal` ist der Standard-Zeichnungstyp, du musst ihn also nicht ausdrücklich angeben.

6.2 Glasartige Nodes

Der Unterschied zwischen glasartigen (`glasslike`) und normalen Nodes: Steht ein glasartiger Node neben einem normalen, wird die angrenzende Seite des normalen Nodes trotzdem gezeichnet. Das ist nützlich, weil glasartige Nodes meist transparent sind – mit dem Zeichnungstyp `normal` könnte man sonst durch die Welt hindurchsehen.



Abbildung 6.2: Kanten bei glasslike

```
core.register_node("default:obsidian_glass", {
    description = "Obsidianglas",
    drawtype = "glasslike",
    tiles = { "default_obsidian_glass.png" },
    paramtype = "light",
    is_ground_content = false,
    sunlight_propagates = true,
    sounds = default.node_sound_glass_defaults(),
    groups = { cracky = 3, oddly_breakable_by_hand = 3 },
})
```

6.2.1 glasslike_framed

Hier läuft der Rand mit einem 3D-Effekt um die gesamte zusammenhängende Fläche statt um jeden einzelnen Node:

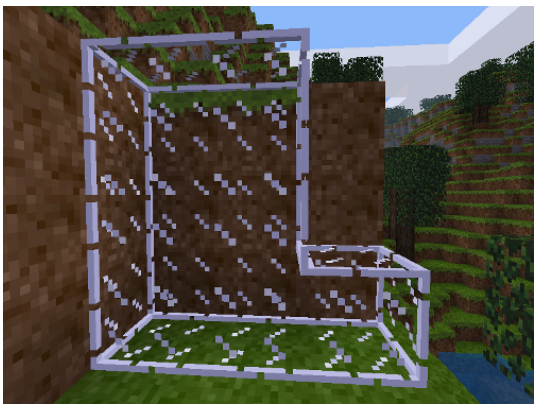


Abbildung 6.3: Kanten bei glasslike_framed

Mit dem Zeichnungstyp `glasslike_framed_optional` kann der Benutzer das gerahmte Aussehen selbst *einschalten*.

```
core.register_node("default:glass", {
    description = "Glas",
    drawtype = "glasslike_framed",
    tiles = {
        "default_glass.png",
        "default_glass_detail.png",
    },
    inventory_image = core.inventorycube(
```

```
        "default_glass.png"
    ),
    paramtype = "light",
    sunlight_propagates = true, -- Sonnenlicht geht durch
    groups = { cracky = 3, oddly_breakable_by_hand = 3 },
    sounds = default.node_sound_glass_defaults(),
})
```

6.3 Luftartige Nodes

Diese Nodes werden nicht gezeichnet und haben daher keine Texturen.

```
core.register_node("myair:air", {
    description = "MyAir (du Hacker, du!)",
    drawtype = "airlike",
    paramtype = "light",
    sunlight_propagates = true,

    walkable = false, -- sonst kollidiert der Spieler
    pointable = false, -- Node kann nicht ausgewählt werden
    diggable = false, -- Node kann nicht abgebaut werden
    buildable_to = true, -- Nodes können diesen Node ersetzen
    -- (man kann einen Node an seine Stelle
    -- setzen, die Luft verschwindet dann)

    air_equivalent = true,
    drop = "",
    groups = { not_in_creative_inventory = 1 },
})
```

6.4 Beleuchtung und Ausbreitung des Sonnenlichts

Die Beleuchtung eines Nodes wird in `param1` gespeichert. Um zu berechnen, wie eine Seite eines Nodes schattiert wird, nimmt Luanti den Lichtwert des benachbarten Nodes. Deshalb haben feste Nodes keine Lichtwerte – sie blockieren das Licht ja.

Standardmäßig erlaubt ein Node-Typ nicht, Licht in seinen Instanzen zu speichern. Bei manchen Nodes wie Glas und Luft soll Licht aber durchgehen. Dazu müssen zwei Eigenschaften gesetzt werden:

```
paramtype = "light",
sunlight_propagates = true,
```

Die erste Zeile bedeutet, dass `param1` tatsächlich den Lichtwert speichert. Die zweite, dass Sonnenlicht ohne Abschwächung durch den Node hindurchgeht.

6.5 Flüssige Nodes

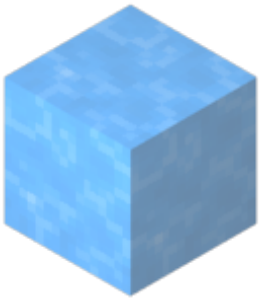


Abbildung 6.4: Zeichnungstyp liquid

Jede Flüssigkeit braucht zwei Node-Definitionen: eine für die Quelle und eine für die fließende Flüssigkeit.

```
-- Einige Eigenschaften wurden weggelassen, weil sie über
-- dieses Kapitel hinausgehen.
core.register_node("default:water_source", {
    drawtype = "liquid",
    paramtype = "light",

    inventory_image = core.inventorycube(
        "default_water.png"
    ),
    -- ^ nötig, damit das Inventarbild nicht animiert wird

    tiles = {
        {
            name = "default_water_source_animated.png",
            animation = {
                type = "vertical_frames",
                aspect_w = 16,
                aspect_h = 16,
```

```
                length = 2.0,
            },
        },
    },

    special_tiles = {
        -- Wasserquellen neuen Typs (kaum genutzt)
        {
            name = "default_water_source_animated.png",
            animation = {
                type = "vertical_frames",
                aspect_w = 16,
                aspect_h = 16,
                length = 2.0,
            },
            backface_culling = false,
        },
    },

    --
    -- Verhalten
    --
    walkable = false, -- Spieler fällt hindurch
    pointable = false, -- Spieler kann es nicht auswählen
    diggable = false, -- Spieler kann es nicht abbauen
    buildable_to = true, -- Nodes können diesen Node ersetzen

    alpha = 160,

    --
    -- Flüssigkeitseigenschaften
    --
    drowning = 1,
    liquidtype = "source",

    liquid_alternative_flowing = "default:water_flowing",
    -- ^ wenn die Flüssigkeit fließt

    liquid_alternative_source = "default:water_source",
    -- ^ wenn die Flüssigkeit eine Quelle ist

    liquid_viscosity = WATER_VISC,
    -- ^ wie schnell

    liquid_range = 8,
```

```
    -- ^ wie weit

    post_effect_color = { a = 64, r = 100, g = 100, b = 200 },
    -- ^ Bildschirmfarbe unter Wasser
})
```

Fließende Nodes werden ähnlich definiert, nur mit anderem Namen und anderer Animation. Ein vollständiges Beispiel ist `default:water_flowing` im Mod `default` von Minetest Game.

6.6 Nodeboxen



Abbildung 6.5: Zeichnungstyp nodebox

Mit Nodeboxen erstellst du Nodes, die nicht würfelförmig sind, sondern aus beliebig vielen Quadern bestehen.

```
core.register_node("stairs:stair_stone", {
    drawtype = "nodebox",
    paramtype = "light",
    node_box = {
        type = "fixed",
        fixed = {
            { -0.5, -0.5, -0.5, 0.5, 0, 0.5 },
            { -0.5, 0, 0, 0.5, 0.5, 0.5 },
        },
    },
})
```

Der wichtigste Teil ist die Nodebox-Tabelle:

```
{-0.5, -0.5, -0.5,       0.5,    0,  0.5},
{-0.5,    0,    0,       0.5,  0.5,  0.5}
```

Jede Zeile ist ein Quader; zusammen ergeben sie einen einzigen Node. Die ersten drei Zahlen sind die Koordinaten (von −0,5 bis einschließlich 0,5) der unteren, vorderen, linken Ecke, die letzten drei die der gegenüberliegenden Ecke. Die Reihenfolge ist X, Y, Z, wobei Y nach oben zeigt.

Mit dem NodeBoxEditor[1] kannst du Nodeboxen erstellen, indem du die Kanten ziehst. Das ist anschaulicher als die Arbeit von Hand.

6.6.1 Nodeboxen für Wandmontage

Manchmal braucht man unterschiedliche Nodeboxen, je nachdem, ob der Node auf dem Boden, an der Wand oder an der Decke platziert wird, etwa bei Fackeln.

```
core.register_node("default:sign_wall", {
    drawtype = "nodebox",
    node_box = {
        type = "wallmounted",

        -- Decke
        wall_top = {
            {
                -0.4375,
                0.4375,
                -0.3125,
                0.4375,
                0.5,
                0.3125,
            },
        },

        -- Boden
        wall_bottom = {
            {
                -0.4375,
                -0.5,
                -0.3125,
```

[1] https://forum.luanti.org/viewtopic.php?f=14&t=2840

```
                0.4375,
                -0.4375,
                0.3125,
            },
        },

        -- Wand
        wall_side = {
            {
                -0.5,
                -0.3125,
                -0.4375,
                -0.4375,
                0.3125,
                0.4375,
            },
        },
    },
})
```

6.7 Mesh-Nodes

Nodeboxen sind zwar meist einfacher zu erstellen, bestehen aber nur aus Quadern. Außerdem sind sie nicht optimiert: Innenflächen werden auch dann gezeichnet, wenn sie vollständig verdeckt sind.

Eine Fläche (*face*) ist eine ebene Oberfläche eines Meshs. Eine Innenfläche entsteht, wenn sich die Flächen zweier Nodeboxen überlappen. Teile des Modells sind dann unsichtbar, werden aber trotzdem gezeichnet.

So registrierst du einen Mesh-Node:

```
core.register_node("mymod:meshy", {
    drawtype = "mesh",

    -- Enthält die Textur für jedes „Material“
    tiles = {
        "mymod_meshy.png",
    },

    -- Pfad zum Mesh
```

```
    mesh = "mymod_meshy.b3d",
})
```

Achte darauf, dass das Mesh in einem Verzeichnis `models` liegt. Meist gehört es in den Ordner deines eigenen Mods; du kannst aber auch ein Mesh aus einem anderen Mod verwenden, von dem deiner abhängt. Ein Mod, der weitere Möbel hinzufügt, kann zum Beispiel die Modelle eines grundlegenden Möbel-Mods mitnutzen.

6.8 Signlike-Nodes

Signlike-Nodes sind flache Nodes, die an den Seiten anderer Nodes angebracht werden. Trotz des Namens verwenden Schilder meist nicht `signlike`, sondern `nodebox`, um einen 3D-Effekt zu erzielen. Leitern dagegen nutzen häufig `signlike`.

```
core.register_node("default:ladder_wood", {
    drawtype = "signlike",

    tiles = { "default_ladder_wood.png" },

    -- Erforderlich: Drehung in param2 speichern
    paramtype2 = "wallmounted",

    selection_box = {
        type = "wallmounted",
    },
})
```

6.9 Plantlike-Nodes



Abbildung 6.6: Zeichnungstyp plantlike

Plantlike-Nodes zeichnen ihre Textur in einem X-förmigen Muster.

```
core.register_node("default:papyrus", {
    drawtype = "plantlike",

    -- Nur eine Textur
    tiles = { "default_papyrus.png" },

    selection_box = {
        type = "fixed",
        fixed = {
            -6 / 16,
            -0.5,
            -6 / 16,
            6 / 16,
            0.5,
            6 / 16,
        },
    },
})
```

6.10 Firelike-Nodes

`firelike` ähnelt `plantlike`, ist aber so gestaltet, dass es an Wänden und Decken „haftet“.



Abbildung 6.7: Firelike-Nodes

```
core.register_node("mymod:clinger", {
    drawtype = "firelike",

    -- Nur eine Textur
    tiles = { "mymod_clinger.png" },
})
```

6.11 Weitere Zeichnungstypen

Die Liste ist nicht vollständig; es gibt noch weitere Typen, darunter:

- `fencelike`
- `plantlike_rooted` – für Unterwasserpflanzen
- `raillike` – für Schienen
- `torchlike` – für 2D-Nodes an Wand, Boden oder Decke. Die Fackeln in Minetest Game verwenden allerdings Mesh-Nodes mit drei getrennten Definitionen (`default:torch`, `default:torch_wall` und `default:torch_ceiling`).

Die vollständige Liste findest du wie immer in der Lua-API-Dokumentation[2].

[2] `https://api.luanti.org/nodes/#node-drawtypes`

7 ItemStacks und Inventare

In diesem Kapitel lernst du, wie man Inventare verwendet und verändert – egal ob Spielerinventar, Node-Inventar oder freistehendes Inventar.

7.1 Was sind ItemStacks und Inventare?

Ein ItemStack sind die Daten hinter einem einzelnen Feld eines Inventars.

Ein *Inventar* ist eine Sammlung von *Inventarlisten*, von denen jede ein 2D-Gitter aus ItemStacks ist. Im Zusammenhang mit Inventaren heißen Inventarlisten einfach *Listen*.

Spieler und Nodes haben jeweils nur ein einziges Inventar; Listen erlauben mehrere Gitter innerhalb dieses Inventars. Standardmäßig hat der Spieler die Liste `main` für den Großteil seines Inventars und einige Listen für das Crafting.

7.2 ItemStacks

ItemStacks bestehen aus vier Teilen: `name`, `count`, `wear` und Metadaten.

Der Itemname kann der Name eines registrierten Items, ein Alias oder ein unbekannter Itemname sein. Unbekannte Items entstehen häufig, wenn Benutzer Mods deinstallieren oder Mods Items ohne Vorkehrungen wie Aliase entfernen.

```
print(stack:get_name())
stack:set_name("default:dirt")

if not stack:is_known() then
    print("Unbekanntes Item!")
end
```

Die Anzahl ist immer 0 oder größer. Im normalen Spielverlauf sollte sie die maximale Stapelgröße des Items, `stack_max`, nicht überschreiten. Admin-Befehle und fehlerhafte Mods können aber dazu führen, dass Stacks größer werden.

```
print(stack:get_stack_max())
```

Ein ItemStack kann leer sein; dann ist die Anzahl 0.

```
print(stack:get_count())
stack:set_count(10)
```

ItemStacks lassen sich mit der Funktion `ItemStack` auf verschiedene Weise erzeugen:

```
ItemStack() -- name="", count=0
ItemStack("default:pick_stone") -- count=1
ItemStack("default:stone 30")
ItemStack({ name = "default:wood", count = 10 })
```

Item-Metadaten sind ein unbegrenzter Schlüssel-Wert-Speicher für Daten zum Item. Schlüssel-Wert heißt: Über einen Namen (den Schlüssel, *key*) greifst du auf die Daten (den Wert, *value*) zu. Manche Schlüssel haben eine besondere Bedeutung, etwa `description`, mit dem ein Stack eine eigene Beschreibung bekommt. Mehr dazu im Kapitel Speicherung und Metadaten (Kapitel 10).

7.3 Inventarorte

Der Inventarort legt fest, wo und wie ein Inventar gespeichert wird. Es gibt drei Arten: Spieler, Node und freistehend. Ein Inventar ist fest an genau einen Ort gebunden – Änderungen am Inventar wirken sofort.

7.3.1 Node-Inventare

Node-Inventare gehören zur Position eines bestimmten Nodes, etwa einer Truhe. Der Node muss geladen sein, weil das Inventar in den Node-Metadaten (Kapitel 10) gespeichert wird.

```
on_punch = function(pos, node)
    local inv =
        core.get_inventory({ type = "node", pos = pos })
    -- jetzt das Inventar verwenden
end,
```

So erhältst du eine *Inventar-Referenz*, meist *InvRef* genannt. Mit Inventar-Referenzen veränderst du ein Inventar. *Referenz* bedeutet, dass die Daten nicht im Objekt selbst liegen; das Objekt ändert die Daten direkt an Ort und Stelle.

Den Ort einer Inventar-Referenz ermittelst du so:

```
local location = inv:get_location()
```

7.3.2 Spielerinventare

Spielerinventare erhältst du auf ähnliche Weise oder über eine Spieler-Referenz. Der Spieler muss online sein, damit du auf sein Inventar zugreifen kannst.

```
local inv = core.get_inventory({
    type = "player",
    name = "player1",
})
-- oder
local inv = player:get_inventory()
```

7.3.3 Freistehende Inventare

Ein freistehendes Inventar (*detached inventory*) ist unabhängig von Spielern und Nodes. Es wird bei einem Neustart nicht gespeichert.

```
local inv = core.get_inventory({
    type = "detached",
    name = "inventory_name",
})
```

Anders als die anderen Inventararten musst du ein freistehendes Inventar erst anlegen, bevor du darauf zugreifst:

```
core.create_detached_inventory("inventory_name")
```

`create_detached_inventory` nimmt drei Argumente entgegen, von denen nur das erste – der Inventarname – Pflicht ist. Das zweite ist eine Tabelle mit Callbacks, mit denen du steuerst, wie Spieler mit dem Inventar umgehen können:

```
-- Freistehendes Inventar, in das man nur hineinlegen kann
core.create_detached_inventory("inventory_name", {
    allow_move = function(
        inv,
        from_list,
        from_index,
        to_list,
        to_index,
        count,
        player
    )
        return count -- Verschieben erlauben
    end,

    allow_put = function(
        inv,
        listname,
        index,
        stack,
        player
    )
        return stack:get_count() -- Hineinlegen erlauben
    end,

    allow_take = function(
        inv,
        listname,
        index,
        stack,
        player
    )
        return 0 -- Herausnehmen verbieten
    end,

    on_put = function(inv, listname, index, stack, player)
        core.chat_send_all(
            player:get_player_name()
                .. " hat "
                .. stack:to_string()
                .. " in die Spendenkiste gelegt, von "
```

```
                .. core.pos_to_string(player:get_pos())
        )
    end,
})
```

Berechtigungs-Callbacks – also die mit `allow_` am Anfang – geben die Anzahl der zu übertragenden Items zurück; 0 verhindert die Übertragung ganz.

Aktions-Callbacks – die mit `on_` am Anfang – haben dagegen keinen Rückgabewert.

7.4 Listen

Mit Inventarlisten lassen sich mehrere Gitter an einem einzigen Ort speichern. Das ist vor allem für Spieler nützlich, denn es gibt einige Listen, die alle Spiele haben, etwa das Hauptinventar (`main`) und die Craftingfelder (`craft`).

7.4.1 Größe und Breite

Listen haben eine Größe, also die Gesamtzahl der Felder, und eine Breite, die nur intern in der Engine verwendet wird. Beim Anzeigen des Inventars in einem Fenster spielt die Breite der Liste keine Rolle, weil der Code des Fensters die Breite selbst festlegt.

```
if inv:set_size("main", 32) then
    inv:set_width("main", 8)
    print("Größe:  " .. inv:get_size("main"))
    print("Breite: " .. inv:get_width("main"))
else
    print("Fehler: ungültige Liste oder Größe")
end
```

`set_size` schlägt fehl und gibt `false` zurück, wenn der Listenname oder die Größe ungültig ist – etwa wenn die neue Größe zu klein für die Items ist, die gerade im Inventar liegen.

7.4.2 Inhalt prüfen

Mit `is_empty` prüfst du, ob eine Liste überhaupt Items enthält:

```
if inv:is_empty("main") then
    print("Die Liste ist leer!")
end
```

Mit `contains_item` prüfst du, ob eine Liste ein bestimmtes Item enthält:

```
if inv:contains_item("main", "default:stone") then
    print("Ich habe Stein gefunden!")
end
```

7.5 Inventare und ItemStacks verändern

7.5.1 Zu einer Liste hinzufügen

`add_item` fügt Items zu einer Liste hinzu (hier `"main"`). Im folgenden Beispiel wird dabei auch die maximale Stapelgröße beachtet:

```
local stack = ItemStack("default:stone 99")
local leftover = inv:add_item("main", stack)
if leftover:get_count() > 0 then
    print(
        "Inventar ist voll! "
            .. leftover:get_count()
            .. " Items wurden nicht hinzugefügt"
    )
end
```

7.5.2 Items herausnehmen

So entfernst du Items aus einer Liste:

```
local taken = inv:remove_item("main", stack)
print("Herausgenommen: " .. taken:get_count())
```

7.5.3 Stacks verändern

Einzelne Stacks veränderst du, indem du sie zuerst holst:

```
local stack = inv:get_stack(listname, 1)
```

Dann änderst du sie, indem du Eigenschaften setzt oder Methoden verwendest, die die Stapelgröße beachten:

```
local stack = ItemStack("default:stone 50")
local to_add = ItemStack("default:stone 100")
local leftover = stack:add_item(to_add)
local taken = stack:take_item(19)

print(
    "Konnte "
        .. leftover:get_count()
        .. " Items nicht hinzufügen."
)
-- ^ ergibt 51

print("Habe " .. stack:get_count() .. " Items")
-- ^ ergibt 80
--   min(50+100, stack_max) - 19 = 80
--     mit stack_max = 99
```

`add_item` fügt einem ItemStack Items hinzu und gibt alles zurück, was nicht hineingepasst hat. `take_item` nimmt höchstens die angegebene Anzahl heraus, eventuell auch weniger, und gibt den herausgenommenen Stack zurück.

Zum Schluss setzt du den ItemStack wieder ein:

```
inv:set_stack(listname, 1, stack)
```

Die Felder einer Liste werden, wie in Lua üblich, ab 1 gezählt.

7.6 Abnutzung

Werkzeuge können sich abnutzen. Die Abnutzung wird als Balken angezeigt, und ist das Werkzeug vollständig abgenutzt, zerbricht es. Die Abnutzung ist eine Zahl bis 65535; je höher sie ist, desto stärker ist das Werkzeug abgenutzt.

Du veränderst sie mit `add_wear()`, `get_wear()` und `set_wear(wear)`.

```
local stack = ItemStack("default:pick_mese")
local max_uses = 10

-- Passiert automatisch beim Abbauen mit einem Werkzeug:
-- erhöht die Abnutzung um eine Benutzung.
stack:add_wear(65535 / (max_uses - 1))
```

Beim Abbauen kann die Abnutzung eines Werkzeugs vom abgebauten Node abhängen; `max_uses` ist also je nach Node verschieden.

7.7 Lua-Tabellen

ItemStacks und Inventare lassen sich in Tabellen umwandeln und umgekehrt. Das ist praktisch zum Kopieren und für Massenoperationen.

```
-- Ganzes Inventar
local data = inv1:get_lists()
inv2:set_lists(data)

-- Eine Liste
local listdata = inv1:get_list("main")
inv2:set_list("main", listdata)
```

Die von `get_lists()` zurückgegebene Tabelle sieht so aus:

```
{
    list_one = {
        ItemStack,
        ItemStack,
        ItemStack,
        ItemStack,
```

```
        -- inv:get_size("list_one") Elemente
    },
    list_two = {
        ItemStack,
        ItemStack,
        ItemStack,
        ItemStack,
        -- inv:get_size("list_two") Elemente
    },
},
```

`get_list()` gibt eine einzelne Liste zurück, also einfach eine Liste von ItemStacks.

Wichtig: Die `set`-Methoden oben verändern die Größe der Listen nicht. Du kannst eine Liste also leeren, indem du sie auf eine leere Tabelle setzt, ohne dass sie kleiner wird:

```
inv:set_list("main", {})
```

Teil III

Karte

8 Grundlegende Kartenoperationen

In diesem Kapitel lernst du grundlegende Operationen auf der Karte kennen, etwa Nodes hinzufügen, entfernen und finden.

8.1 Aufbau der Karte

Die Luanti-Karte ist in Mapblöcke aufgeteilt; jeder Mapblock ist ein Würfel mit der Kantenlänge 16. Während Spieler sich über die Karte bewegen, werden Mapblöcke erzeugt, geladen, aktiviert und wieder entladen. Noch nicht geladene Bereiche der Karte sind mit *ignore*-Nodes gefüllt, einem unpassierbaren, nicht auswählbaren Platzhalter. Leerer Raum ist mit *air*-Nodes gefüllt, unsichtbaren Nodes, durch die man hindurchgehen kann.

Ein aktiver Mapblock ist einer, der geladen ist und auf dem Aktualisierungen laufen.

Geladene Mapblöcke werden oft *aktive Blöcke* genannt. Mods und Spieler können aus aktiven Blöcken lesen und in sie schreiben, und sie enthalten aktive Entities. Auch die Engine führt Operationen auf der Karte aus, etwa die Flüssigkeitsphysik.

Mapblöcke werden entweder aus der Weltdatenbank geladen oder neu generiert. Generiert werden sie bis zur Kartengenerierungsgrenze (`mapgen_limit`), die standardmäßig auf ihren Höchstwert 31000 eingestellt ist. Bereits vorhandene Mapblöcke können aber auch jenseits dieser Grenze aus der Weltdatenbank geladen werden.

8.2 Lesen

8.2.1 Nodes lesen

Sobald du eine Position hast, kannst du aus der Karte lesen:

```
local node = core.get_node({ x = 1, y = 3, z = 4 })
print(dump(node)) --> { name=.., param1=.., param2=.. }
```

Ist die Position keine ganze Zahl, wird sie auf den Node gerundet, in dem sie liegt. Die Funktion gibt immer eine Tabelle mit den Node-Informationen zurück:

- `name` – der Node-Name; *ignore*, wenn der Bereich nicht geladen ist.
- `param1` – siehe Node-Definition; meist die Beleuchtung.
- `param2` – siehe Node-Definition.

Beachte, dass die Funktion einen inaktiven Block nicht lädt, sondern eine Tabelle mit dem Namen `ignore` zurückgibt.

Stattdessen kannst du `core.get_node_or_nil` verwenden; sie gibt dann `nil` zurück statt einer Tabelle mit dem Namen `ignore`. Auch sie lädt den Block nicht. Sie kann trotzdem `ignore` zurückgeben, wenn ein Block tatsächlich *ignore* enthält – das passiert am Rand der Karte, wie ihn die Kartengenerierungsgrenze (`mapgen_limit`) festlegt.

8.2.2 Nodes finden

Luanti bietet eine Reihe von Hilfsfunktionen, die häufige Kartenoperationen beschleunigen. Am häufigsten werden die Funktionen zum Finden von Nodes verwendet.

Angenommen, du möchtest eine Pflanze erstellen, die in der Nähe von Mese besser wächst. Dann musst du nach Mese-Nodes in der Umgebung suchen und die Wachstumsrate entsprechend anpassen.

`core.find_node_near` gibt den ersten gefundenen Node in einem bestimmten Radius zurück, der zu den angegebenen Node-Namen oder Gruppen passt. Im folgenden Beispiel suchen wir einen Mese-Node im Umkreis von 5 Nodes um die Position:

```
local grow_speed = 1
local node_pos =
    core.find_node_near(pos, 5, { "default:mese" })
if node_pos then
    core.chat_send_all(
        "Node gefunden bei: " .. dump(node_pos)
    )
    grow_speed = 2
end
```

Angenommen, die Wachstumsrate soll umso höher sein, je mehr Mese in der Nähe ist. Dann brauchst du eine Funktion, die mehrere Nodes in einem Bereich findet:

```
local pos1 = vector.subtract(pos, { x = 5, y = 5, z = 5 })
local pos2 = vector.add(pos, { x = 5, y = 5, z = 5 })
local pos_list =
    core.find_nodes_in_area(pos1, pos2, { "default:mese" })
local grow_speed = 1 + #pos_list
```

Dieser Code zählt die Nodes in einem *Quader*. Das unterscheidet sich von `find_node_near`, das den Abstand zur Position verwendet (also eine *Kugel*). Um das zu korrigieren, müssen wir den Abstand selbst prüfen:

```
local pos1 = vector.subtract(pos, { x = 5, y = 5, z = 5 })
local pos2 = vector.add(pos, { x = 5, y = 5, z = 5 })
local pos_list =
    core.find_nodes_in_area(pos1, pos2, { "default:mese" })
local grow_speed = 1
for i = 1, #pos_list do
    local delta = vector.subtract(pos_list[i], pos)
    if
        delta.x * delta.x
            + delta.y * delta.y
            + delta.z * delta.z
        <= 5 * 5
    then
        grow_speed = grow_speed + 1
    end
end
```

Jetzt erhöht der Code `grow_speed` korrekt anhand der Mese-Nodes im Umkreis.

Beachte, dass wir den *quadrierten* Abstand vergleichen, statt die Wurzel zu ziehen und den echten Abstand zu berechnen. Wurzeln sind für Computer rechenaufwendig und sollten daher möglichst vermieden werden.

Es gibt weitere Varianten der beiden Funktionen, etwa `find_nodes_with_meta` und `find_nodes_in_area_under_air`. Sie funktionieren ähnlich und sind in anderen Situationen nützlich.

8.3 Schreiben

8.3.1 Nodes schreiben

Mit `set_node` schreibst du in die Karte. Jeder Aufruf von `set_node` führt dazu, dass die Beleuchtung neu berechnet wird und Node-Callbacks laufen. Für viele Nodes ist `set_node` deshalb recht langsam.

```
core.set_node(
    { x = 1, y = 3, z = 4 },
    { name = "default:mese" }
)

local node = core.get_node({ x = 1, y = 3, z = 4 })
print(node.name) --> default:mese
```

`set_node` entfernt alle Metadaten und das Inventar an dieser Position. Das ist nicht immer erwünscht, besonders wenn du mehrere Node-Definitionen für einen einzigen gedachten Node verwendest. Ein Beispiel ist der Ofen: Man denkt sich ihn als einen Node, tatsächlich sind es aber zwei.

So setzt du einen Node, ohne Metadaten und Inventar zu löschen:

```
core.swap_node(
    { x = 1, y = 3, z = 4 },
    { name = "default:mese" }
)
```

8.3.2 Nodes entfernen

An jeder Position muss ein Node sein. Um einen Node zu entfernen, setzt du die Position auf `air`.

Die folgenden beiden Zeilen entfernen beide einen Node und sind gleichwertig:

```
core.remove_node(pos)
core.set_node(pos, { name = "air" })
```

`remove_node` ist tatsächlich nur eine Hilfsfunktion, die `set_node` mit `"air"` aufruft.

8.4 Mapblöcke laden

Mit `core.emerge_area` lädst du Mapblöcke. Das geschieht asynchron, die Blöcke sind also nicht sofort geladen. Sie werden in Kürze geladen, und für jeden Block wird der Callback aufgerufen.

```
-- Einen Bereich von 20×20×20 laden
local halfsize = { x = 10, y = 10, z = 10 }
local pos1 = vector.subtract(pos, halfsize)
local pos2 = vector.add(pos, halfsize)

local context = {} -- Daten zwischen den Callback-Aufrufen
core.emerge_area(pos1, pos2, emerge_callback, context)
```

Luanti ruft `emerge_callback` jedes Mal auf, wenn ein Block geladen wurde, und übergibt dabei Informationen zum Fortschritt. Die Funktion muss definiert sein, bevor `emerge_area` aufgerufen wird:

```
local function emerge_callback(
    pos,
    action,
    num_calls_remaining,
    context
)
    -- Beim ersten Aufruf die Anzahl der Blöcke merken
    if not context.total_blocks then
        context.total_blocks = num_calls_remaining + 1
        context.loaded_blocks = 0
    end

    -- Anzahl der geladenen Blöcke erhöhen
    context.loaded_blocks = context.loaded_blocks + 1

    -- Fortschritt melden
    if context.total_blocks == context.loaded_blocks then
        core.chat_send_all("Alle Blöcke geladen!")
    else
        local perc = 100
            * context.loaded_blocks
            / context.total_blocks
        local msg = string.format(
            "Lade Blöcke %d/%d (%.2f%%)",
            context.loaded_blocks,
```

```
            context.total_blocks,
            perc
        )
        core.chat_send_all(msg)
    end
end
```

Das ist nicht die einzige Möglichkeit, Blöcke zu laden: Auch ein Lua Voxel Manipulator (LVM) (Kapitel 18) lädt die betroffenen Blöcke, und zwar synchron.

8.5 Mapblöcke löschen

Mit `core.delete_area` löschst du einen Bereich von Mapblöcken:

```
-- Einen Bereich von 20×20×20 löschen
local halfsize = { x = 10, y = 10, z = 10 }
local pos1 = vector.subtract(pos, halfsize)
local pos2 = vector.add(pos, halfsize)

core.delete_area(pos1, pos2)
```

Dabei werden alle Mapblöcke in diesem Bereich gelöscht, und zwar *einschließlich* der Ränder. Es werden also auch Nodes außerhalb des Bereichs gelöscht, wenn sie in einem Mapblock liegen, der über die Bereichsgrenzen hinausragt.

9 Node-Timer und ABMs

Eine Funktion regelmäßig auf bestimmten Nodes auszuführen, ist eine häufige Aufgabe. Luanti bietet dafür zwei Wege: *Active Block Modifiers* (ABMs) und Node-Timer.

ABMs durchsuchen alle geladenen Mapblöcke nach Nodes, die bestimmte Kriterien erfüllen. Sie eignen sich am besten für Nodes, die in der Welt häufig vorkommen, etwa Gras. Sie kosten viel Rechenzeit, aber wenig Arbeits- und Speicherplatz.

Für seltene Nodes oder solche, die ohnehin Metadaten verwenden, etwa Öfen und Maschinen, solltest du stattdessen Node-Timer verwenden. Node-Timer merken sich die anstehenden Timer in jedem Mapblock und führen sie aus, sobald sie ablaufen. Sie müssen also nicht alle geladenen Nodes nach Treffern durchsuchen, brauchen dafür aber etwas mehr Arbeits- und Speicherplatz, um die anstehenden Timer zu verwalten.

9.1 Node-Timer

Node-Timer sind direkt an einen einzelnen Node gebunden. Du steuerst sie über ein `NodeTimerRef`-Objekt:

```
local timer = core.get_node_timer(pos)
timer:start(10.5) -- in Sekunden
```

Läuft ein Node-Timer ab, wird die Methode `on_timer` in der Definitionstabelle des Nodes aufgerufen. Sie bekommt nur einen Parameter, die Position des Nodes:

```
core.register_node("autodoors:door_open", {
    on_timer = function(pos)
        core.set_node(pos, { name = "autodoors:door" })
        return false
    end,
})
```

Gibt `on_timer` `true` zurück, läuft der Timer mit demselben Intervall erneut. Du kannst auch innerhalb von `on_timer` `get_node_timer(pos)` verwenden; gib dann aber `false` zurück, um Konflikte zu vermeiden.

Timer haben eine Einschränkung: Aus Optimierungsgründen gibt es je Node-Typ nur eine Art von Timer, und je Node kann nur ein Timer laufen.

9.2 Active Block Modifiers

Alien-Gras ist für dieses Kapitel eine Grasart, die mit einer gewissen Wahrscheinlichkeit in der Nähe von Wasser erscheint.

```
core.register_node("aliens:grass", {
    description = "Alien-Gras",
    light_source = 3, -- Der Node leuchtet; min. 0, max. 14
    tiles = { "aliens_grass.png" },
    groups = { choppy = 1 },
    on_use = core.item_eat(20),
})

core.register_abm({
    nodenames = { "default:dirt_with_grass" },
    neighbors = {
        "default:water_source",
        "default:water_flowing",
    },
    interval = 10.0, -- alle 10 Sekunden ausführen
    chance = 50, -- jeder Node wird mit Chance 1:50 gewählt
    action = function(
        pos,
        node,
        active_object_count,
        active_object_count_wider
    )
        local above =
            { x = pos.x, y = pos.y + 1, z = pos.z }
        core.set_node(above, { name = "aliens:grass" })
    end,
})
```

Dieser ABM läuft alle zehn Sekunden, und für jeden passenden Node wird er mit einer Wahrscheinlichkeit von 1 zu 50 ausgeführt. Läuft der ABM auf einem Node, wird darüber ein Alien-Gras-Node gesetzt. Achtung: Dabei wird jeder

Node gelöscht, der sich vorher an dieser Stelle befand. Um das zu verhindern, solltest du mit `core.get_node` prüfen, ob dort Platz für das Gras ist.

Die Angabe von Nachbarn (`neighbors`) ist optional. Gibst du mehrere Nachbarn an, genügt es, wenn einer davon vorhanden ist.

Auch `chance` ist optional. Ohne Angabe läuft der ABM immer, wenn die übrigen Bedingungen erfüllt sind.

9.3 Jetzt bist du dran

- Midas-Berührung: Lass Wasser alle 5 Sekunden mit einer Wahrscheinlichkeit von 1 zu 100 zu Goldblöcken werden.
- Verfall: Lass Holz zu Erde werden, wenn Wasser angrenzt.
- Lauffeuer: Lass jeden Luft-Node in Flammen aufgehen. (Tipp: `"air"` und `"fire:basic_flame"`.) Warnung: Rechne damit, dass das Spiel abstürzt.

10 Speicherung und Metadaten

In diesem Kapitel lernst du, wie du Daten speichern kannst.

10.1 Metadaten

10.1.1 Was sind Metadaten?

Metadaten sind in Luanti ein Schlüssel-Wert-Speicher, mit dem du eigene Daten an etwas anhängst. Du kannst damit Informationen zu einem Node, einem Spieler oder einem ItemStack speichern.

Alle Arten von Metadaten verwenden genau dieselbe API. Metadaten speichern Werte als Strings, es gibt aber eine Reihe von Methoden, um andere einfache Datentypen umzuwandeln und zu speichern.

Manche Schlüssel haben eine besondere Bedeutung. So speichert `infotext` in Node-Metadaten den Tooltip, der erscheint, wenn man mit dem Fadenkreuz auf den Node zeigt. Um Konflikte mit anderen Mods zu vermeiden, solltest du für Schlüssel die übliche Namensraum-Konvention verwenden: `modname:keyname`. Ausnahme sind gebräuchliche Angaben wie der Name des Besitzers, der als `owner` gespeichert wird.

Metadaten sind Daten über Daten. Die Daten selbst, etwa der Typ eines Nodes oder die Anzahl in einem Stack, sind keine Metadaten.

10.1.2 Ein Metadaten-Objekt holen

Kennst du die Position eines Nodes, kannst du seine Metadaten abrufen:

```
local meta = core.get_meta({ x = 1, y = 2, z = 3 })
```

Spieler- und ItemStack-Metadaten erhältst du mit `get_meta()`:

```
local pmeta = player:get_meta()
local imeta = stack:get_meta()
```

10.1.3 Lesen und Schreiben

Meist liest und schreibst du Metadaten mit den Methoden `get_<typ>()` und `set_<typ>()`. Da Metadaten Strings speichern, setzen und lesen die String-Methoden den Wert direkt.

```
print(meta:get_string("foo")) --> ""
meta:set_string("foo", "bar")
print(meta:get_string("foo")) --> "bar"
```

Alle typisierten Getter geben einen neutralen Standardwert zurück, wenn der Schlüssel nicht existiert, etwa `""` oder `0`. Mit `get()` erhältst du einen String oder `nil`.

Metadaten-Objekte sind Referenzen; Änderungen landen also automatisch an der Quelle. ItemStacks sind dagegen keine Referenzen; nach einer Änderung musst du den ItemStack wieder ins Inventar setzen.

Die Getter und Setter für andere Typen wandeln von und nach String um:

```
print(meta:get_int("count")) --> 0
meta:set_int("count", 3)
print(meta:get_int("count")) --> 3
print(meta:get_string("count")) --> "3"
```

10.1.4 Besondere Schlüssel

`infotext` zeigt in Node-Metadaten einen Tooltip an, wenn das Fadenkreuz auf den Node zeigt. Das ist nützlich, um Besitzer oder Zustand eines Nodes anzuzeigen.

`description` überschreibt in ItemStack-Metadaten die Beschreibung, die beim Überfahren des Stacks im Inventar erscheint. Farben kannst du mit `core.colorize()` einfügen.

`owner` ist ein gebräuchlicher Schlüssel für den Namen des Spielers, dem das Item oder der Node gehört.

10.1.5 Tabellen speichern

Tabellen müssen in Strings umgewandelt werden, bevor du sie speichern kannst. Luanti bietet dafür zwei Formate: Lua und JSON.

Das Lua-Format ist meist deutlich schneller und entspricht der Tabellenschreibweise von Lua. JSON ist ein verbreiteterer Standard, besser strukturiert und gut geeignet, wenn du Daten mit einem anderen Programm austauschen musst. Für JSON gibt es `core.write_json()` und `core.parse_json()`.

```
local data = { username = "player1", score = 1234 }
meta:set_string("foo", core.serialize(data))

data = core.deserialize(meta:get_string("foo"))
```

10.1.6 Private Metadaten

Standardmäßig werden alle Node-Metadaten an den Client geschickt. Um das zu verhindern, kannst du Schlüssel als privat markieren:

```
meta:set_string("secret", "asd34dn")
meta:mark_as_private("secret")
```

10.1.7 Lua-Tabellen

Mit `to_table` und `from_table` wandelst du Metadaten in Lua-Tabellen um und zurück:

```
local tmp = meta:to_table()
tmp.foo = "bar"
meta:from_table(tmp)
```

10.2 Mod-Speicher

Der Mod-Speicher (*mod storage*) verwendet genau dieselbe API wie Metadaten, auch wenn er technisch gesehen keine Metadaten sind. Er gehört jeweils zu einem Mod und kann nur während der Ladezeit abgerufen werden, damit Luanti weiß, welcher Mod ihn anfordert.

```
local storage = core.get_mod_storage()
```

Jetzt kannst du den Speicher genau wie Metadaten verwenden:

```
storage:set_string("foo", "bar")
```

10.3 Datenbanken

Wird ein Mod wahrscheinlich auf Servern eingesetzt und speichert er viele Daten, lohnt es sich, eine Datenbank als Speicher anzubieten. Mach das optional, indem du trennst, *wie* die Daten gespeichert werden und *wo* sie verwendet werden.

```
local backend
if use_database then
    backend = dofile(
        core.get_modpath("mymod") .. "/backend_sqlite.lua"
    )
else
    backend = dofile(
        core.get_modpath("mymod") .. "/backend_storage.lua"
    )
end

backend.get_foo("a")
backend.set_foo("a", { score = 3 })
```

Die Datei `backend_storage.lua` enthält dann eine Umsetzung mit dem Mod-Speicher:

```
local storage = core.get_mod_storage()
local backend = {}

function backend.set_foo(key, value)
    storage:set_string(key, core.serialize(value))
end

function backend.get_foo(key)
```

```
    return core.deserialize(storage:get_string(key))
end

return backend
```

`backend_sqlite.lua` macht dasselbe, verwendet aber statt des Mod-Speichers die Lua-Bibliothek *lsqlite3*.

Für eine Datenbank wie SQLite brauchst du eine unsichere Umgebung (*insecure environment*). Das ist eine Tabelle, die nur Mods zur Verfügung steht, die der Benutzer ausdrücklich freigegeben hat. Sie enthält eine weniger eingeschränkte Kopie der Lua-API, die von bösartigen Mods missbraucht werden könnte. Mehr dazu im Kapitel Sicherheit (Kapitel 23).

```
local ie = core.request_insecure_environment()
if not ie then
    error("Bitte mymod zu secure.trusted_mods hinzufügen")
end

local _sql = ie.require("lsqlite3")
-- globale sqlite3-Bibliothek vor anderen Mods verbergen
if sqlite3 then
    sqlite3 = nil
end
```

SQL oder die Bibliothek lsqlite3 zu erklären, würde den Rahmen dieses Buches sprengen.

10.4 Was soll ich verwenden?

Welche Methode passt, hängt davon ab, worauf sich die Daten beziehen, wie sie aufgebaut sind und wie groß sie sind. Als Faustregel: Kleine Datenmengen gehen bis 10 KB, mittlere bis 10 MB, alles darüber ist groß.

Node-Metadaten sind eine gute Wahl für Daten, die zu einem Node gehören. Mittlere Datenmengen lassen sich recht effizient speichern, wenn du sie privat markierst.

Item-Metadaten solltest du nur für kleine Datenmengen verwenden, denn sie werden immer an den Client geschickt. Außerdem werden die Daten jedes Mal kopiert, wenn der Stack bewegt oder aus Lua darauf zugegriffen wird.

Der Mod-Speicher eignet sich für mittlere Datenmengen; große Datenmengen zu schreiben, kann aber ineffizient sein. Für große Datenmengen nimmst du besser eine Datenbank, damit nicht bei jedem Speichern alle Daten geschrieben werden müssen.

Datenbanken kommen nur für Server in Frage, weil der Mod für die unsichere Umgebung freigegeben werden muss. Für große Datenmengen sind sie gut geeignet.

10.5 Jetzt bist du dran

- Erstelle einen Node, der verschwindet, nachdem er fünfmal geschlagen wurde. (Verwende `on_punch` in der Node-Definition und `core.set_node`.)

11 Objekte, Spieler und Entities

In diesem Kapitel lernst du, wie du Objekte veränderst und eigene definierst.

11.1 Was sind Objekte, Spieler und Entities?

Spieler und Entities sind beides Objekte. Ein Objekt ist etwas, das sich unabhängig vom Node-Gitter bewegen kann und Eigenschaften wie Geschwindigkeit und Größe hat. Objekte sind keine Items und haben ein eigenes Registrierungssystem.

Zwischen Spielern und Entities gibt es einige Unterschiede. Der wichtigste: Spieler werden von Menschen gesteuert, Entities von Mods. Deshalb können Mods die Geschwindigkeit eines Spielers nicht setzen – Spieler werden auf dem Client berechnet, Entities auf dem Server. Ein weiterer Unterschied: Spieler sorgen dafür, dass Mapblöcke geladen werden, Entities werden dagegen einfach gespeichert und inaktiv.

Verwischt wird diese Unterscheidung dadurch, dass Entities über eine Tabelle gesteuert werden, die *Lua-Entity* heißt; dazu später mehr.

11.2 Position und Geschwindigkeit

Mit `get_pos` und `set_pos` liest und setzt du die Position eines Objekts.

```
local object = core.get_player_by_name("bob")
local pos = object:get_pos()
object:set_pos({ x = pos.x, y = pos.y + 1, z = pos.z })
```

`set_pos` setzt die Position sofort, ohne Animation. Soll das Objekt sich fließend an die neue Position bewegen, verwendest du `move_to`. Das funktioniert leider nur bei Entities.

```
object:move_to({ x = pos.x, y = pos.y + 1, z = pos.z })
```

Bei Entities musst du immer an die Netzwerklatenz denken. Im Idealfall kämen Nachrichten über Bewegungen sofort an, in der richtigen Reihenfolge und in ähnlichen Abständen, wie du sie gesendet hast. Außerhalb des Einzelspielermodus ist die Welt aber nicht ideal. Nachrichten brauchen eine Weile. Positionsnachrichten können in falscher Reihenfolge ankommen, sodass manche `set_pos`-Aufrufe übersprungen werden – es hat keinen Sinn, an eine Position zu springen, die älter ist als die aktuell bekannte. Bewegungen kommen womöglich nicht in gleichmäßigen Abständen an, was sie für Animationen schwer nutzbar macht. All das führt dazu, dass der Client etwas anderes sieht als der Server. Dessen solltest du dir bewusst sein.

11.3 Objekteigenschaften

Objekteigenschaften sagen dem Client, wie er ein Objekt darstellen und behandeln soll. Eigene Eigenschaften kannst du nicht festlegen, denn die Eigenschaften sind per Definition für die Engine bestimmt.

Anders als Nodes haben Objekte kein festes, sondern ein veränderliches Aussehen. Wie ein Objekt aussieht – und einiges mehr –, kannst du jederzeit ändern, indem du seine Eigenschaften aktualisierst.

```
object:set_properties({
    visual = "mesh",
    mesh = "character.b3d",
    textures = { "character_texture.png" },
    visual_size = { x = 1, y = 1 },
})
```

Die geänderten Eigenschaften werden an alle Spieler in Reichweite geschickt. So erreichst du sehr günstig viel Abwechslung, etwa unterschiedliche Skins für jeden Spieler.

Wie der nächste Abschnitt zeigt, können Entities in ihrer Definition Anfangseigenschaften haben. Die Standardeigenschaften von Spielern sind dagegen in der Engine festgelegt; für neu beigetretene Spieler setzt du die Eigenschaften deshalb mit `set_properties()` in `on_joinplayer`.

11.4 Entities

Eine Entity hat eine Definitionstabelle, die einer Item-Definition ähnelt. Sie kann Callback-Methoden, Anfangseigenschaften des Objekts und eigene Felder

enthalten.

```
local MyEntity = {
    initial_properties = {
        hp_max = 1,
        physical = true,
        collide_with_objects = false,
        collisionbox = { -0.3, -0.3, -0.3, 0.3, 0.3, 0.3 },
        visual = "wielditem",
        visual_size = { x = 0.4, y = 0.4 },
        textures = { "" },
        spritediv = { x = 1, y = 1 },
        initial_sprite_basepos = { x = 0, y = 0 },
    },

    message = "Standardnachricht",
}

function MyEntity:set_message(msg)
    self.message = msg
end
```

In einem sehr wichtigen Punkt unterscheiden sich Entity-Definitionen von Item-Definitionen: Wird eine Entity *emerged* (also geladen oder erzeugt), wird für sie eine neue Tabelle angelegt, die von der Definitionstabelle *erbt*.

Über das ObjectRef kommst du an die Entity-Tabelle und umgekehrt:

```
local entity = object:get_luaentity()
local object = entity.object
print(
    "Entity ist bei "
        .. core.pos_to_string(object:get_pos())
)
```

Für Entities gibt es eine Reihe von Callbacks. Eine vollständige Liste findest du in der Lua-API-Referenz[1].

[1] https://api.luanti.org/minetest-namespace-reference/#registered-definition-tables

```
function MyEntity:on_step(dtime)
    local pos = self.object:get_pos()
    local pos_down =
        vector.subtract(pos, vector.new(0, 1, 0))

    local delta
    if core.get_node(pos_down).name == "air" then
        delta = vector.new(0, -1, 0)
    elseif core.get_node(pos).name == "air" then
        delta = vector.new(0, 0, 1)
    else
        delta = vector.new(0, 1, 0)
    end

    delta = vector.multiply(delta, dtime)

    self.object:move_to(vector.add(pos, delta))
end

function MyEntity:on_punch(hitter)
    core.chat_send_player(
        hitter:get_player_name(),
        self.message
    )
end
```

Wenn du diese Entity erzeugst und benutzt, fällt dir auf, dass die Nachricht vergessen wird, sobald die Entity inaktiv und wieder aktiv wird. Das liegt daran, dass die Nachricht nicht gespeichert wird. Statt einfach die ganze Entity-Tabelle zu speichern, überlässt Luanti dir, was gespeichert wird. *Staticdata* ist ein String mit allen eigenen Informationen, die gespeichert werden müssen.

```
function MyEntity:get_staticdata()
    return core.write_json({
        message = self.message,
    })
end

function MyEntity:on_activate(staticdata, dtime_s)
    if staticdata ~= "" and staticdata ~= nil then
        local data = core.parse_json(staticdata) or {}
        self:set_message(data.message)
    end
end
```

Luanti darf `get_staticdata()` beliebig oft und zu jedem Zeitpunkt aufrufen. Luanti wartet nämlich nicht, bis ein Mapblock inaktiv wird, um ihn zu speichern, denn das könnte zu Datenverlust führen. Mapblöcke werden ungefähr alle 18 Sekunden gespeichert; in ähnlichen Abständen wird also auch `get_staticdata()` aufgerufen.

`on_activate()` wird dagegen nur aufgerufen, wenn eine Entity aktiv wird – entweder weil ihr Mapblock aktiv wird oder weil sie neu erzeugt wird. Staticdata kann deshalb leer sein.

Zum Schluss registrierst du die Typ-Tabelle mit der passend benannten Funktion `register_entity`.

```
core.register_entity("mymod:entity", MyEntity)
```

Ein Mod kann die Entity so erzeugen:

```
local pos = { x = 1, y = 2, z = 3 }
local obj = core.add_entity(pos, "mymod:entity", nil)
```

Der dritte Parameter sind die anfänglichen Staticdata. Die Nachricht setzt du über die Methode der Entity-Tabelle:

```
obj:get_luaentity():set_message("Hallo!")
```

Spieler mit dem Privileg (Kapitel 12) *give* können Entities mit einem Chatbefehl (Kapitel 13) erzeugen:

```
/spawnentity mymod:entity
```

11.5 Lebenspunkte und Schaden

11.5.1 Lebenspunkte (HP)

Jedes Objekt hat einen Wert für Lebenspunkte (*health points*, HP), der seine aktuelle Gesundheit angibt. Die maximalen HP legt die Objekteigenschaft `hp_max` fest. Fallen die HP auf 0, stirbt das Objekt.

```
local hp = object:get_hp()
object:set_hp(hp + 3)
```

11.5.2 Schläge, Schadensgruppen und Rüstungsgruppen

Schaden verringert die HP eines Objekts. Ein Objekt kann ein anderes *schlagen* (*punch*), um ihm Schaden zuzufügen. Ein Schlag muss kein echter Schlag sein – es kann auch eine Explosion, ein Schwerthieb oder etwas anderes sein.

Der Gesamtschaden ergibt sich, indem die Schadensgruppen des Schlags mit der Verwundbarkeit des Ziels multipliziert werden. Das Ergebnis wird dann begrenzt, je nachdem, wie kurz der letzte Schlag zurückliegt. Gleich rechnen wir das an einem Beispiel durch.

Wie die Abbautypen von Nodes (Kapitel 3) können diese Gruppen beliebige Namen haben und müssen nicht registriert werden. Üblich ist aber, dieselben Gruppennamen wie beim Abbauen von Nodes zu verwenden.

Wie verwundbar ein Objekt für bestimmte Schadensarten ist, hängt von seinen `armor_groups` ab. Trotz des irreführenden Namens geben `armor_groups` an, wie viel Prozent Schaden das Objekt von einer Schadensgruppe erleidet – nicht, wie viel es abwehrt. Fehlt eine Schadensgruppe in den Rüstungsgruppen eines Objekts, ist es gegen diese völlig unverwundbar.

```
target:set_armor_groups({
    fleshy = 90,
    crumbly = 50,
})
```

Im Beispiel erleidet das Objekt 90 % des `fleshy`-Schadens und 50 % des `crumbly`-Schadens.

Schlägt ein Spieler ein Objekt, kommen die Schadensgruppen von dem Item, das er gerade in der Hand hält. In anderen Fällen legen Mods fest, welche Schadensgruppen verwendet werden.

11.5.3 Beispiel für eine Schadensberechnung

Schlagen wir das Objekt `target`:

```
local tool_capabilities = {
    full_punch_interval = 0.8,
    damage_groups = { fleshy = 5, choppy = 10 },

    -- Nur fürs Abbauen von Nodes, aber trotzdem nötig
    max_drop_level = 1,
    groupcaps = {
        fleshy = {
            times = { [1] = 2.5, [2] = 1.20, [3] = 0.35 },
            uses = 30,
            maxlevel = 2,
        },
    },
}

local time_since_last_punch =
    tool_capabilities.full_punch_interval
target:punch(
    object,
    time_since_last_punch,
    tool_capabilities
)
```

Jetzt berechnen wir den Schaden. Die Schadensgruppen des Schlags sind `fleshy=5` und `choppy=10`; `target` erleidet 90 % des `fleshy`-Schadens und 0 % des `choppy`-Schadens.

Zuerst multiplizieren wir die Schadensgruppen mit der Verwundbarkeit und addieren die Ergebnisse. Dann multiplizieren wir mit einer Zahl zwischen 0 und 1, die von `time_since_last_punch` abhängt.

```
t = limit(time_since_last_punch / full_punch_interval, 0, 1)
  = 1

Schaden = (5*90/100 + 10*0/100) * t
        = 4.5
```

Da HP ganze Zahlen sind, wird der Schaden auf 5 Punkte gerundet.

11.6 Anhängen

Angehängte Objekte bewegen sich mit, wenn sich das Elternobjekt – das Objekt, an das sie angehängt sind – bewegt. Ein angehängtes Objekt heißt Kind des Elternobjekts. Ein Objekt kann beliebig viele Kinder, aber höchstens ein Elternobjekt haben.

```
child:set_attach(parent, bone, position, rotation)
```

`get_pos()` gibt immer die globale Position eines Objekts zurück, egal ob es angehängt ist oder nicht. `set_attach` erwartet eine relative Position, aber nicht so, wie man es erwarten würde: Die Position bezieht sich auf den Ursprung des Elternobjekts, mit dem Faktor 10 skaliert. `0,5,0` ist also einen halben Node über dem Ursprung des Elternobjekts.

> **Grad und Bogenmaß**
>
> Die Drehung beim Anhängen wird in Grad angegeben, die Drehung von Objekten dagegen im Bogenmaß. Achte darauf, in die richtige Einheit umzurechnen.

Bei animierten 3D-Modellen hängst du die Entity mit dem Argument bone an einen Knochen. 3D-Animationen basieren auf Skeletten – einem Netz aus Knochen im Modell, denen jeweils eine Position und Drehung gegeben werden kann, um das Modell zu verändern, etwa um den Arm zu bewegen. Das Anhängen an einen Knochen ist nützlich, wenn eine Figur etwas in der Hand halten soll:

```
obj:set_attach(player,
    "Arm_Right",            -- Standardknochen
    {x=0.2, y=6.5, z=3},    -- Standardposition
    {x=-100, y=225, z=90}) -- Standarddrehung
```

11.7 Jetzt bist du dran

- Baue eine Windmühle aus Nodes und einer Entity.
- Erstelle einen Mob deiner Wahl (nur mit der Entity-API und ohne andere Mods).

Teil IV

Spieler

12 Privilegien

Privilegien, kurz *Privs*, erlauben Spielern bestimmte Aktionen. Serverbetreiber können Privilegien vergeben und entziehen und so steuern, was jeder Spieler darf.

12.1 Wann sind Privilegien sinnvoll?

Ein Privileg sollte einem Spieler erlauben, etwas zu *tun*. Privilegien sind **nicht** dazu da, eine Klasse oder einen Rang anzuzeigen.

Gute Privilegien:

- `interact`
- `shout`
- `noclip`
- `fly`
- `kick`
- `ban`
- `vote`
- `worldedit`
- `area_admin` – Admin-Funktionen eines einzelnen Mods sind in Ordnung

Schlechte Privilegien:

- `moderator`
- `admin`
- `elf`
- `dwarf`

12.2 Privilegien deklarieren

Ein neues Privileg legst du mit `register_privilege` an:

```
core.register_privilege("vote", {
    description = "Darf abstimmen",
    give_to_singleplayer = true,
})
```

`give_to_singleplayer` ist standardmäßig `true`; in der Definition oben wäre es also eigentlich nicht nötig.

12.3 Privilegien prüfen

So prüfst du schnell, ob ein Spieler alle nötigen Privilegien hat:

```
local has, missing =
    core.check_player_privs(player_or_name, {
        interact = true,
        vote = true,
    })
```

Hier ist `has` gleich `true`, wenn der Spieler alle nötigen Privilegien hat. Ist `has` gleich `false`, enthält `missing` eine Schlüssel-Wert-Tabelle der fehlenden Privilegien.

```
local has, missing = core.check_player_privs(name, {
    interact = true,
    vote = true,
})

if has then
    print("Spieler hat alle Privilegien!")
else
    print("Spieler fehlen Privilegien: " .. dump(missing))
end
```

Brauchst du die fehlenden Privilegien nicht, kannst du `check_player_privs` direkt in die `if`-Anweisung schreiben.

```
if
    not core.check_player_privs(name, { interact = true })
then
    return false, "Dafür brauchst du interact!"
end
```

12.4 Privilegien lesen und setzen

Die Privilegien eines Spielers kannst du lesen und ändern, egal ob er online ist oder nicht.

```
local privs = core.get_player_privs(name)
print(dump(privs))

privs.vote = true
core.set_player_privs(name, privs)
```

Privilegien werden immer als Schlüssel-Wert-Tabelle angegeben: Der Schlüssel ist der Name des Privilegs, der Wert ein Wahrheitswert.

```
{
    fly = true,
    interact = true,
    shout = true,
},
```

12.5 Privilegien zu basic_privs hinzufügen

Spieler mit dem Privileg `basic_privs` dürfen eine begrenzte Auswahl an Privilegien vergeben und entziehen. Üblicherweise bekommen Moderatoren dieses Privileg, damit sie `interact` und `shout` vergeben und entziehen können, sich selbst oder anderen aber keine leichter missbrauchbaren Privilegien wie `give` oder `server` geben können.

Welche Privilegien deine Moderatoren vergeben und entziehen dürfen, legst du über die Einstellung `basic_privs` fest.

Standardmäßig hat `basic_privs` diesen Wert:

```
basic_privs = interact, shout
```

Um `vote` hinzuzufügen, änderst du ihn so:

```
basic_privs = interact, shout, vote
```

Damit dürfen Spieler mit `basic_privs` das Privileg `vote` vergeben und entziehen.

13 Chat und Befehle

Mods können mit dem Spielerchat arbeiten: Nachrichten senden, Nachrichten abfangen und Chatbefehle registrieren.

13.1 Nachrichten senden

13.1.1 An alle Spieler

Um eine Nachricht an alle Spieler im Spiel zu senden, rufst du `chat_send_all` auf:

```
core.chat_send_all(
    "Das ist eine Chatnachricht an alle Spieler"
)
```

Im Spiel sieht das zum Beispiel so aus:

```
<player1> Schau dir diesen Eingang an
Das ist eine Chatnachricht an alle Spieler
<player2> Was ist damit?
```

Die Nachricht steht in einer eigenen Zeile, damit man sie vom Chat der Spieler unterscheiden kann.

13.1.2 An bestimmte Spieler

Um eine Nachricht an einen bestimmten Spieler zu senden, rufst du `chat_send_player` auf:

```
core.chat_send_player(
    "player1",
    "Diese Nachricht ist für player1"
)
```

Die Nachricht wird genauso angezeigt wie Nachrichten an alle Spieler, ist aber nur für den genannten Spieler sichtbar, hier also für `player1`.

13.2 Chatbefehle

Einen Chatbefehl, zum Beispiel `/foo`, registrierst du mit `register_chatcommand`:

```
core.register_chatcommand("foo", {
    params = "<text>",
    description = "Wiederholt den Text",
    privs = {
        interact = true,
    },
    func = function(name, param)
        return true, "Du hast " .. param .. " gesagt!"
    end,
})
```

Hier ist `interact` als nötiges Privileg (Kapitel 12) angegeben; nur Spieler mit dem Privileg `interact` können den Befehl also ausführen. `params` und `description` erscheinen in der Hilfe, die `/help` anzeigt.

`param` ist ein String mit allem, was der Spieler nach dem Namen des Befehls schreibt. Tippt jemand zum Beispiel `/grantme one,two,three`, ist `param` gleich `one,two,three`.

Chatbefehle können bis zu zwei Werte zurückgeben: zuerst einen Wahrheitswert, der angibt, ob der Befehl erfolgreich war, und dann eine Nachricht an den Benutzer.

> **Auch Offline-Spieler können Befehle ausführen**
>
> Statt eines Spielerobjekts wird ein Spielername übergeben, weil Mods Befehle im Namen von Spielern ausführen können, die nicht online sind. Die IRC-Brücke erlaubt Spielern zum Beispiel, Befehle auszuführen, ohne dem Spiel beizutreten.
>
> Geh also nicht davon aus, dass der Spieler online ist. Das kannst du prüfen, indem du schaust, ob `core.get_player_by_name` einen Spieler zurückgibt.

13.2.1 Mehrere Argumente

`param` enthält alle Argumente eines Chatbefehls in einem einzigen String. Oft muss ein Befehl daraus mehrere Argumente herauslösen. Dafür gibt es zwei Wege: die String-Funktion `split` von Luanti oder Lua-Patterns.

Mit string.split

Mit `string.split(" ")` zerlegst du einen String in Wörter:

```lua
local parts = param:split(" ")
local cmd = parts[1]

if cmd == "join" then
    local team_name = parts[2]
    team.join(name, team_name)
    return true, "Team beigetreten!"
elseif cmd == "max_users" then
    local team_name = parts[2]
    local max_users = tonumber(parts[3])
    if team_name and max_users then
        return true,
            "Maximale Spielerzahl von "
                .. team_name
                .. " auf "
                .. max_users
                .. " gesetzt"
    else
        return false,
            "Aufruf: /team max_users <team_name> <zahl>"
    end
else
    return false, "Befehl fehlt"
end
```

Mit Lua-Patterns

Mit Lua-Patterns[1] (englisch) ziehst du anhand von Regeln Teile aus einem Text heraus. Sie eignen sich am besten, wenn Argumente Leerzeichen enthalten können oder du genauer steuern willst, wie Parameter erfasst werden.

[1] `https://www.lua.org/pil/20.2.html`

```
local to, msg = param:match("^([%a%d_-]+) (.+)$")
```

Dieser Code setzt `/msg <an> <nachricht>` um. Gehen wir ihn von links nach rechts durch:

- ^ passt auf den Anfang des Strings.
- `()` ist eine Gruppe – was darin passt, gibt `string.match` zurück.
- `[]` akzeptiert die Zeichen in dieser Liste.
- %a akzeptiert jeden Buchstaben, %d jede Ziffer.
- `[%a%d_-]` akzeptiert also jeden Buchstaben, jede Ziffer, _ und -.
- + passt, wenn das Vorherige ein- oder mehrmals vorkommt.
- `.` passt hier auf jedes beliebige Zeichen.
- $ passt auf das Ende des Strings.

Kurz gesagt: Das Pattern erfasst den Namen (ein Wort nur aus Buchstaben, Ziffern, - und _), dann ein Leerzeichen, dann die Nachricht (ein oder mehr beliebige Zeichen). Name und Nachricht werden zurückgegeben, weil sie in Klammern stehen.

So setzen die meisten Mods komplexe Chatbefehle um. Ausführlichere Anleitungen zu Lua-Patterns sind das Tutorial auf lua-users.org[2] (englisch) und die Dokumentation in PIL[3] (englisch).

13.3 Nachrichten abfangen

Um Nachrichten abzufangen, verwendest du `register_on_chat_message`:

```
core.register_on_chat_message(function(name, message)
    print(name .. " sagt " .. message)
    return false
end)
```

Mit `return false` lässt du zu, dass die Nachricht wie üblich gesendet wird. Du könntest die Zeile `return false` auch weglassen, ohne dass sich etwas ändert, denn dann wird implizit `nil` zurückgegeben, und das wird wie `false` behandelt.

[2] http://lua-users.org/wiki/PatternsTutorial
[3] https://www.lua.org/pil/20.2.html

> **Privilegien und Chatbefehle**
>
> Ein Spieler braucht das Privileg `shout` nicht, um diesen Callback auszulösen. Chatbefehle sind nämlich in Lua umgesetzt und einfach Chatnachrichten, die mit / beginnen.

Berücksichtige also, dass es sich um einen Chatbefehl handeln kann oder dass dem Spieler `shout` fehlt.

```
core.register_on_chat_message(function(name, message)
    if message:sub(1, 1) == "/" then
        print(name .. " hat einen Chatbefehl ausgeführt")
    elseif
        core.check_player_privs(name, { shout = true })
    then
        print(name .. " sagt " .. message)
    else
        print(
            name
                .. " wollte "
                .. message
                .. " sagen, hat aber kein shout"
        )
    end

    return false
end)
```

14 Spielerphysik

Die Physik eines Spielers lässt sich mit *Physik-Overrides* verändern. Damit setzt du Gehgeschwindigkeit, Sprungkraft und Schwerkraft. Physik-Overrides gelten jeweils für einen einzelnen Spieler und sind Faktoren: Ein Wert von 2 für die Schwerkraft macht sie zum Beispiel doppelt so stark.

14.1 Ein einfaches Beispiel

Hier ein Befehl `/antigravity`, der den aufrufenden Spieler in geringe Schwerkraft versetzt:

```
core.register_chatcommand("antigravity", {
    func = function(name, param)
        local player = core.get_player_by_name(name)
        if not player then
            return false, "Nur für Spieler im Spiel"
        end
        player:set_physics_override({
            gravity = 0.1, -- 10 % der normalen Schwerkraft
        })
    end,
})
```

14.2 Verfügbare Overrides

`player:set_physics_override()` bekommt eine Tabelle mit Overrides. Laut Lua-API-Referenz[1] sind die wichtigsten:

- `speed`: Faktor für die Gehgeschwindigkeit (Standard: 1)
- `jump`: Faktor für die Sprungkraft (Standard: 1)
- `gravity`: Faktor für die Schwerkraft (Standard: 1)
- `sneak`: ob der Spieler schleichen kann (Standard: `true`)

[1] `https://api.luanti.org/class-reference/#player-only-no-op-for-other-objects`

Seit Luanti 5.8 gibt es weitere Overrides, etwa `speed_climb`, `speed_crouch`, `liquid_fluidity`, `liquid_sink`, `acceleration_default` und `acceleration_air`. Die vollständige Liste steht in der Lua-API-Referenz.

14.2.1 Altes Bewegungsverhalten

Vor Version 0.4.16 gab es den *Sneak-Glitch*, der verschiedene Bewegungstricks erlaubte. So konnte man mit einer bestimmten Anordnung von Nodes einen „Aufzug“ hochklettern, indem man schlich (Shift) und dabei die Leertaste drückte. Das Verhalten war zwar unbeabsichtigt, ist aber in den Overrides erhalten geblieben, weil viele Server es nutzen.

Um das alte Bewegungsverhalten vollständig wiederherzustellen, braucht es zwei Overrides:

- `new_move`: ob der Spieler die neue Bewegung verwendet (Standard: `true`)
- `sneak_glitch`: ob der Spieler „Schleich-Aufzüge“ nutzen kann (Standard: `false`)

14.3 Unverträglichkeit zwischen Mods

Achtung: Mods, die denselben Physikwert eines Spielers überschreiben, vertragen sich meist nicht miteinander. Ein Override überschreibt alle zuvor gesetzten. Setzen also mehrere Mods die Geschwindigkeit eines Spielers, gilt nur der zuletzt gesetzte Wert.

14.4 Jetzt bist du dran

- **Sonic**: Setze den Geschwindigkeitsfaktor auf einen hohen Wert (mindestens 6), wenn ein Spieler dem Spiel beitritt.
- **Superfeder**: Erhöhe die Sprungkraft, sodass der Spieler 20 Meter hoch springen kann (1 Meter entspricht 1 Node).
- **Weltraum**: Lass die Schwerkraft abnehmen, je höher der Spieler kommt.

15 GUIs (Formspecs)

In diesem Kapitel lernst du, wie du eine Formspec erstellst und dem Spieler anzeigst. Eine Formspec ist der Beschreibungscode für ein Formular. Formulare sind in Luanti Fenster wie das Spielerinventar und können verschiedene Elemente enthalten, etwa Beschriftungen, Schaltflächen und Eingabefelder.

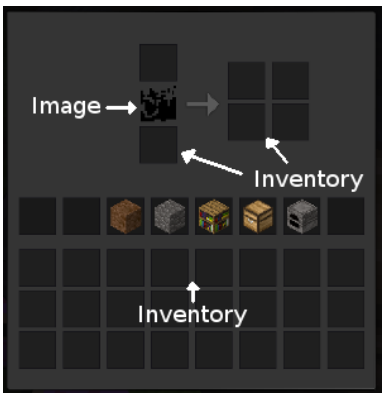


Abbildung 15.1: Formspec eines Ofens mit Beschriftung der Elemente

Musst du keine Eingaben vom Spieler entgegennehmen, sondern ihm nur Informationen anzeigen, solltest du statt eines Formulars lieber HUD-Elemente (Kapitel 16) verwenden. Unerwartet aufgehende Fenster stören nämlich meist das Spiel.

15.1 Echte oder alte Koordinaten

In älteren Versionen von Luanti waren Formspecs uneinheitlich. Elemente wurden auf unerwartet unterschiedliche Weise positioniert; es war schwer vorherzusagen, wo sie landen, und sie aneinander auszurichten. Luanti 5.1.0 führte daher *echte Koordinaten* (*real coordinates*) ein, ein einheitliches Koordinatensystem. Echte Koordinaten sind sehr zu empfehlen, deshalb verwendet dieses Kapitel ausschließlich sie.

Ab `formspec_version` 2 sind echte Koordinaten eingeschaltet.

15.2 Aufbau einer Formspec

15.2.1 Elemente

Formspec ist eine domänenspezifische Sprache mit einem ungewöhnlichen Format. Sie besteht aus Elementen dieser Form:

```
typ[param1;param2]
```

Zuerst steht der Elementtyp, dann folgen die Parameter in eckigen Klammern. Mehrere Elemente können direkt aneinandergehängt oder auf mehrere Zeilen verteilt werden:

```
foo[param1]bar[param1]
bo[param1]
```

Elemente sind zum Beispiel Textfelder oder Schaltflächen, können aber auch Metadaten wie Größe oder Hintergrund sein. Eine Liste aller Elemente findest du in der Lua-API-Referenz[1].

15.2.2 Kopf

Der Kopf einer Formspec enthält Angaben, die zuerst stehen müssen: die Größe der Formspec, ihre Position, den Anker und ob das spielweite Theme angewendet werden soll.

Die Elemente im Kopf müssen in einer bestimmten Reihenfolge stehen, sonst gibt es einen Fehler. Diese Reihenfolge ist die im vorigen Absatz genannte; sie ist, wie immer, auch in der Lua-API-Referenz dokumentiert.

Die Größe wird in Formspec-Feldern angegeben, einer Maßeinheit von ungefähr 64 Pixeln, die aber je nach Bildschirmdichte und Skalierungseinstellungen des Clients schwankt. Hier eine Formspec der Größe `2,2`:

```
formspec_version[4]
size[2,2]
```

[1] `https://api.luanti.org/formspec/`

Beachte, dass wir die Sprachversion der Formspec ausdrücklich angegeben haben. Ohne sie wird das alte System verwendet, und du kannst weder die einheitliche Positionierung noch andere neue Funktionen nutzen.

Mit `position` und `anchor` platzierst du die Formspec auf dem Bildschirm. `position` legt fest, wo auf dem Bildschirm die Formspec erscheint; Standard ist die Mitte (`0.5,0.5`). `anchor` legt fest, welcher Punkt der Formspec an dieser Position liegt; so kannst du sie am Bildschirmrand ausrichten. Am linken Rand platzierst du die Formspec so:

```
formspec_version[4]
size[2,2]
position[0,0.5]
anchor[0,0.5]
```

Damit liegt der Anker in der Mitte des linken Rands der Formspec, und dieser Anker wird an den linken Bildschirmrand gesetzt.

15.3 Ein Ratespiel

Am besten lernt man, indem man etwas baut. Bauen wir also ein Ratespiel. Das Prinzip ist einfach: Der Mod denkt sich eine Zahl aus, und der Spieler rät. Der Mod sagt dann, ob der Tipp zu hoch oder zu niedrig war.

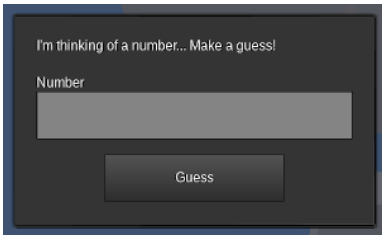


Abbildung 15.2: Die Formspec des Ratespiels

Zuerst schreiben wir eine Funktion, die den Formspec-Code erzeugt. Das ist gute Praxis, weil sich der Code so leichter an anderer Stelle wiederverwenden lässt.

```
guessing = {}

function guessing.get_formspec(name)
    -- TODO: anzeigen, ob der Tipp zu hoch/niedrig war
    local text = "Ich denke an eine Zahl ... Rate mal!"

    local formspec = {
        "formspec_version[4]",
        "size[6,3.476]",
        "label[0.375,0.5;",
        core.formspec_escape(text),
        "]",
        "field[0.375,1.25;5.25,0.8;number;Zahl;]",
        "button[1.5,2.3;3,0.8;guess;Raten]",
    }

    -- table.concat ist schneller als Verkettung mit `..`
    return table.concat(formspec, "")
end
```

Hier platzieren wir ein Eingabefeld, eine Beschriftung und eine Schaltfläche. In das Feld kann man Text eingeben, die Schaltfläche schickt das Formular ab. Die Elemente sind sorgfältig positioniert, um Innenabstand und Zwischenraum zu erzeugen; das wird gleich erklärt.

Als Nächstes soll der Spieler die Formspec öffnen können. Der übliche Weg dafür ist `show_formspec`:

```
function guessing.show_to(name)
    core.show_formspec(
        name,
        "guessing:game",
        guessing.get_formspec(name)
    )
end

core.register_chatcommand("game", {
    func = function(name)
        guessing.show_to(name)
    end,
})
```

`show_formspec` erwartet einen Spielernamen, den Namen der Formspec und

die Formspec selbst. Der Name der Formspec sollte ein gültiger Itemname sein, also im Format `modname:itemname`.

15.3.1 Innenabstand und Zwischenraum

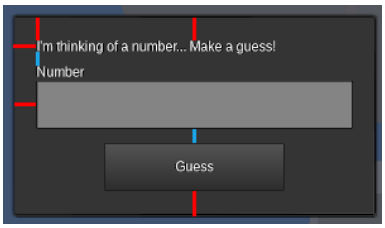


Abbildung 15.3: Innenabstand (rot) und Zwischenraum (blau)

Der Innenabstand (*padding*, rot) ist der Abstand zwischen dem Rand der Formspec und ihrem Inhalt oder zwischen Elementen, die nicht zusammengehören. Der Zwischenraum (*spacing*, blau) ist der Abstand zwischen zusammengehörenden Elementen.

Üblich sind ein Innenabstand von `0.375` und ein Zwischenraum von `0.25`.

15.3.2 Eingaben empfangen

Beim Aufruf von `show_formspec` wird die Formspec zum Client geschickt und dort angezeigt. Nützlich wird sie aber erst, wenn Informationen vom Client zum Server zurückkommen. Dieser Vorgang heißt *Formspec field submission*; bei `show_formspec` empfängst du die Eingaben mit einem globalen Callback:

```
core.register_on_player_receive_fields(
    function(player, formname, fields)
        if formname ~= "guessing:game" then
            return
        end

        if fields.guess then
            local pname = player:get_player_name()
            core.chat_send_all(
                pname
                    .. " hat "
                    .. fields.number
                    .. " geraten"
```

```
                )
            end
        end
)
```

Die an `core.register_on_player_receive_fields` übergebene Funktion wird jedes Mal aufgerufen, wenn ein Spieler ein Formular abschickt. Die meisten Callbacks müssen den übergebenen Formularnamen prüfen und abbrechen, wenn es nicht das richtige Formular ist; manche Callbacks arbeiten aber auch mit mehreren oder allen Formularen.

Der Parameter `fields` ist eine Tabelle mit den Werten, die der Spieler abgeschickt hat, indiziert mit Strings. Benannte Elemente erscheinen je nach Ereignis unter ihrem eigenen Namen. Manche Elemente, etwa Schaltflächen, werden nur übermittelt, wenn sie das Ereignis ausgelöst haben; andere, etwa Eingabefelder, sind immer dabei.

> **Bösartige Clients können jederzeit alles schicken**
>
> Vertraue niemals den abgeschickten Daten einer Formspec. Ein bösartiger Client kann jederzeit alles Mögliche schicken – selbst wenn du ihm die Formspec nie angezeigt hast. Prüfe also Privilegien und stelle sicher, dass der Spieler die Aktion wirklich ausführen darf.

Die Formspec wird jetzt also zum Client geschickt, und der Client schickt Informationen zurück. Als Nächstes müssen wir die Zielzahl irgendwie erzeugen und uns merken und die Formspec abhängig von den Tipps aktualisieren. Dafür gibt es das Konzept der *Kontexte*.

15.3.3 Kontexte

Oft soll `core.show_formspec` dem Callback Informationen mitgeben, die der Client nicht sehen soll – etwa, mit welchen Argumenten ein Chatbefehl aufgerufen wurde oder worum es in dem Dialog geht. Hier ist es die Zielzahl, die wir uns merken müssen.

Ein Kontext ist eine Tabelle je Spieler, in der Informationen gespeichert werden. Die Kontexte aller Spieler, die gerade online sind, liegen in einer lokalen Variablen der Datei:

```
local _contexts = {}
local function get_context(name)
    local context = _contexts[name] or {}
    _contexts[name] = context
    return context
end

core.register_on_leaveplayer(function(player)
    _contexts[player:get_player_name()] = nil
end)
```

Nun ändern wir den Code zum Anzeigen so, dass er den Kontext aktualisiert, bevor er die Formspec anzeigt:

```
function guessing.show_to(name)
    local context = get_context(name)
    context.target = context.target or math.random(1, 10)

    local fs = guessing.get_formspec(name, context)
    core.show_formspec(name, "guessing:game", fs)
end
```

Auch der Code, der die Formspec erzeugt, muss den Kontext verwenden:

```
function guessing.get_formspec(name, context)
    local text
    if not context.guess then
        text = "Ich denke an eine Zahl ... Rate mal!"
    elseif context.guess == context.target then
        text = "Hurra, richtig geraten!"
    elseif context.guess > context.target then
        text = "Zu hoch!"
    else
        text = "Zu niedrig!"
    end

    -- … weiter wie oben
end
```

Es ist gute Praxis, dass `get_formspec` den Kontext nur liest und nie verändert. Das hält die Funktion einfacher und leichter testbar.

Zum Schluss muss der Handler den Kontext mit dem Tipp aktualisieren:

```
if fields.guess then
    local name = player:get_player_name()
    local context = get_context(name)
    context.guess = tonumber(fields.number)
    guessing.show_to(name)
end
```

15.4 Wege zur Formspec

Eine Formspec kann auf drei Wegen zum Client gelangen:

- show_formspec: der oben verwendete Weg; die Eingaben empfängt `register_on_player_receive_fields`.
- Formspecs in Node-Metadaten: Der Node hat eine Formspec in seinen Metadaten, und der Client zeigt sie *sofort* an, wenn der Spieler rechts klickt. Die Eingaben empfängt die Methode `on_receive_fields` in der Node-Definition.
- Formspecs im Spielerinventar: Die Formspec wird irgendwann zum Client geschickt und sofort angezeigt, wenn der Spieler `i` drückt. Die Eingaben empfängt `register_on_player_receive_fields`.

15.4.1 Formspecs in Node-Metadaten

`core.show_formspec` ist nicht der einzige Weg, eine Formspec anzuzeigen; du kannst Formspecs auch in die Metadaten eines Nodes (Kapitel 10) schreiben. Truhen machen das zum Beispiel, damit sie schneller aufgehen – man muss nicht warten, bis der Server die Formspec der Truhe schickt.

```
core.register_node("mymod:rightclick", {
    description = "Klick mich mit rechts!",
    tiles = { "mymod_rightclick.png" },
    groups = { cracky = 1 },
    after_place_node = function(pos, placer)
        -- Läuft, wenn der Node platziert wird, und setzt
        -- die Formspec in den Metadaten des Nodes.
        local meta = core.get_meta(pos)
        meta:set_string(
            "formspec",
            "formspec_version[4]"
                .. "size[5,5]"
```

```
                .. "label[1,1;Erscheint beim Rechtsklick]"
                .. "field[1,2;2,1;x;x;]"
        )
    end,
    on_receive_fields = function(
        pos,
        formname,
        fields,
        player
    )
        if fields.quit then
            return
        end

        print(fields.x)
    end,
})
```

So gesetzte Formspecs lösen nicht denselben Callback aus. Um Eingaben aus Formspecs in Metadaten zu empfangen, musst du bei der Registrierung des Nodes einen Eintrag `on_receive_fields` angeben.

Dieser Callback wird auch ausgelöst, wenn du in einem Feld die Eingabetaste drückst, was mit `core.show_formspec` nicht möglich ist. Dafür lässt sich diese Art von Formular nur durch einen Rechtsklick auf den Node öffnen, nicht per Programmcode.

15.4.2 Formspecs im Spielerinventar

Die Formspec des Spielerinventars erscheint, wenn der Spieler `i` drückt. Ihre Ereignisse empfängst du mit dem globalen Callback; der Formularname ist dabei `""`.

Es gibt verschiedene Mods, mit denen mehrere Mods gemeinsam das Spielerinventar anpassen können. Minetest Game verwendet SFINV[2].

15.4.3 Jetzt bist du dran

- Erweitere das Ratespiel so, dass es sich für jeden Spieler die Bestleistung merkt, also mit wie wenigen Tipps er die Zahl erraten hat.

[2] `https://github.com/rubenwardy/sfinv/blob/master/Tutorial.md`

- Erstelle einen Node „Briefkasten“, in dem Spieler über eine Formspec Nachrichten hinterlassen können. Der Node soll den Namen des Platzierenden als `owner` in den Metadaten speichern und mit `show_formspec` unterschiedlichen Spielern unterschiedliche Formspecs zeigen.

16 HUD

Mit HUD-Elementen (*Heads-up-Display*) zeigst du Text, Bilder und andere grafische Elemente an.

Das HUD nimmt keine Eingaben entgegen; dafür verwendest du eine Formspec (Kapitel 15).

> **type statt hud_elem_type**
>
> Seit Luanti 5.9 heißt die Eigenschaft für den Elementtyp `type`. Der alte Name `hud_elem_type` funktioniert noch, gilt aber als veraltet. Dieses Kapitel verwendet durchgehend `type`.

16.1 Positionierung

16.1.1 Position und Versatz

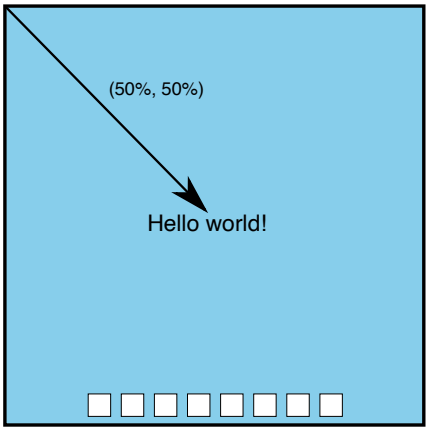


Abbildung 16.1: Ein zentriertes Textelement

Bildschirme gibt es in vielen Größen und Auflösungen, und das HUD muss auf allen gut funktionieren.

Luanti löst das, indem die Lage eines Elements aus einer prozentualen Position und einem Versatz (*offset*) bestimmt wird. Die prozentuale Position bezieht sich

auf die Bildschirmgröße. Um ein Element in der Bildschirmmitte zu platzieren, gibst du also die halbe Bildschirmgröße als Position an, etwa (50 %, 50 %), und einen Versatz von (0, 0).

Mit dem Versatz verschiebst du das Element dann relativ zur prozentualen Position.

16.1.2 Ausrichtung

Die Ausrichtung (*alignment*) legt fest, wo am Element der aus Position und Versatz berechnete Punkt liegt. Mit `{x = -1.0, y = 0.0}` zeigt dieser Punkt zum Beispiel auf den linken Rand des Elements. Das ist besonders nützlich, um Text links, mittig oder rechts auszurichten.

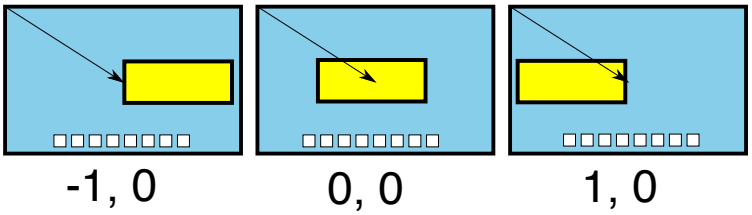


Abbildung 16.2: Ausrichtung

Das Diagramm zeigt drei Fenster (blau) mit jeweils einem HUD-Element (gelb) und unterschiedlicher Ausrichtung. Der Pfeil ist das Ergebnis aus Position und Versatz.

16.1.3 Punkteanzeige

In diesem Kapitel lernst du, eine Punkteanzeige wie diese zu positionieren und zu aktualisieren:



Abbildung 16.3: So soll das HUD am Ende aussehen

Im Bild haben alle Elemente dieselbe prozentuale Position (100 %, 50 %), aber unterschiedliche Versätze. So ist das Ganze am rechten Fensterrand verankert und bleibt beim Ändern der Fenstergröße intakt.

16.2 Textelemente

Sobald du das Spielerobjekt hast, kannst du ein HUD-Element anlegen:

```
local player = core.get_player_by_name("username")
local idx = player:hud_add({
    type = "text",
    position = { x = 0.5, y = 0.5 },
    offset = { x = 0, y = 0 },
    text = "Hallo Welt!",
    alignment = { x = 0, y = 0 }, -- zentriert
    scale = { x = 100, y = 100 }, -- siehe unten
})
```

hud_add gibt eine Element-ID zurück, mit der du das HUD-Element später ändern oder entfernen kannst.

16.2.1 Parameter

Den Elementtyp gibst du mit der Eigenschaft `type` in der Definitionstabelle an. Die Bedeutung der übrigen Eigenschaften hängt vom Typ ab.

`scale` ist der maximale Bereich für den Text; Text außerhalb wird abgeschnitten, z. B. `{x=100, y=100}`.

`number` ist die Textfarbe als Hexadezimalzahl, z. B. `0xFF0000`.

16.2.2 Unser Beispiel

Platzieren wir nun alle Texte unserer Punkteanzeige:

```
-- Zähler aus dem Speicher holen, Standard 0
local meta = player:get_meta()
local digs_text = "Abgebaut: " .. meta:get_int("score:digs")
local places_text = "Platziert: "
    .. meta:get_int("score:places")

player:hud_add({
    type = "text",
    position = { x = 1, y = 0.5 },
    offset = { x = -120, y = -25 },
```

```
    text = "Statistik",
    alignment = { x = 0, y = 0 },
    scale = { x = 100, y = 30 },
    number = 0xFFFFFF,
})

player:hud_add({
    type = "text",
    position = { x = 1, y = 0.5 },
    offset = { x = -180, y = 0 },
    text = digs_text,
    alignment = { x = -1, y = 0 },
    scale = { x = 50, y = 10 },
    number = 0xFFFFFF,
})

player:hud_add({
    type = "text",
    position = { x = 1, y = 0.5 },
    offset = { x = -70, y = 0 },
    text = places_text,
    alignment = { x = -1, y = 0 },
    scale = { x = 50, y = 10 },
    number = 0xFFFFFF,
})
```

Das ergibt Folgendes:

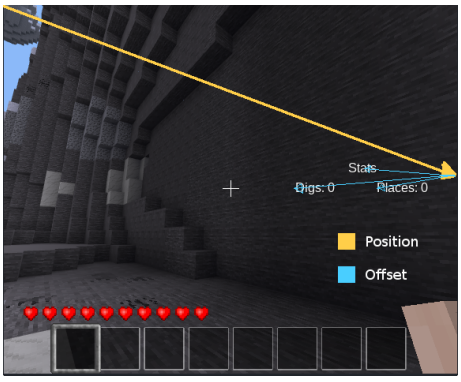


Abbildung 16.4: Die Textelemente der Punkteanzeige

16.3 Bildelemente

Bildelemente legst du ganz ähnlich an wie Textelemente:

```
player:hud_add({
    type = "image",
    position = { x = 1, y = 0.5 },
    offset = { x = -220, y = 0 },
    text = "score_background.png",
    scale = { x = 1, y = 1 },
    alignment = { x = 1, y = 0 },
})
```

Jetzt sieht es so aus:



Abbildung 16.5: Die Punkteanzeige mit Hintergrundbild

16.3.1 Parameter

Im Feld `text` steht der Name des Bildes.

Ist eine Koordinate von `scale` positiv, ist sie ein Skalierungsfaktor: 1 ist die Originalgröße, 2 die doppelte Größe und so weiter. Ist sie negativ, ist sie ein Prozentwert der Bildschirmgröße; `x=-100` bedeutet zum Beispiel 100 % der Breite.

16.3.2 Skalierung

Als Beispiel für die Skalierung bauen wir den Fortschrittsbalken der Punkteanzeige:

```
local percent = tonumber(meta:get("score:score") or 0.2)

player:hud_add({
    type = "image",
    position = { x = 1, y = 0.5 },
    offset = { x = -215, y = 23 },
    text = "score_bar_empty.png",
    scale = { x = 1, y = 1 },
    alignment = { x = 1, y = 0 },
})

player:hud_add({
    type = "image",
    position = { x = 1, y = 0.5 },
    offset = { x = -215, y = 23 },
    text = "score_bar_full.png",
    scale = { x = percent, y = 1 },
    alignment = { x = 1, y = 0 },
})
```

Jetzt sieht das HUD aus wie das am Anfang des Kapitels! Ein Problem gibt es allerdings: Es aktualisiert sich nicht, wenn sich die Werte ändern.

16.4 Ein Element ändern

Mit der ID, die hud_add zurückgibt, kannst du das Element später ändern oder entfernen.

```
local idx = player:hud_add({
    type = "text",
    text = "Hallo Welt!",
    -- weitere Parameter der Kürze halber weggelassen
})

player:hud_change(idx, "text", "Neuer Text")
player:hud_remove(idx)
```

hud_change erwartet die Element-ID, die zu ändernde Eigenschaft und den neuen Wert. Der Aufruf oben ändert die Eigenschaft `text` von „Hallo Welt!“ in „Neuer Text“.

hud_change direkt nach hud_add aufzurufen, hat also dieselbe Wirkung wie Folgendes, nur deutlich umständlicher:

```
local idx = player:hud_add({
    type = "text",
    text = "Neuer Text",
})
```

16.5 IDs speichern

```
score = {}
local saved_huds = {}

function score.update_hud(player)
    local player_name = player:get_player_name()

    -- Zähler aus dem Speicher holen, Standard 0
    local meta = player:get_meta()
    local digs_text = "Abgebaut: "
        .. meta:get_int("score:digs")
    local places_text = "Platziert: "
        .. meta:get_int("score:places")
    local percent = tonumber(meta:get("score:score") or 0.2)

    local ids = saved_huds[player_name]
    if ids then
        player:hud_change(
            ids["places"],
            "text",
            places_text
        )
        player:hud_change(ids["digs"], "text", digs_text)
        player:hud_change(
            ids["bar_foreground"],
            "scale",
            { x = percent, y = 1 }
        )
    else
        ids = {}
        saved_huds[player_name] = ids

        -- HUD-Elemente anlegen, IDs in `ids` merken
```

```
    end
end

core.register_on_joinplayer(score.update_hud)

core.register_on_leaveplayer(function(player)
    saved_huds[player:get_player_name()] = nil
end)
```

16.6 Weitere Elemente

Eine vollständige Liste der HUD-Elemente findest du in der Lua-API-Referenz[1].

[1] https://api.luanti.org/hud/

Teil V

Fortgeschrittene Kartenbearbeitung

17 Biome und Dekorationen

Biome und Dekorationen registrieren zu können, ist entscheidend für eine interessante und abwechslungsreiche Spielwelt. Dieses Kapitel zeigt, wie du Biome registrierst, ihre Verteilung steuerst und Dekorationen in Biomen platzierst.

17.1 Was sind Biome?

Ein Biom ist in Luanti eine bestimmte Umgebung in der Spielwelt. Beim Registrieren eines Bioms legst du fest, welche Nodes bei der Kartengenerierung darin vorkommen. Zu den Node-Arten, die sich am häufigsten von Biom zu Biom unterscheiden, gehören:

- Oberflächen-Node (*top node*): der Node, der an der Oberfläche am häufigsten vorkommt. Ein bekanntes Beispiel ist „Erde mit Gras“ aus Minetest Game.
- Füll-Node (*filler node*): die Schicht direkt unter dem Oberflächen-Node. In Biomen mit Gras ist das oft Erde.
- Stein-Node: der Node, den man unter der Erde am häufigsten sieht.
- Wasser-Node: meist eine Flüssigkeit; dieser Node erscheint dort, wo man Gewässer erwartet.

Auch andere Node-Arten können sich zwischen Biomen unterscheiden. So lassen sich im selben Spiel sehr unterschiedliche Umgebungen schaffen.

17.2 Platzierung von Biomen

17.2.1 Wärme und Feuchtigkeit

Ein Biom nur zu registrieren, genügt nicht; du musst auch festlegen, wo es im Spiel vorkommen kann. Dazu bekommt jedes Biom einen Wert für Wärme (*heat*) und Feuchtigkeit (*humidity*).

Überlege dir diese Werte gut, denn sie bestimmen, welche Biome aneinandergrenzen können. Bei schlechter Wahl grenzt eine heiße Wüste womöglich an

einen Gletscher, und es entstehen andere unwahrscheinliche Kombinationen, die du lieber vermeiden möchtest.

Im Spiel liegen Wärme und Feuchtigkeit an jedem Punkt der Karte meist zwischen 0 und 100. Die Werte ändern sich allmählich, während man sich über die Karte bewegt. An jedem Punkt entsteht das registrierte Biom, dessen Wärme- und Feuchtigkeitswerte denen an dieser Stelle der Karte am nächsten kommen.

Da sich Wärme und Feuchtigkeit nur allmählich ändern, solltest du die Werte eines Bioms an dem ausrichten, was man von seiner Umgebung erwarten würde. Zum Beispiel:

- Eine Wüste hat hohe Wärme und niedrige Feuchtigkeit.
- Ein verschneiter Wald hat niedrige Wärme und mittlere Feuchtigkeit.
- Ein Sumpf hat meist hohe Feuchtigkeit.

Solange du eine vielfältige Auswahl an Biomen hast, ergibt sich so in der Praxis meist ein logischer Übergang zwischen benachbarten Biomen.

17.2.2 Grenzen mit Voronoi-Diagrammen veranschaulichen

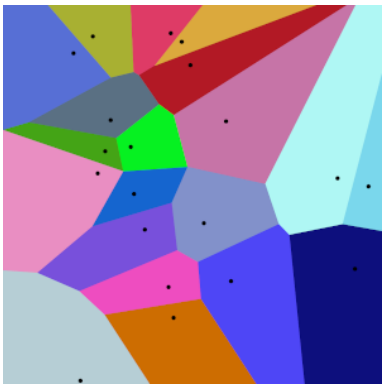


Abbildung 17.1: Voronoi-Diagramm: Jede Fläche enthält die Punkte, die ihrem Mittelpunkt am nächsten liegen. Bild: Balu Ertl, CC BY-SA 4.0

Wärme- und Feuchtigkeitswerte lassen sich leichter feinabstimmen, wenn du die Beziehungen zwischen deinen Biomen sichtbar machst. Das ist besonders wichtig, wenn du einen kompletten eigenen Satz an Biomen erstellst, kann aber auch beim Ergänzen eines bestehenden Satzes helfen.

Am einfachsten machst du sichtbar, welche Biome aneinandergrenzen können, mit einem Voronoi-Diagramm. Es zeigt für jede Stelle eines zweidimensionalen Diagramms, welchem Punkt sie am nächsten liegt.

Ein Voronoi-Diagramm zeigt, wo Biome, die aneinandergrenzen sollten, es nicht tun – und wo Biome aneinandergrenzen, die es nicht sollten. Außerdem gibt es einen groben Eindruck davon, wie häufig Biome im Spiel sein werden: Große und zentral liegende Biome sind häufiger als kleine oder solche am Rand des Diagramms.

Dazu setzt du für jedes Biom einen Punkt anhand seiner Wärme und Feuchtigkeit, mit der Wärme auf der x-Achse und der Feuchtigkeit auf der y-Achse. Dann teilst du das Diagramm in Flächen ein, sodass jede Stelle einer Fläche näher an deren Punkt liegt als an jedem anderen Punkt des Diagramms.

Jede Fläche steht für ein Biom. Haben zwei Flächen eine gemeinsame Grenze, können die zugehörigen Biome im Spiel nebeneinander liegen. Wie lang die gemeinsame Grenze im Vergleich zu den Grenzen mit anderen Flächen ist, verrät, wie häufig die beiden Biome voraussichtlich nebeneinander vorkommen.

17.2.3 Voronoi-Diagramme mit GeoGebra erstellen

Voronoi-Diagramme kannst du von Hand zeichnen oder mit Programmen wie GeoGebra[1] erstellen.

1. Lege Punkte an, indem du in der Werkzeugleiste das Punktwerkzeug wählst (Symbol: ein Punkt mit „A“) und in das Diagramm klickst. Du kannst Punkte verschieben oder ihre Position in der linken Seitenleiste genau eingeben. Gib jedem Punkt außerdem eine Beschriftung, das macht es übersichtlicher.

2. Erzeuge dann das Voronoi-Diagramm, indem du in das Eingabefeld der linken Seitenleiste diese Funktion eingibst:

```
Voronoi({ A, B, C, D, E })
```

   In den geschweiften Klammern stehen alle Punkte, durch Kommas getrennt.

3. Fertig! Du hast jetzt ein Voronoi-Diagramm, dessen Punkte du verschieben kannst.

[1] `https://www.geogebra.org`

17.3 Ein Biom registrieren

Der folgende Code registriert ein einfaches Biom namens `grasslands`:

```
core.register_biome({
    name = "grasslands",
    node_top = "default:dirt_with_grass",
    depth_top = 1,
    node_filler = "default:dirt",
    depth_filler = 3,
    y_max = 1000,
    y_min = -3,
    heat_point = 50,
    humidity_point = 50,
})
```

Dieses Biom hat an der Oberfläche eine Schicht „Erde mit Gras“ und darunter drei Schichten Erde. Es gibt keinen Stein-Node an; unter der Erde liegt daher der Node, der als Mapgen-Alias `mapgen_stone` registriert ist.

Beim Registrieren eines Bioms gibt es viele Optionen; sie sind wie immer in der Lua-API-Referenz[2] dokumentiert.

Du musst nicht für jedes Biom jede Option angeben. Fehlt aber eine bestimmte Option oder ein passender Mapgen-Alias, kann es zu Fehlern bei der Kartengenerierung kommen.

17.4 Was sind Dekorationen?

Dekorationen sind Nodes oder Schematics, die bei der Kartengenerierung auf der Karte platziert werden. Typische Beispiele sind Blumen, Büsche und Bäume. Kreativere Einsätze sind etwa Eiszapfen oder Stalagmiten in Höhlen, unterirdische Kristallformationen oder sogar kleine Gebäude.

Dekorationen lassen sich auf bestimmte Biome, Höhen oder Untergrund-Nodes beschränken. Oft gestalten sie die Umgebung eines Bioms, indem sie für bestimmte Pflanzen, Bäume oder andere Merkmale sorgen.

[2] `https://api.luanti.org/definition-tables/#biome-definition`

17.5 Eine einfache Dekoration registrieren

Einfache Dekorationen platzieren bei der Kartengenerierung einzelne Nodes. Du gibst an, welcher Node als Dekoration platziert wird, wo er platziert werden darf und wie häufig er vorkommt.

Zum Beispiel:

```
core.register_decoration({
    deco_type = "simple",
    place_on = { "base:dirt_with_grass" },
    sidelen = 16,
    fill_ratio = 0.1,
    biomes = { "grassy_plains" },
    y_max = 200,
    y_min = 1,
    decoration = "plants:grass",
})
```

Hier wird der Node `plants:grass` im Biom `grassy_plains` auf `base:dirt_with_grass` platziert, zwischen den Höhen `y = 1` und `y = 200`.

`fill_ratio` bestimmt, wie häufig die Dekoration vorkommt; höhere Werte bis 1 führen zu sehr vielen Dekorationen. Alternativ kannst du die Platzierung über Rauschparameter (*noise parameters*) steuern.

17.6 Eine Schematic-Dekoration registrieren

Schematic-Dekorationen funktionieren ganz ähnlich wie einfache Dekorationen, platzieren aber eine Schematic statt eines einzelnen Nodes. Zum Beispiel:

```
core.register_decoration({
    deco_type = "schematic",
    place_on = { "base:desert_sand" },
    sidelen = 16,
    fill_ratio = 0.0001,
    biomes = { "desert" },
    y_max = 200,
    y_min = 1,
    schematic = core.get_modpath("plants")
        .. "/schematics/cactus.mts",
```

```
    flags = "place_center_x, place_center_z",
    rotation = "random",
})
```

Hier wird die Schematic `cactus.mts` in Wüstenbiomen platziert. Du musst den Pfad zu einer Schematic angeben; in diesem Fall liegt sie in einem eigenen Schematic-Verzeichnis innerhalb des Mods.

Das Beispiel setzt außerdem Flags, um die Schematic mittig zu platzieren, und eine zufällige Drehung. Zufällig gedrehte Schematics sorgen für mehr Abwechslung, wenn sie nicht symmetrisch sind.

17.7 Mapgen-Aliase

Bestehende Spiele bringen bereits passende Mapgen-Aliase mit. Eigene musst du nur registrieren, wenn du ein eigenes Spiel entwickelst.

Mapgen-Aliase geben dem Kartengenerator der Engine Informationen und werden so registriert:

```
core.register_alias("mapgen_stone", "base:smoke_stone")
```

Mindestens diese solltest du registrieren:

- `mapgen_stone`
- `mapgen_water_source`
- `mapgen_river_water_source`

Legst du nicht für alle Biome Höhlenflüssigkeiten fest, solltest du außerdem registrieren:

- `mapgen_lava_source`

18 Lua Voxel Manipulators

Die Funktionen aus dem Kapitel Grundlegende Kartenoperationen (Kapitel 8) sind bequem und einfach zu benutzen, für große Bereiche aber ineffizient. Bei jedem Aufruf von `set_node` oder `get_node` muss dein Mod mit der Engine kommunizieren. Das führt zu ständigen einzelnen Kopiervorgängen zwischen Engine und Mod, was langsam ist und die Leistung deines Spiels schnell verschlechtert. Ein *Lua Voxel Manipulator* (LVM) kann hier die bessere Wahl sein.

18.1 Konzept

Mit einem LVM lädst du große Bereiche der Karte in den Speicher deines Mods. Diese Daten kannst du dann lesen und schreiben, ohne weiter mit der Engine zu kommunizieren und ohne dass Callbacks laufen – diese Operationen sind also sehr schnell. Danach schreibst du den Bereich zurück in die Engine und lässt die Beleuchtung berechnen.

18.2 Daten in den LVM lesen

In einen LVM kannst du nur einen Quader laden. Du musst also die kleinste und größte Position ermitteln, die du verändern willst. Dann erzeugst du einen LVM und liest den Bereich ein, zum Beispiel so:

```
local vm = core.get_voxel_manip()
local emin, emax = vm:read_from_map(pos1, pos2)
```

Aus Leistungsgründen liest ein LVM fast nie genau den angegebenen Bereich, sondern meist einen größeren. Diesen größeren Bereich geben `emin` und `emax` an (*emerged min pos* und *emerged max pos*). Der LVM lädt den Bereich für dich – egal ob dafür aus dem Arbeitsspeicher, von der Festplatte gelesen oder der Kartengenerator aufgerufen werden muss.

> **LVMs und Kartengenerierung**
>
> Verwende `core.get_voxel_manip()` nicht in der Kartengenerierung, das kann zu Fehlern führen. Verwende dort stattdessen `core.get_mapgen_object("voxelmanip")`.

18.3 Nodes lesen

Um die Node-Typen an bestimmten Positionen zu lesen, verwendest du `get_data()`. Die Methode gibt ein flaches Array zurück, in dem jeder Eintrag für den Typ eines Nodes steht.

```
local data = vm:get_data()
```

`param2` und Lichtwerte erhältst du mit `get_light_data()` und `get_param2_data()`.

Mit `emin` und `emax` berechnest du, wo ein Node in diesen flachen Arrays liegt. Die Hilfsklasse `VoxelArea` übernimmt die Berechnung für dich.

```
local a = VoxelArea:new({
    MinEdge = emin,
    MaxEdge = emax,
})

-- Index des Nodes ermitteln
local idx = a:index(x, y, z)

-- Node lesen
print(data[idx])
```

Dabei fällt auf, dass `data[idx]` eine ganze Zahl ist. Die Engine speichert Nodes aus Leistungsgründen nicht als Strings, sondern als Zahl, die *Content-ID* heißt. Die Content-ID eines Node-Typs erhältst du mit `get_content_id()`:

```
local c_stone = core.get_content_id("default:stone")
```

Dann kannst du prüfen, ob der Node Stein ist:

```
local idx = a:index(x, y, z)
if data[idx] == c_stone then
    print("Ist Stein!")
end
```

Die Content-IDs von Node-Typen können sich während der Ladezeit noch ändern. Ermittle sie deshalb nicht während dieser Zeit.

Die Nodes im Datenarray eines LVM liegen in umgekehrter Koordinatenreihenfolge. Iteriere deshalb immer in der Reihenfolge `z, y, x`:

```
for z = pos1.z, pos2.z do
    for y = pos1.y, pos2.y do
        for x = pos1.x, pos2.x do
            -- vi (voxel index) ist ein üblicher Name
            local vi = a:index(x, y, z)
            if data[vi] == c_stone then
                print("Ist Stein!")
            end
        end
    end
end
```

Der Grund dafür liegt in der Rechnerarchitektur. Aus dem Arbeitsspeicher zu lesen, ist recht teuer, deshalb haben Prozessoren mehrere Cache-Ebenen. Liegen die angeforderten Daten im Cache, sind sie sehr schnell verfügbar. Liegen sie nicht dort, kommt es zu einem *Cache-Miss*, und die Daten werden aus dem Arbeitsspeicher geholt. Dabei werden auch die umliegenden Daten mitgeholt und ersetzen Daten im Cache, weil das Programm sehr wahrscheinlich bald Daten aus der Nähe anfordert. Eine gute Optimierungsregel ist daher, Daten der Reihe nach zu lesen und *Cache-Thrashing* zu vermeiden.

18.4 Nodes schreiben

Zuerst setzt du die neue Content-ID im Datenarray:

```
local c_air = core.get_content_id("air")

for z = pos1.z, pos2.z do
    for y = pos1.y, pos2.y do
        for x = pos1.x, pos2.x do
            local vi = a:index(x, y, z)
            if data[vi] == c_stone then
                data[vi] = c_air
            end
        end
    end
end
```

Bist du mit dem Setzen fertig, lädst du das Datenarray zurück in die Engine:

```
vm:set_data(data)
vm:write_to_map(true)
```

Für Licht- und param2-Daten gibt es die passend benannten Methoden `set_light_data()` und `set_param2_data()`.

`write_to_map()` erwartet einen Wahrheitswert: `true`, wenn die Beleuchtung berechnet werden soll. Übergibst du `false`, musst du die Beleuchtung später mit `core.fix_light` neu berechnen.

18.5 Beispiel

```
local function grass_to_dirt(pos1, pos2)
    local c_dirt = core.get_content_id("default:dirt")
    local c_grass =
        core.get_content_id("default:dirt_with_grass")

    -- Daten in den LVM lesen
    local vm = core.get_voxel_manip()
    local emin, emax = vm:read_from_map(pos1, pos2)
    local a = VoxelArea:new({
        MinEdge = emin,
        MaxEdge = emax,
    })
    local data = vm:get_data()
```

```
    -- Daten ändern
    for z = pos1.z, pos2.z do
        for y = pos1.y, pos2.y do
            for x = pos1.x, pos2.x do
                local vi = a:index(x, y, z)
                if data[vi] == c_grass then
                    data[vi] = c_dirt
                end
            end
        end
    end

    -- Daten schreiben
    vm:set_data(data)
    vm:write_to_map(true)
end
```

18.6 Jetzt bist du dran

- Schreibe `replace_in_area(from, to, pos1, pos2)`, das im angegebenen Bereich jedes Vorkommen von `from` durch `to` ersetzt; `from` und `to` sind Node-Namen.
- Schreibe eine Funktion, die alle Truhen um 90° dreht.
- Schreibe eine Funktion, die mit einem LVM bemoosten Bruchstein auf benachbarten Stein und Bruchstein übergreifen lässt. Breitet sich der Moosbewuchs bei deiner Umsetzung jedes Mal weiter als einen Node aus? Wenn ja, wie könntest du das verhindern?

Teil VI

Spiele

19 Spiele erstellen

Die Stärke von Luanti liegt darin, dass sich Spiele einfach entwickeln lassen, ohne eigene Voxel-Grafik, Voxel-Algorithmen oder ausgefeilten Netzwerkcode schreiben zu müssen.

19.1 Was ist ein Spiel?

Ein Spiel ist eine Sammlung von Mods, die zusammen ein stimmiges Ganzes ergeben. Ein gutes Spiel hat ein durchgängiges Thema und eine Richtung: Es kann zum Beispiel ein klassisches Bau- und Abbauspiel mit harten Überlebenselementen sein oder eine Weltraumsimulation mit Steampunk-Automatisierung.

Spieldesign ist ein komplexes Thema und ein eigenes Fachgebiet. Dieses Buch kann es nur kurz streifen.

19.2 Spielverzeichnis

Aufbau und Ort eines Spiels kommen dir nach der Arbeit mit Mods recht vertraut vor. Spiele liegen in einem Spielverzeichnis, etwa `minetest/games/foo_game`.

```
foo_game
├── game.conf
├── menu
│   ├── header.png
│   ├── background.png
│   └── icon.png
├── minetest.conf
├── mods
│   └── ... Mods
├── README.txt
└── settingtypes.txt
```

Nötig ist nur der Ordner `mods`; `game.conf` und `menu/icon.png` werden aber empfohlen. Eine einfache `game.conf` sieht so aus:

```
title = Foo Game
description = Ein Spiel über Foo.
```

19.3 Kompatibilität zwischen Spielen

19.3.1 API-Kompatibilität

Es lohnt sich, so viel API-Kompatibilität mit Minetest Game zu wahren, wie es sich einrichten lässt. Dann lassen sich Mods viel leichter auf ein anderes Spiel übertragen.

Am besten wahrst du die Kompatibilität mit einem anderen Spiel, indem du die API-Kompatibilität mit gleichnamigen Mods wahrst. Trägt ein Mod denselben Namen wie ein anderer – auch wie ein Mod von Dritten –, sollte er also eine kompatible API haben. Enthält ein Spiel zum Beispiel einen Mod namens `doors`, sollte er dieselbe API haben wie `doors` in Minetest Game.

Die API-Kompatibilität eines Mods setzt sich zusammen aus:

- der Lua-API-Tabelle – alle dokumentierten oder beworbenen Funktionen in der gleichnamigen globalen Tabelle, zum Beispiel `mobs.register_mob`;
- den registrierten Nodes und Items – also dass diese vorhanden sind.

Kleine Brüche sind nicht so schlimm, etwa wenn eine beiläufige Hilfsfunktion fehlt, die eigentlich nur intern verwendet wurde. Größere Brüche bei Kernfunktionen sind dagegen sehr schlecht.

Mit einem riesigen Alleskönner-Mod wie *default* in Minetest Game ist API-Kompatibilität kaum zu wahren. In diesem Fall sollte das Spiel keinen Mod namens `default` enthalten.

API-Kompatibilität betrifft auch andere Mods und Spiele von Dritten. Achte also darauf, dass neue Mods einen eindeutigen Namen haben. Ob ein Mod-Name schon vergeben ist, findest du mit einer Suche auf `https://content.luanti.org/` heraus.

19.3.2 Gruppen und Aliase

Gruppen und Aliase helfen beide, die Kompatibilität zwischen Spielen zu wahren, denn mit ihnen dürfen sich Itemnamen zwischen Spielen unterscheiden. Häufige Nodes wie Stein und Holz sollten Gruppen haben, die das Material angeben. Es ist außerdem sinnvoll, Aliase von `default`-Nodes auf ihre direkten Entsprechungen anzulegen.

19.4 Jetzt bist du dran

- Erstelle ein einfaches Spiel, in dem der Spieler Punkte bekommt, wenn er besondere Blöcke abbaut.

Teil VII

Qualität

20 Übersetzungen

Wenn deine Mods und Spiele übersetzt werden können, haben mehr Menschen Freude daran. Laut Google Play haben 64 % der Luanti-Nutzer unter Android nicht Englisch als Hauptsprache. Für alle Plattformen zusammen erhebt Luanti keine Statistik zu den Sprachen der Nutzer, aber der Anteil nicht englischsprachiger Spieler dürfte hoch sein.

In Luanti schreibst du deine Texte auf Englisch und bildest sie mit Übersetzungsdateien auf andere Sprachen ab. Übersetzt wird auf dem Client jedes Spielers, sodass jeder Spieler eine andere Sprache sehen kann.

20.1 Wie funktioniert die Übersetzung auf dem Client?

20.1.1 Markierter Text

Der Server muss den Clients mitteilen, wie Text übersetzt werden soll. Dazu werden Steuerzeichen in den Text eingefügt, die Luanti sagen, wo und wie übersetzt werden soll. Das nennt man *markierten Text* (*marked up text*); dazu später mehr.

Um Text als übersetzbar zu markieren, verwendest du eine Übersetzerfunktion (`S()`), die du mit `core.get_translator(textdomain)` erhältst:

```
local S = core.get_translator("mymod")

core.register_craftitem("mymod:item", {
    description = S("My Item"),
})
```

Das erste Argument von `get_translator` ist die *Textdomain*, eine Art Namensraum. Statt alle Übersetzungen einer Sprache in einer Datei zu sammeln, werden sie nach Textdomains getrennt, mit einer Datei je Textdomain und Sprache. Die Textdomain sollte dem Mod-Namen entsprechen, um Konflikte zwischen Mods zu vermeiden.

Markierten Text kannst du fast überall verwenden, wo lesbarer Text erwartet wird, etwa in Formspecs, Feldern von Item-Definitionen, `infotext` und vielem

mehr. In Formspecs musst du markierten Text mit `core.formspec_escape` maskieren.

Trifft der Client auf übersetzbaren Text, etwa in `description`, sucht er ihn in der Übersetzungsdatei für die Sprache des Spielers. Findet er keine Übersetzung, verwendet er den englischen Text.

Übersetzbarer markierter Text enthält den englischen Ausgangstext, die Textdomain und alle weiteren Argumente von `S()`. Er ist im Grunde eine Textkodierung des S-Aufrufs mit allen nötigen Informationen.

Eine weitere Art markierten Texts liefert `core.colorize`.

> **Kodierung von markiertem Text**
>
> Das genaue Format von markiertem Text musst du nicht kennen, es hilft aber beim Verständnis.
>
> ```
> "\27(T@mymod)Hello everyone!\27E"
> ```
>
> - `\27` ist das Escape-Zeichen. Es sagt Luanti, dass etwas Besonderes folgt. Es wird sowohl für Übersetzungen als auch für das Einfärben von Text verwendet.
> - `(T@mymod)` besagt, dass der folgende Text mit der Textdomain `mymod` übersetzbar ist.
> - `Hello everyone!` ist der übersetzbare englische Text, wie er der Übersetzerfunktion übergeben wurde.
> - `\27E` ist wieder das Escape-Zeichen, gefolgt von `E`; es markiert das Ende.

20.1.2 Übersetzungsdateien

Übersetzungsdateien sind Mediendateien im Ordner `locale` des jeweiligen Mods.

Luanti unterstützt derzeit zwei Formate: *gettext* und ein eigenes, älteres Format. Seit Luanti 5.10 gibt es gettext, ein verbreitetes und bewährtes Format. Wenn möglich, solltest du gettext verwenden. Einziger Nachteil: Luanti-Clients vor Version 5.10 können damit noch nicht umgehen.

Bei gettext erwartet Luanti Übersetzungsdateien im Format `[textdomain].[sprache].po` im Ordner `locale`. Zum Erstellen und Pflegen kannst du externe Werkzeuge verwenden. Die Luanti-Dokumentation enthält einen Artikel über Weblate[1] (englisch).

[1] `https://docs.luanti.org/for-creators/translation/weblate/`

Lies außerdem die offizielle Luanti-Dokumentation zu Übersetzungen[2] (englisch).

Um Strings automatisch aus deinem Quellcode zu ziehen und `.pot`-Vorlagen zu erzeugen, kannst du `gettext` installieren und diesen Befehl anpassen:

```
xgettext -L lua -kS -kPS:1,2 \
  -kcore.translate:1c,2 -kcore.translate_n:1c,2,3 \
  -d your_textdomain -o locale/your_textdomain.pot *.lua
```

Für jeden verwendeten Funktionsnamen kannst du ein weiteres `-k` angeben. Die Angaben nach dem Funktionsnamen betreffen Pluralformen; Einzelheiten stehen im Handbuch von gettext.

Altes Format

Übersetzungsdateien im alten Format heißen `[textdomain].[sprache].tr`.

Sie beginnen mit einem Kommentar, der die Textdomain angibt; die weiteren Zeilen bilden den englischen Ausgangstext auf die Übersetzung ab.

Zum Beispiel `mymod.de.tr`:

```
# textdomain: mymod
Hello everyone!=Hallo zusammen!
I like grapefruit=Ich mag Grapefruit
```

Übersetzungsdateien solltest du aus dem Quellcode deines Mods oder Spiels erzeugen, etwa mit update_translations[3]. Das Werkzeug sucht in deinem Lua-Code nach `S(` und erstellt automatisch eine Vorlage, die Übersetzer in ihre Sprache übertragen können. Es aktualisiert die Übersetzungsdateien auch, wenn sich dein Quellcode ändert.

Schreib in `S()` immer wörtlichen Text (in `"`) statt einer Variablen; so finden Werkzeuge die zu übersetzenden Texte.

[2] https://api.luanti.org/translations/
[3] https://github.com/minetest-tools/update_translations

20.2 Platzhalter

Oft müssen variable Informationen in einen übersetzbaren Text. Dabei darf der Text nicht einfach verkettet werden, denn sonst können Übersetzer die Reihenfolge der Variablen im Satz nicht ändern. Verwende stattdessen das Platzhaltersystem der Übersetzung:

```
core.register_on_joinplayer(function(player)
    core.chat_send_all(
        S(
            "Everyone, say hi to @1!",
            player:get_player_name()
        )
    )
end)
```

Ein wörtliches `@` schreibst du in einer Übersetzung als `@@`.

Innerhalb eines Satzes solltest du nicht verketten, mehrere Sätze verkettest du aber am besten. Das hilft Übersetzern, weil die Strings kürzer bleiben.

```
S("Hello @1!", player_name)
    .. " "
    .. S("You have @1 new messages.", #msgs),
```

20.3 Gute Praxis und verbreitete Irrtümer

- Verkette keinen Text, sondern verwende Platzhalter. So haben Übersetzer volle Kontrolle über die Reihenfolge.
- Variablen verändern oft den umgebenden Text, etwa durch Geschlecht und Pluralbildung. Das ist schwer zu handhaben und wird deshalb oft übergangen oder mit geschlechtsneutralen Formulierungen umgangen.
- Übersetzungen können viel länger oder kürzer sein als der englische Text. Lass genug Platz.
- Andere Sprachen schreiben Zahlen womöglich anders, etwa mit Komma als Dezimaltrennzeichen: `1.000,23`, `1'000'000,32`.
- Geh nicht davon aus, dass andere Sprachen Groß- und Kleinschreibung genauso verwenden wie Englisch.

20.4 Übersetzung auf dem Server

Manchmal brauchst du die Übersetzung eines Textes auf dem Server, etwa um Text zu sortieren oder zu durchsuchen. Die Sprache eines Spielers erhältst du mit `get_player_information`, und mit `get_translated_string` übersetzt du markierten Text.

```
local list = {
    S("Hello world!"),
    S("Potato"),
}

core.register_chatcommand("find", {
    func = function(name, param)
        local info = core.get_player_information(name)
        local language = info and info.language or "en"

        for _, line in ipairs(list) do
            local trans =
                core.get_translated_string(language, line)
            if trans:find(param, 1, true) then
                return true, line
            end
        end
        return false, S("Nothing found.")
    end,
})
```

20.5 Fazit

Mit der Übersetzungs-API machst du Mods und Spiele mehr Menschen zugänglich. Sie richtig einzusetzen, erfordert aber etwas Sorgfalt.

21 Häufige Fehler

Dieses Kapitel beschreibt häufige Fehler und wie du sie vermeidest.

21.1 Vorsicht beim Speichern von ObjectRefs

Ein ObjectRef (also ein Spieler oder eine Entity) wird ungültig, wenn der Spieler oder die Entity, für die es steht, das Spiel verlässt. Das passiert, wenn der Spieler offline geht oder die Entity entladen oder entfernt wird.

Seit Luanti 5.2 geben die Methoden eines ungültigen ObjectRefs immer `nil` zurück; jeder Aufruf wird im Grunde ignoriert.

Speichere ObjectRefs möglichst nicht. Wenn du es doch tust, prüfe vor der Verwendung, ob das ObjectRef noch gültig ist:

```
-- Funktioniert erst ab Luanti 5.2
if obj:get_pos() then
    -- ist gültig!
end
```

21.2 Formspec-Eingaben nicht vertrauen

Bösartige Clients können Formspecs jederzeit und mit beliebigem Inhalt abschicken.

Der folgende Code hat zum Beispiel eine Sicherheitslücke, durch die Spieler sich selbst Moderatorrechte geben können:

```
local function show_formspec(name)
    if
        not core.check_player_privs(name, { privs = true })
    then
        return false
    end
```

```
    core.show_formspec(
        name,
        "modman:modman",
        [[
        size[3,2]
        field[0,0;3,1;target;Name;]
        button_exit[0,1;3,1;sub;Promote]
    ]]
    )
    return true
end

core.register_on_player_receive_fields(
    function(player, formname, fields)
        -- SCHLECHT! Hier fehlt die Prüfung der Privilegien!

        local privs = core.get_player_privs(fields.target)
        privs.kick = true
        privs.ban = true
        core.set_player_privs(fields.target, privs)
        return true
    end
)
```

Mit einer Prüfung der Privilegien ist die Lücke geschlossen:

```
core.register_on_player_receive_fields(
    function(player, formname, fields)
        local name = player:get_player_name()
        if
            not core.check_player_privs(
                name,
                { privs = true }
            )
        then
            return false
        end

        -- Code
    end
)
```

21.3 ItemStacks nach dem Ändern zurücksetzen

Ist dir aufgefallen, dass es in der API einfach `ItemStack` heißt und nicht `ItemStackRef`, analog zu `InvRef`? Ein `ItemStack` ist nämlich keine Referenz, sondern eine Kopie. Du arbeitest mit einer Kopie der Daten, nicht mit dem Stack im Inventar. Änderst du den Stack, ändert sich der Stack im Inventar also nicht.

Mach das zum Beispiel nicht:

```
local inv = player:get_inventory()
local stack = inv:get_stack("main", 1)
stack:get_meta():set_string("description", "Angebissen")
-- SCHLECHT! Die Änderung geht verloren
```

Sondern so:

```
local inv = player:get_inventory()
local stack = inv:get_stack("main", 1)
stack:get_meta():set_string("description", "Angebissen")
inv:set_stack("main", 1, stack)
-- Richtig! Der ItemStack wird gesetzt
```

Bei Callbacks ist es etwas komplizierter. Änderst du einen übergebenen `ItemStack`, ändert er sich auch für den Aufrufer und alle folgenden Callbacks. In der Engine gespeichert wird er aber nur, wenn der Aufrufer des Callbacks ihn setzt.

```
core.register_on_item_eat(
    function(
        hp_change,
        replace_with_item,
        itemstack,
        user,
        pointed_thing
    )
        itemstack
            :get_meta()
            :set_string("description", "Angebissen")
        -- Fast richtig! Die Daten gehen verloren,
        -- wenn ein anderer Callback das Verhalten abbricht
```

```
    end
)
```

Bricht kein Callback ab, wird der Stack gesetzt und die Beschreibung aktualisiert. Bricht aber ein Callback ab, kann die Änderung verloren gehen.

Besser ist es so:

```
core.register_on_item_eat(
    function(
        hp_change,
        replace_with_item,
        itemstack,
        user,
        pointed_thing
    )
        itemstack
            :get_meta()
            :set_string("description", "Angebissen")
        user:get_inventory():set_stack(
            "main",
            user:get_wield_index(),
            itemstack
        )
        -- Richtig, die Beschreibung wird immer gesetzt!
    end
)
```

Brechen die Callbacks ab oder setzt der Aufrufer den Stack nicht, wird die Änderung trotzdem übernommen. Setzen die Callbacks oder der Aufrufer den Stack, schadet das zusätzliche `set_stack` nicht.

22 Automatische Fehlersuche mit LuaCheck

In diesem Kapitel lernst du, wie du deinen Mod mit dem Werkzeug LuaCheck automatisch nach Fehlern durchsuchst. LuaCheck lässt sich mit deinem Editor verbinden, der dich dann auf Fehler hinweist.

22.1 LuaCheck installieren

22.1.1 Windows

Lade einfach `luacheck.exe` von der Release-Seite auf GitHub[1] herunter.

22.1.2 Linux

Installiere zuerst LuaRocks:

```
sudo apt install luarocks
```

Dann installierst du LuaCheck global:

```
sudo luarocks install luacheck
```

Mit diesem Befehl prüfst du, ob die Installation geklappt hat:

```
luacheck -v
```

[1] `https://github.com/lunarmodules/luacheck/releases`

22.2 LuaCheck ausführen

Beim ersten Durchlauf meldet LuaCheck wahrscheinlich viele Fehler, die keine sind. Das liegt daran, dass es noch nicht eingerichtet ist.

Unter Windows öffnest du PowerShell oder Bash im Hauptordner deines Projekts und führst `path\to\luacheck.exe .` aus.

Unter Linux führst du im Hauptordner deines Projekts `luacheck .` aus.

22.3 LuaCheck einrichten

Lege im Hauptordner deines Projekts eine Datei `.luacheckrc` an. Das kann der Hauptordner deines Spiels, Mod-Pakets oder Mods sein.

Schreib Folgendes hinein:

```
unused_args = false
allow_defined_top = true

globals = {
    "core",
}

read_globals = {
    string = { fields = { "split" } },
    table = { fields = { "copy", "getn" } },

    -- Eingebaut
    "vector",
    "ItemStack",
    "dump",
    "DIR_DELIM",
    "VoxelArea",
    "Settings",

    -- Minetest Game
    "default",
    "sfinv",
    "creative",
}
```

Verwendest du in älterem Code noch den Namensraum `minetest`, nimm ihn zusätzlich in `globals` auf.

Prüfe dann mit einem neuen Durchlauf, ob es funktioniert. Diesmal solltest du deutlich weniger Fehler bekommen. Arbeite dich ab dem ersten Fehler durch: Behebe das Problem im Code oder passe die Konfiguration an, wenn der Code korrekt ist. Hilfe dazu gibt die folgende Liste.

22.3.1 Fehlerbehebung

- **accessing undefined variable foobar** – Soll `foobar` eine globale Variable sein, nimm sie in `read_globals` auf. Andernfalls ergänze die fehlenden `local`.
- **setting non-standard global variable foobar** – Soll `foobar` eine globale Variable sein, nimm sie in `globals` auf und entferne sie gegebenenfalls aus `read_globals`. Andernfalls ergänze die fehlenden `local`.
- **mutating read-only global variable 'foobar'** – Verschiebe `foobar` von `read_globals` nach `globals` oder schreib nicht mehr in `foobar`.

22.4 Im Editor verwenden

Es ist sehr zu empfehlen, für deinen Editor eine Erweiterung zu installieren, die Fehler anzeigt, ohne dass du einen Befehl ausführen musst. Für die meisten Editoren gibt es eine.

- **VSCode** – Strg+P drücken und dann einfügen: `ext install dwenegar.vscode-luacheck`
- **Sublime Text** – Über Package Control installieren: SublimeLinter[2] und SublimeLinter-luacheck[3].

[2] `https://github.com/SublimeLinter/SublimeLinter`
[3] `https://github.com/SublimeLinter/SublimeLinter-luacheck`

23 Sicherheit

Sicherheit ist sehr wichtig, damit dein Mod nicht dazu führt, dass der Serverbetreiber Daten oder die Kontrolle verliert.

23.1 Grundprinzipien

Das wichtigste Prinzip der Sicherheit lautet: **Vertraue niemals dem Benutzer.** Alles, was ein Benutzer übermittelt, ist als bösartig zu behandeln. Prüfe also immer, ob seine Eingaben gültig sind, ob er die nötigen Rechte hat und ob er die Aktion auch sonst ausführen darf (also etwa in Reichweite oder Besitzer ist).

Eine bösartige Aktion muss nicht unbedingt Daten verändern oder zerstören; sie kann auch auf sensible Daten zugreifen, etwa auf Passwort-Hashes oder private Nachrichten. Besonders schlimm ist das, wenn der Server Angaben wie E-Mail-Adressen oder das Alter speichert, wie es manche zur Verifizierung tun.

23.2 Formspecs

23.2.1 Eingaben niemals vertrauen

Jeder Benutzer kann fast jede Formspec mit beliebigen Werten jederzeit abschicken.

Hier echter Code aus einem Mod:

```
core.register_on_player_receive_fields(
    function(player, formname, fields)
        for key, field in pairs(fields) do
            local x, y, z = string.match(
                key,
                "goto_([%d-]+)_([%d-]+)_([%d-]+)"
            )
            if x and y and z then
```

```
                player:set_pos({
                    x = tonumber(x),
                    y = tonumber(y),
                    z = tonumber(z),
                })
                return true
            end
        end
    end
)
```

Siehst du das Problem? Ein bösartiger Benutzer kann eine Formspec mit eigenen Positionswerten abschicken und sich so an jeden beliebigen Ort teleportieren. Mit modifizierten Clients ließe sich das sogar automatisieren – praktisch ein `/teleport` ganz ohne Privileg.

Die Lösung für solche Probleme ist ein Kontext (Kapitel 15), wie im Kapitel über Formspecs gezeigt.

23.2.2 Zeitpunkt der Prüfung ist nicht Zeitpunkt der Verwendung

Jeder Benutzer kann jede Formspec mit beliebigen Werten jederzeit abschicken, außer wo die Engine es verhindert:

- Die Eingabe einer Node-Formspec wird blockiert, wenn der Benutzer zu weit entfernt ist.
- Seit Version 5.0 werden benannte Formspecs blockiert, die noch nicht angezeigt wurden.

Prüfe deshalb im Handler, ob der Benutzer die Bedingungen erfüllt, unter denen die Formspec überhaupt angezeigt wird, und ebenso die Bedingungen für die jeweiligen Aktionen.

Die Sicherheitslücke, die entsteht, wenn Rechte beim Anzeigen der Formspec, aber nicht beim Verarbeiten geprüft werden, heißt *Time of Check is not Time of Use* (TOCTOU).

23.3 (Unsichere) Umgebungen

Luanti erlaubt Mods, eine Umgebung ohne Sandbox anzufordern, die Zugriff auf die vollständige Lua-API gibt.

Siehst du die Sicherheitslücke im folgenden Code?

```
local ie = core.request_insecure_environment()
ie.os.execute(("path/to/prog %d"):format(3))
```

`string.format` ist eine Funktion in der global geteilten Tabelle `string`. Ein bösartiger Mod könnte diese Funktion überschreiben und so etwas an `os.execute` übergeben:

```
string.format = function()
    return "xdg-open 'http://example.com'"
end
```

Der Mod könnte dabei viel Bösartigeres übergeben, als eine Website zu öffnen, etwa einem entfernten Angreifer die Kontrolle über den Rechner geben.

Einige Regeln für unsichere Umgebungen:

- Speichere sie immer in einer lokalen Variablen und übergib sie nie an eine Funktion.
- Stelle sicher, dass du allen Eingaben an unsichere Funktionen vertrauen kannst, um das obige Problem zu vermeiden. Vermeide also global überschreibbare Funktionen.

24 Saubere Architektur

Hat dein Mod eine ordentliche Größe erreicht, wird es immer schwerer, den Code sauber und fehlerfrei zu halten. Bei einer dynamisch typisierten Sprache wie Lua ist das ein besonders großes Problem, denn der Compiler hilft dir kaum dabei, etwa Typen richtig zu verwenden.

Dieses Kapitel behandelt wichtige Konzepte, die deinen Code sauber halten, und verbreitete Entwurfsmuster dafür. Es ist nicht als Vorschrift gemeint, sondern soll dir zeigen, welche Möglichkeiten es gibt. Den einen richtigen Weg, einen Mod zu entwerfen, gibt es nicht; was guter Mod-Entwurf ist, ist sehr subjektiv.

24.1 Kohäsion, Kopplung und Trennung der Belange

Ohne Planung verkommt ein Programmierprojekt mit der Zeit zu Spaghetticode. Spaghetticode zeichnet sich durch fehlende Struktur aus – aller Code ist zusammengewürfelt, ohne klare Grenzen. Am Ende ist das Projekt nicht mehr wartbar und wird aufgegeben.

Das Gegenteil ist, dein Projekt als Sammlung kleinerer, zusammenwirkender Programme oder Codebereiche zu entwerfen.

> Inside every large program, there is a small program trying to get out.
>
> In jedem großen Programm steckt ein kleines, das herauswill.
>
> — C. A. R. Hoare

Dabei solltest du eine *Trennung der Belange* (*separation of concerns*) erreichen: Jeder Bereich ist klar abgegrenzt und kümmert sich um eine eigene Aufgabe.

Diese Programme oder Bereiche sollten zwei Eigenschaften haben:

- **Hohe Kohäsion** – was in einem Bereich steht, gehört eng zusammen.
- **Geringe Kopplung** – halte die Abhängigkeiten zwischen Bereichen so gering wie möglich und verlass dich nicht auf interne Umsetzungen. Geringe Kopplung ist sehr sinnvoll, denn dann lassen sich die APIs einzelner Bereiche leichter ändern.

Das gilt sowohl für die Beziehungen zwischen Mods als auch für die Beziehungen zwischen Bereichen innerhalb eines Mods.

24.2 Beobachter

Ein einfacher Weg, Codebereiche voneinander zu trennen, ist das Beobachter-Muster (*Observer*).

Nehmen wir als Beispiel eine Errungenschaft, die freigeschaltet wird, wenn ein Spieler zum ersten Mal ein seltenes Tier erlegt. Der naive Ansatz wäre, Errungenschaftscode in die Funktion für den Tod eines Mobs zu schreiben, die den Mob-Namen prüft und bei Übereinstimmung die Errungenschaft freischaltet. Das ist aber keine gute Idee, denn damit ist der Mob-Mod an den Errungenschaftscode gekoppelt. Machst du so weiter – fügst etwa auch noch Erfahrungspunkte in den Code für den Tod eines Mobs ein –, entsteht schnell ein Gewirr von Abhängigkeiten.

Hier kommt das Beobachter-Muster ins Spiel. Statt dass sich der Mod `mymobs` um Errungenschaften kümmert, bietet er anderen Codebereichen die Möglichkeit, ihr Interesse an einem Ereignis anzumelden und Daten dazu zu erhalten.

```
mymobs.registered_on_death = {}
function mymobs.register_on_death(func)
    table.insert(mymobs.registered_on_death, func)
end

-- im Code für den Tod eines Mobs
for i = 1, #mymobs.registered_on_death do
    mymobs.registered_on_death[i](entity, reason)
end
```

Der andere Code meldet dann sein Interesse an:

```
mymobs.register_on_death(function(mob, reason)
    if
        reason.type == "punch"
        and reason.object
        and reason.object:is_player()
    then
        awards.notify_mob_kill(reason.object, mob.name)
    end
end)
```

Vielleicht denkst du jetzt: Moment, das kommt mir bekannt vor. Richtig! Die Luanti-API baut stark auf dem Beobachter-Muster auf, damit sich die Engine nicht darum kümmern muss, wer auf etwas wartet.

24.3 Model-View-Controller

Im nächsten Kapitel geht es darum, deinen Code automatisch zu testen. Eine Schwierigkeit dabei ist, die Logik (Berechnungen, was getan werden soll) möglichst weit von API-Aufrufen (`core.*`, andere Mods) zu trennen.

Ein Weg dahin ist, dir Folgendes zu überlegen:

- Welche **Daten** hast du?
- Welche **Aktionen** kannst du mit diesen Daten ausführen?
- Wie lösen **Ereignisse** (etwa Formspecs oder Schläge) diese Aktionen aus, und wie bewirken die Aktionen etwas in der Engine?

Nehmen wir einen Mod zum Schutz von Grundstücken. Die Daten sind die Grundstücke und die zugehörigen Metadaten. Die Aktionen sind `create`, `edit` und `delete`. Ausgelöst werden sie durch Chatbefehle und Formspec-Eingaben. Diese drei Bereiche lassen sich meist gut voneinander trennen.

In deinen Tests kannst du dann sicherstellen, dass eine ausgelöste Aktion das Richtige mit den Daten macht. Dass ein Ereignis eine Aktion auslöst, musst du nicht testen – dafür bräuchtest du die Luanti-API, und dieser Codebereich sollte ohnehin so klein wie möglich sein.

Die Datenschicht schreibst du in reinem Lua. „Rein“ heißt hier, dass die Funktionen auch außerhalb von Luanti laufen könnten – keine Funktion der Engine wird aufgerufen.

```
-- Daten
function land.create(name, area_name)
    land.lands[area_name] = {
        name = area_name,
        owner = name,
        -- weitere Angaben
    }
end

function land.get_by_name(area_name)
    return land.lands[area_name]
end
```

Auch die Aktionen sollten rein sein, wobei Aufrufe anderer Funktionen hier eher vertretbar sind als oben.

```
-- Controller
function land.handle_create_submit(name, area_name)
    -- Verarbeitung (etwa Überschneidungen,
    -- Kontingente und Rechte prüfen)

    land.create(name, area_name)
end

function land.handle_creation_request(name)
    -- Ein schlechtes Beispiel, wie später erklärt
    land.show_create_formspec(name)
end
```

Die Ereignis-Handler müssen mit der Luanti-API arbeiten. Halte die Berechnungen hier so gering wie möglich, denn diesen Bereich kannst du nur schwer testen.

```
-- View
function land.show_create_formspec(name)
    -- Hier gibt es keine komplexen Berechnungen!
    core.show_formspec(
        name,
        "land:create",
        [[
        size[4,3]
        label[1,0;Das ist ein Beispiel]
        field[0,1;3,1;area_name;]
        button_exit[0,2;1,1;exit;Beenden]
    ]]
    )
end

core.register_chatcommand("land", {
    privs = { land = true },
    func = function(name)
        land.handle_creation_request(name)
    end,
})

core.register_on_player_receive_fields(
    function(player, formname, fields)
```

```
        if formname ~= "land:create" then
            return
        end
        land.handle_create_submit(
            player:get_player_name(),
            fields.area_name
        )
    end
)
```

Das ist das Model-View-Controller-Muster. Das *Model* ist eine Sammlung von Daten mit möglichst wenigen Funktionen. Die *View* ist eine Sammlung von Funktionen, die auf Ereignisse warten und sie an den Controller weitergeben; außerdem nimmt sie Aufrufe des Controllers entgegen, um etwas mit der Luanti-API zu tun. Im *Controller* fallen die Entscheidungen, und hier finden die meisten Berechnungen statt.

Der Controller sollte nichts über die Luanti-API wissen – beachte, dass es dort keine Luanti-Aufrufe gibt und auch keine View-Funktionen, die ihnen ähneln. Eine Funktion wie `view.hud_add(player, def)` sollte es *nicht* geben. Stattdessen legt die View Aktionen fest, die der Controller bei ihr anstoßen kann, etwa `view.add_hud(info)`, wobei `info` ein Wert oder eine Tabelle ist, die nichts mit der Luanti-API zu tun hat.

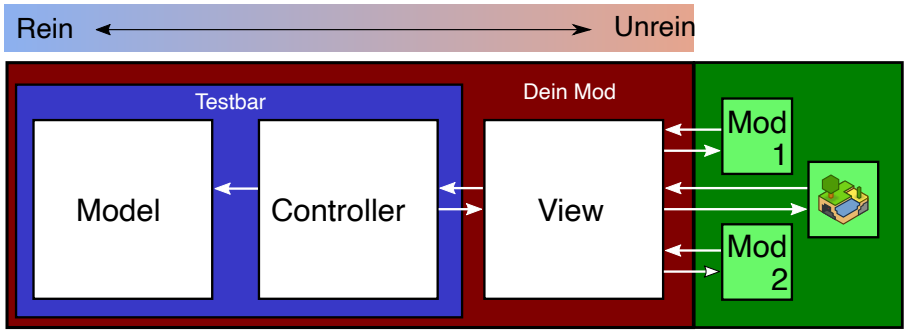


Abbildung 24.1: Model, View und Controller und ihre Verbindungen

Wichtig ist, dass jeder Bereich nur mit seinen direkten Nachbarn kommuniziert, wie im Diagramm gezeigt. So musst du weniger ändern, wenn du das Innere oder die Schnittstelle eines Bereichs änderst. Um die Formspec zu ändern, musst du zum Beispiel nur die View anpassen. Um die API der View zu ändern, musst du nur View und Controller anpassen, das Model aber gar nicht.

In der Praxis wird dieser Entwurf selten verwendet, weil er die Komplexität erhöht und den meisten Mods nicht viel bringt. Häufiger sieht man einen weniger formalen und strengen Entwurf – Varianten von API-View.

24.3.1 API-View

In einer idealen Welt wären die drei Bereiche oben sauber getrennt, und alle Ereignisse gingen erst durch den Controller, bevor sie zur View zurückkehren. Die Welt ist aber nicht ideal. Ein guter Kompromiss ist, den Mod in zwei Teile zu gliedern:

- **API** – Das waren oben Model und Controller. Hier sollte `core.` nirgends vorkommen.
- **View** – Das war auch oben die View. Am besten teilst du sie in eigene Dateien für jede Art von Ereignis auf.

rubenwardys crafting-Mod[1] folgt ungefähr diesem Entwurf. `api.lua` besteht fast nur aus reinen Lua-Funktionen für die Datenhaltung und Berechnungen im Stil eines Controllers. `gui.lua` ist die View für Formspecs und deren Eingaben, und `async_crafter.lua` ist View und Controller für eine Node-Formspec und Node-Timer.

So aufgeteilt, lässt sich der API-Teil sehr einfach testen, weil er keine Luanti-APIs verwendet – wie im nächsten Kapitel (Kapitel 25) gezeigt und im crafting-Mod zu sehen.

24.4 Fazit

Guter Code-Entwurf ist subjektiv und hängt stark vom jeweiligen Projekt ab. Als Faustregel gilt: Halte die Kohäsion hoch und die Kopplung gering. Anders gesagt: Halte zusammen, was zusammengehört, trenne, was nicht zusammengehört, und halte die Abhängigkeiten einfach.

Sehr zu empfehlen ist das Buch Game Programming Patterns[2] (englisch). Es ist kostenlos online lesbar[3] und geht viel ausführlicher auf verbreitete Programmiermuster für Spiele ein.

[1] `https://github.com/rubenwardy/crafting`
[2] `http://gameprogrammingpatterns.com/`
[3] `http://gameprogrammingpatterns.com/contents.html`

25 Automatische Unit-Tests

Unit-Tests sind ein unverzichtbares Werkzeug, um nachzuweisen und dich zu vergewissern, dass dein Code korrekt ist. Dieses Kapitel zeigt, wie du mit *Busted* Tests für Luanti-Mods und -Spiele schreibst. Funktionen zu testen, die Luanti-Funktionen aufrufen, ist ziemlich schwierig. Zum Glück haben wir im vorigen Kapitel (Kapitel 24) besprochen, wie du deinen Code so aufbaust, dass sich das vermeiden lässt.

25.1 Busted installieren

Zuerst installierst du LuaRocks:

- Windows: Folge der Installationsanleitung im LuaRocks-Wiki[1] (englisch).
- Debian/Ubuntu: `sudo apt install luarocks`

Dann installierst du Busted global:

```
sudo luarocks install busted
```

Zum Schluss prüfst du, ob es installiert ist:

```
busted --version
```

25.2 Dein erster Test

Busted ist das führende Framework für Unit-Tests in Lua. Es sucht nach Lua-Dateien, deren Name auf `_spec` endet, und führt sie in einer eigenständigen Lua-Umgebung aus.

[1] `https://github.com/luarocks/luarocks/wiki/Installation-instructions-for-Windows`

```
mymod/
├── init.lua
├── api.lua
└── tests
    └── api_spec.lua
```

25.2.1 init.lua

```
mymod = {}

dofile(core.get_modpath("mymod") .. "/api.lua")
```

25.2.2 api.lua

```
function mymod.add(x, y)
    return x + y
end
```

25.2.3 tests/api_spec.lua

```
-- Benötigte Dateien im Mod-Verzeichnis suchen
package.path = "./?.lua;" .. package.path

-- Globale Tabelle mymod anlegen, in die die API schreibt
_G.mymod = {}
-- api.lua ausführen
require("api")

-- Tests
describe("add", function()
    it("addiert", function()
        assert.equals(2, mymod.add(1, 1))
    end)

    it("kann mit negativen Zahlen umgehen", function()
        assert.equals(0, mymod.add(-1, 1))
```

```
        assert.equals(-2, mymod.add(-1, -1))
    end)
end)
```

Jetzt kannst du die Tests ausführen, indem du im Verzeichnis des Mods ein Terminal öffnest und `busted .` aufrufst.

Wichtig ist, dass die API-Datei die Tabelle nicht selbst anlegt, denn globale Variablen funktionieren in Busted anders. Jede Variable, die in Luanti global wäre, ist in Busted lokal zur Datei. Das wäre auch für Luanti der bessere Weg gewesen, aber dafür ist es jetzt zu spät.

Außerdem sollten die Dateien, die du testest, möglichst keine Funktionen aufrufen, die nicht in ihnen selbst stehen. Meist schreibst du Tests jeweils nur für eine einzelne Datei.

25.3 Mocking: Externe Funktionen ersetzen

Mocking heißt, Funktionen zu ersetzen, von denen das Getestete abhängt. Das hat zwei Zwecke: Zum einen steht die Funktion in der Testumgebung womöglich nicht zur Verfügung, zum anderen möchtest du vielleicht Aufrufe der Funktion und ihre Argumente mitschneiden.

Hast du die Ratschläge aus dem Kapitel Saubere Architektur (Kapitel 24) befolgt, hast du schon eine recht saubere Datei zum Testen. Dinge außerhalb deines Bereichs musst du aber trotzdem ersetzen – etwa die View, wenn du Controller oder API testest. Hast du die Ratschläge nicht befolgt, wird es etwas schwieriger, weil du womöglich die Luanti-API ersetzen musst.

```
-- Wie oben eine Tabelle anlegen
_G.core = {}

-- Die Mock-Funktion definieren
local chat_send_player_calls = {}
function core.chat_send_player(name, message)
    table.insert(
        chat_send_player_calls,
        { name = name, message = message }
    )
end

-- Tests
```

```
describe("list_areas", function()
    it("gibt eine Zeile je Gebiet aus", function()
        chat_send_player_calls = {} -- Tabelle zurücksetzen

        mymod.list_areas_to_chat(
            "singleplayer",
            "singleplayer"
        )

        assert.equals(2, #chat_send_player_calls)
    end)

    it("sendet an den richtigen Spieler", function()
        chat_send_player_calls = {} -- Tabelle zurücksetzen

        mymod.list_areas_to_chat(
            "singleplayer",
            "singleplayer"
        )

        for _, call in pairs(chat_send_player_calls) do
            assert.equals("singleplayer", call.name)
        end
    end)

    -- Die beiden Tests oben sind eigentlich überflüssig,
    -- denn dieser prüft beides
    it("gibt das Richtige aus", function()
        chat_send_player_calls = {} -- Tabelle zurücksetzen

        mymod.list_areas_to_chat(
            "singleplayer",
            "singleplayer"
        )

        local expected = {
            {
                name = "singleplayer",
                message = "Rathaus (2,43,63)",
            },
            {
                name = "singleplayer",
                message = "Flughafen (43,45,63)",
            },
        }
```

```
        assert.same(expected, chat_send_player_calls)
    end)
end)
```

25.4 Fazit

Gut eingesetzt, verbessern Unit-Tests die Qualität und Zuverlässigkeit deines Projekts erheblich. Sie verlangen aber, dass du deinen Code anders aufbaust als üblich.

Ein Beispiel für einen Mod mit vielen Unit-Tests ist crafting von rubenwardy[2].

[2] https://github.com/rubenwardy/crafting

26 Einen Mod veröffentlichen

Wenn du einen Mod veröffentlichst, können andere ihn nutzen. Ein veröffentlichter Mod wird womöglich in Einzelspielerwelten oder auf Servern verwendet, auch auf öffentlichen.

26.1 Eine Lizenz wählen

Du musst für deinen Mod eine Lizenz angeben. Sie sagt anderen, auf welche Weise sie dein Werk nutzen dürfen. Hat dein Mod keine Lizenz, wissen andere nicht, ob sie ihn verändern, weitergeben oder auf einem öffentlichen Server einsetzen dürfen.

Code und Grafiken stellen unterschiedliche Anforderungen an ihre Lizenzen. Creative-Commons-Lizenzen solltest du zum Beispiel nicht für Quellcode verwenden; für künstlerische Werke wie Bilder, Texte und Meshes können sie aber gut geeignet sein.

Du darfst jede Lizenz verwenden. Mods, die keine Bearbeitungen erlauben, sind im offiziellen Luanti-Forum allerdings nicht zugelassen. (Damit ein Mod im Forum erlaubt ist, müssen andere Entwickler ihn verändern und die veränderte Fassung veröffentlichen dürfen.)

Beachte, dass **Gemeinfreiheit (*public domain*) keine gültige Lizenz ist**, weil sie in verschiedenen Ländern unterschiedlich definiert ist.

Von der WTFPL wird dringend abgeraten[1] (englisch); manche verzichten auf deinen Mod, wenn er unter dieser Lizenz steht.

26.1.1 LGPL und CC BY-SA

Diese Kombination ist in der Luanti-Community verbreitet; auch Luanti und Minetest Game verwenden sie.

Du stellst deinen Code unter die LGPL 2.1 und deine Grafiken unter CC BY-SA.

Das bedeutet:

[1] `https://content.luanti.org/help/wtfpl/`

- Jeder darf veränderte und unveränderte Fassungen bearbeiten, weitergeben und verkaufen.
- Wer deinen Mod verändert, muss seine Fassung unter dieselbe Lizenz stellen.
- Dein Urheberrechtsvermerk muss erhalten bleiben.

26.1.2 CC0

Diese Lizenz eignet sich für Code und Grafiken und erlaubt jedem, mit deinem Werk zu tun, was er will. Andere dürfen es also verändern, weitergeben, verkaufen und auch ohne Namensnennung verwenden.

26.1.3 MIT

Eine verbreitete Lizenz für Code. Die einzige Bedingung für Nutzer deines Codes ist, dass sie in allen Kopien des Codes oder wesentlicher Teile davon denselben Urheberrechtsvermerk und dieselbe Lizenz beifügen.

26.2 Verpacken

Einige Dateien solltest du deinem Mod oder Spiel vor der Veröffentlichung beilegen.

26.2.1 README.txt

Die README sollte angeben:

- was der Mod oder das Spiel tut und wie man ihn benutzt,
- unter welcher Lizenz er steht,
- optional:
  - wo man Probleme melden oder Hilfe bekommen kann,
  - Danksagungen.

26.2.2 mod.conf / game.conf

Füge unbedingt einen Schlüssel `description` hinzu, der erklärt, was dein Mod oder Spiel tut. Fasse dich kurz, ohne vage zu werden. Die Beschreibung sollte kurz sein, weil sie im Inhaltsinstaller angezeigt wird, wo wenig Platz ist.

Gutes Beispiel:

```
description = Fügt Suppen, Kuchen, Gebäck und Säfte hinzu.
```

Vermeide so etwas:

```
description = Der Essens-Mod für Luanti.
```

Das ist zu vage.

26.2.3 screenshot.png

Screenshots sollten das Seitenverhältnis 3:2 haben (3 Pixel Breite auf 2 Pixel Höhe) und mindestens 300 × 200 Pixel groß sein.

Der Screenshot wird in Luanti als Vorschaubild für den Inhalt angezeigt.

26.3 Hochladen

Damit Interessierte deinen Mod herunterladen können, musst du ihn an einem öffentlich zugänglichen Ort hochladen. Dafür gibt es mehrere Wege. Nimm den, der für dich am besten passt, solange er diese Anforderungen erfüllt (und alle weiteren, die Forenmoderatoren vielleicht noch hinzufügen):

- **Beständig** – Die Website sollte nicht ohne Vorwarnung abgeschaltet werden.
- **Direkter Link** – Ein Klick auf einen Link sollte die Datei herunterladen, ohne dass man erst eine andere Seite ansehen muss.
- **Virenfrei** – Unseriöse Upload-Dienste enthalten womöglich unsichere Werbung.

Bei ContentDB kannst du ZIP-Dateien hochladen; ContentDB erfüllt diese Anforderungen.

26.3.1 Versionsverwaltung

Ein Versionsverwaltungssystem (*Version Control System*, VCS) verwaltet Änderungen an Software und macht es oft leichter, sie zu verbreiten und Beiträge anderer anzunehmen.

Die meisten Luanti-Modder verwenden Git und eine Plattform wie GitHub, um ihren Code zu verbreiten.

Git kann anfangs schwierig sein. Hilfe findest du hier:

- Pro Git[2] – kostenlos online lesbar, auch auf Deutsch.

26.4 Auf ContentDB veröffentlichen

ContentDB ist der offizielle Ort, um Inhalte wie Mods, Spiele und Texturpakete zu finden und zu verbreiten. Benutzer finden Inhalte über die Website oder laden und installieren sie direkt über das Hauptmenü von Luanti.

Registriere dich bei ContentDB[3] und füge deinen Inhalt hinzu. Lies unbedingt die Hinweise im Hilfebereich.

26.5 Forenthema

Du kannst auch ein Forenthema anlegen, in dem Benutzer über dein Werk sprechen können.

Themen für Mods gehören ins Forum „WIP Mods“[4] (*Work in Progress*), Themen für Spiele ins Forum „WIP Games“[5]. Wenn dein Mod für dich nicht mehr in Arbeit ist, kannst du beantragen, dass das Thema verschoben wird[6], und zwar nach „Mod Releases“.

Das Forenthema sollte ähnliche Inhalte wie die README haben, aber werbender formuliert sein und einen Link zum Herunterladen enthalten. Wenn möglich, zeig Screenshots deines Mods in Aktion.

Der Titel des Themas muss eines dieser Formate haben:

- [Mod] Titel des Mods [modname]

[2] `https://git-scm.com/book/de/v2`
[3] `https://content.luanti.org`
[4] `https://forum.luanti.org/viewforum.php?f=9`
[5] `https://forum.luanti.org/viewforum.php?f=50`
[6] `https://forum.luanti.org/viewtopic.php?f=11&t=10418`

- [Mod] Titel des Mods [Versionsnummer] [modname]

Zum Beispiel:

- [Mod] More Blox [0.1] [moreblox]

27 Weiterlesen

Wenn du dieses Buch durchgelesen hast, lohnt sich ein Blick auf die folgenden Quellen.

27.1 Modding für Luanti

- Die Lua-API-Referenz von Luanti: mehrseitige Fassung[1] und einseitige Fassung[2] (englisch).
- Schau dir bestehende Mods[3] an.

27.2 Programmieren in Lua

- Programming in Lua (PIL)[4] (englisch).

27.3 3D-Modellierung

- Blender 3D: Noob to Pro[5] (englisch).
- Blender mit Luanti verwenden[6] (englisch).

[1] https://api.luanti.org/
[2] https://github.com/luanti-org/luanti/blob/master/doc/lua_api.md
[3] https://forum.luanti.org/viewforum.php?f=11
[4] http://www.lua.org/pil/
[5] https://en.wikibooks.org/wiki/Blender_3D:_Noob_to_Pro
[6] https://docs.luanti.org/Using_Blender/

Stichwortverzeichnis

A

Abbautyp, 21, 78
ABM, 63
Abnutzung, 22, 52
Alias, 16, 45, 131

B

Benutzerdatenverzeichnis, 4
Biom, 115
Busted, 159

C

Callback, 25, 48, 61, 75, 91, 99, 121, 143
    `after_dig_node`, 29
    `after_place_node`, 28
    `after_use`, 26
    `on_activate`, 77
    `on_construct`, 28
    `on_dig`, 29
    `on_drop`, 26
    `on_joinplayer`, 74
    `on_place`, 26
    `on_punch`, 25, 72
    `on_receive_fields`, 102
    `on_rightclick`, 26
    `on_secondary_use`, 26
    `on_timer`, 63
    `on_use`, 25
Chatbefehl, 77, 87, 100, 155
Content-ID, 122
ContentDB, 167
`core`
    `core.colorize`, 68, 136
    `core.delete_area`, 62
    `core.dig_node`, 27
    `core.emerge_area`, 61
    `core.find_node_near`, 58
    `core.fix_light`, 124
    `core.formspec_escape`, 136
    `core.get_mapgen_object`, 122
    `core.get_node`, 65
    `core.get_node_or_nil`, 58
    `core.get_player_by_name`, 88
    `core.get_translator`, 135
    `core.get_voxel_manip`, 122
    `core.item_drop`, 26
    `core.item_eat`, 26
    `core.item_place`, 26
    `core.node_dig`, 27
    `core.parse_json`, 69
    `core.place_node`, 27
    `core.register_node`, 33
    `core.register_on_player_receive_fields`, 100
    `core.register_on_punchnode`, 25
    `core.set_node`, 72
    `core.show_formspec`, 100
    `core.write_json`, 69
Craftingrezept, 15
Craftitem, 15, 25, 31

D

Dekoration, 115
`dofile`, 11

E

Entity, 23, 28, 57, 73, 141

F

Formspec, 95, 105, 135, 141, 149, 155

G

`game.conf`, 130
Gruppe, 21, 58, 78, 90, 131

H

HUD, 105

I

`init.lua`, 4, 9
Inventar, 15, 25, 45, 60, 68, 143
`ItemStack`, 28, 45, 67, 143

K

Kartengenerierung, 18, 115, 121

L

Lebenspunkte, 26, 77
Lizenz, 165
Luacheck, 145
LVM, 62, 121

M

Mapblock, 57, 63, 77
Mesh, 42, 165
Metadaten, 5, 45, 60, 63, 67, 96, 155
Mocking, 161
Mod-Paket, 6
Mod-Speicher, 69
`mod.conf`, 4

N

Namensraum, 10, 135, 147
Node-Timer, 63, 158
Nodebox, 39

O

`ObjectRef`, 75, 141

P

Privileg, 77, 83, 88, 100, 142, 150

S

Schadensgruppe, 23, 78

T

Textdomain, 135
Textur, 3, 17, 31, 36

U

Unit-Test, 159

V

Voronoi-Diagramm, 116
`VoxelArea`, 122

Z

Zeichnungstyp, 33